Module 1 -3Basics of Graph Theory
T3-Chapter 2 Foundations of Graphs

15 July 2026 19:35

2 Foundations of Graphs

2.1 Introduction

Graphs are essential representations for real-world data across many different domains. They are primarily used to describe **pairwise relations** between entities.

Key Applications by Domain

- **Social Science:** Used to represent relationships (such as friendships) between individuals.
- **Chemistry:** Chemical compounds are represented as graphs where **atoms are nodes** and **chemical bonds are edges**.
- **Linguistics:** Used to capture the syntactic and compositional structures of sentences.
  - *Parsing trees* represent the syntactic structure of a sentence based on a context-free grammar.
  - *Abstract Meaning Representation (AMR)* encodes the meaning of a sentence as a rooted, directed graph.
- **Other Fields:** Biology, physics, and general network sciences.

Core Concepts Covered in this Chapter

1. **Basic Concepts & Matrix Representations:** Introduction to fundamental properties and algebraic representations such as the **adjacency matrix** and **Laplacian matrix**.
2. **Attributed Graphs:** Modeling attributes as functions or signals on the graph, which forms the basis for **graph Fourier analysis** and **graph signal processing**.
3. **Complex Graphs:** Representations used to capture highly complicated, multi-relational real-world systems.
4. **Downstream Computational Tasks:** Primary machine learning and deep learning tasks executed on graphs.

2.2 Graph Representations

This section focuses on the formal representation of **simple, unweighted, undirected graphs**.

Definition 2.1 (Graph)

A graph is mathematically denoted as:

$$\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$$

Where:

- $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the set of $N = |\mathcal{V}|$ **nodes** (or vertices), representing the entities. The size of the graph is defined by N.
- $\mathcal{E} = \{e_1, \ldots, e_M\}$ is the set of $M = |\mathcal{E}|$ **edges**, describing the connections between nodes.

Key Terminology & Edge Properties:

- An edge $e \in \mathcal{E}$ connecting two nodes v_e^1 and v_e^2 is denoted as (v_e^1, v_e^2).
- **Directed Graphs:** The edge has a orientation, flowing from the source node v_e^1 to the target node v_e^2.
- **Undirected Graphs:** The order of the nodes does not matter, meaning $e = (v_e^1, v_e^2) = (v_e^2, v_e^1)$. *(Unless specified otherwise, the focus remains on undirected graphs).*
- **Incidency:** The nodes v_e^1 and v_e^2 are said to be **incident** to the edge e.
- **Adjacency:** A node v_i is **adjacent** to another node v_j if and only if there exists an edge connecting them.

Definition 2.2 (Adjacency Matrix)

For a given graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, the connectivity between nodes is mathematically expressed using an **Adjacency Matrix** $\mathbf{A} \in \{0, 1\}^{N \times N}$.

The entry $\mathbf{A}_{i,j}$ at the i-th row and j-th column is defined as:

$$\mathbf{A}_{i,j} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j \\ 0 & \text{otherwise} \end{cases}$$

Symmetry in Undirected Graphs:
In an undirected graph, if v_i is adjacent to v_j, then v_j is also adjacent to v_i. Thus, $\mathbf{A}_{i,j} = \mathbf{A}_{j,i}$ for all i, j, making the adjacency matrix $\mathbf{A}$ **symmetric**.

Example 2.3 (Illustrative Example)

Consider an undirected graph containing 5 nodes and 6 edges:

- **Node Set:** $\mathcal{V} = \{v_1, v_2, v_3, v_4, v_5\}$
- **Edge Set:** $\mathcal{E} = \{e_1, e_2, e_3, e_4, e_5, e_6\}$

The connectivity details are as follows:

- v_1 is connected to v_2, v_4, and v_5
- v_2 is connected to v_1 and v_3
- v_3 is connected to v_2 and v_5
- v_4 is connected to v_1 and v_5
- v_5 is connected to v_1, v_3, and v_4

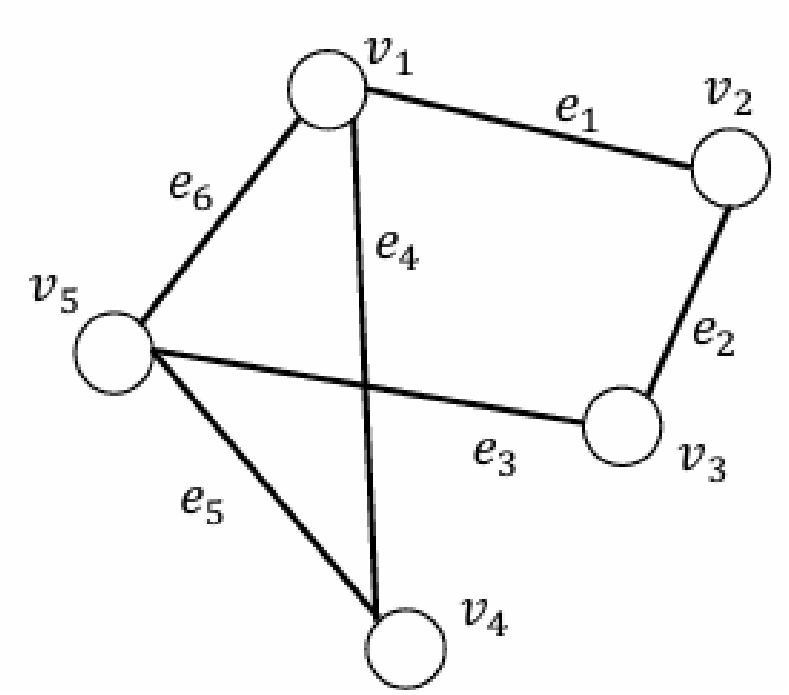


Figure 2.1 A graph with 5 nodes and 6 edges

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

2.3 Properties and Measures

Graphs can have varied structures and properties. In this section, we discuss basic properties and measures for graphs.

2.3.1 Degree

The degree of a node v in a graph $\mathcal{G}$ indicates the number of times that a node is adjacent to other nodes. We have the following formal definition.

Definition 2.4 (Degree)

In a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, the degree of a node $v_i \in \mathcal{V}$ is the number of nodes that are adjacent to v_i.

$$d(v_i) = \sum_{v_j \in \mathcal{V}} 1_{\mathcal{E}}(\{v_i, v_j\}),$$

where $1_{\mathcal{E}}(\cdot)$ is an indicator function:

$$1_{\mathcal{E}}(\{v_i, v_j\}) = \begin{cases} 1 & \text{if } (v_i, v_j) \in \mathcal{E}, \\ 0 & \text{if } (v_i, v_j) \notin \mathcal{E}. \end{cases}$$

The degree of a node v_i in $\mathcal{G}$ can also be calculated from its adjacency matrix. More specifically, for a node v_i, its degree can be computed as:

$$d(v_i) = \sum_{j=1}^{N} \mathbf{A}_{i,j}.$$

Example 2.5

In the graph shown in Figure 2.1, the degree of node v_5 is 3, as it is adjacent to 3 other nodes (i.e., v_1, v_3 and v_4). Furthermore, the 5-th row of the adjacency matrix has 3 non-zero elements, which also indicates that the degree of v_5 is 3.

Definition 2.6 (Neighbors)

For a node v_i in a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, the set of its neighbors $\mathcal{N}(v_i)$ consists of all nodes that are adjacent to v_i.

Note that for a node v_i, the number of nodes in $\mathcal{N}(v_i)$ equals to its degree, i.e., $d(v_i) = |\mathcal{N}(v_i)|$.

Theorem 2.7

For a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, its total degree, i.e., the summation of the degree of all nodes, is twice the number of edges in the graph.

$$\sum_{v_i \in \mathcal{V}} d(v_i) = 2 \cdot |\mathcal{E}|.$$

- **Proof:**

$$\sum_{v_i \in \mathcal{V}} d(v_i) = \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{V}} 1_{\mathcal{E}}(\{v_i, v_j\})$$

$$= \sum_{\{v_i, v_j\} \in \mathcal{E}} 2 \cdot 1_{\mathcal{E}}(\{v_i, v_j\})$$

$$= 2 \cdot \sum_{\{v_i, v_j\} \in \mathcal{E}} 1_{\mathcal{E}}(\{v_i, v_j\})$$

$$= 2 \cdot |\mathcal{E}| \quad \square$$

Corollary 2.8

The number of non-zero elements in the adjacency matrix is also twice the number of the edges.

- **Proof:** The proof follows Theorem 2.7 by using Eq. (2.1). □

Example 2.9
For the graph shown in Figure 2.1, the number of edges is 6. The total degree is 12 and the number of non-zero elements in its adjacent matrix is also 12.

2.3.2 Connectivity

Connectivity is a fundamental property of graphs. To understand connectivity, several foundational concepts such as walks, trails, and paths must first be defined.

Core Definitions

- **Walk (Definition 2.10):** An alternating sequence of nodes and edges, starting and ending with a node, where each edge is incident with the nodes immediately preceding and following it.
  - A walk starting at node u and ending at node v is denoted as a u-v **walk**.
  - The **length of a walk** is the total number of edges it contains.
  - u-v walks are not unique; multiple walks of varying lengths can exist between the same two nodes.
- **Trail (Definition 2.11):** A walk in which all **edges are distinct** (no edge is repeated).
- **Path (Definition 2.12):** A walk in which all **nodes are distinct** (no node is repeated).

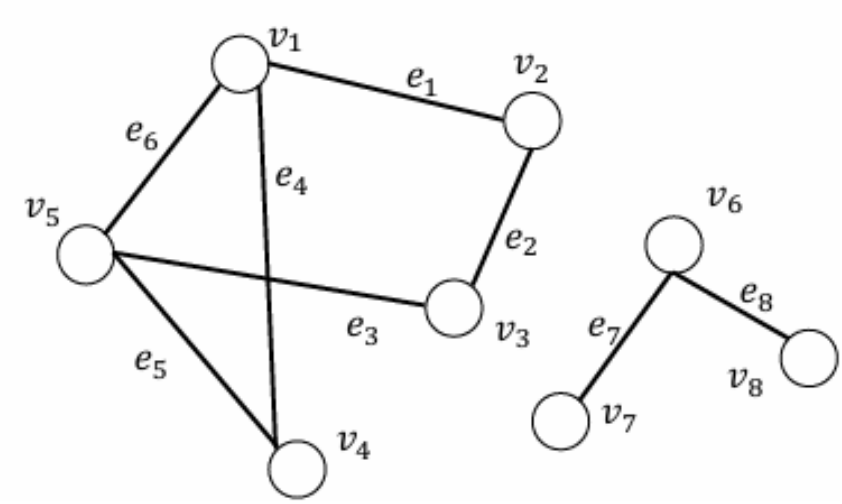


Figure 2.2 A graph with two connected components

Matrix Representation of Walks

Theorem 2.14: For a graph $\mathcal{G} = \{\mathcal{E}, \mathcal{V}\}$ with an adjacency matrix $\mathbf{A}$, let $\mathbf{A}^n$ denote the n-th power of the adjacency matrix. The (i, j)-th element of the matrix $\mathbf{A}^n$ equals the number of v_i-v_j walks of length n.

Proof by Induction:

1. **Base Case ($n = 1$):** By the definition of the adjacency matrix, if $\mathbf{A}_{i,j} = 1$, an edge exists between v_i and v_j, representing a v_i-v_j walk of length 1. If $\mathbf{A}_{i,j} = 0$, no edge (and thus no walk of length 1) exists. The theorem holds.
2. **Inductive Step:** Assume the theorem holds for $n = k$, meaning the (i, h)-th element of $\mathbf{A}^k$ equals the number of v_i-v_h walks of length k.
3. To prove for $n = k + 1$, the (i, j)-th element of $\mathbf{A}^{k+1}$ is computed via matrix multiplication using $\mathbf{A}^k$ and $\mathbf{A}$:

$$\mathbf{A}_{i,j}^{k+1} = \sum_{h=1}^{N} \mathbf{A}_{i,h}^{k} \cdot \mathbf{A}_{h,j}$$

Note: The product $\mathbf{A}^k_{i,h} \cdot \mathbf{A}_{h,j}$ is non-zero only if both terms are non-zero. Since $\mathbf{A}^k_{i,h}$ is the number of length-k walks from v_i to v_h, and $\mathbf{A}_{h,j}$ indicates a length-1 walk from v_h to v_j, their product counts the v_i-v_j walks of length $k+1$ where v_h is the penultimate node. Summing over all possible intermediate nodes v_h yields the total number of v_i-v_j walks of length $k+1$. □

Subgraphs and Components

- **Subgraph (Definition 2.15):** A graph $\mathcal{G}' = \{\mathcal{V}', \mathcal{E}'\}$ derived from a given graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ such that:
  - $\mathcal{V}' \subset \mathcal{V}$ (subset of nodes)
  - $\mathcal{E}' \subset \mathcal{E}$ (subset of edges)
  - The subset $\mathcal{V}'$ must contain all nodes that are endpoints of the edges in $\mathcal{E}'$.

- **Connected Component (Definition 2.17):** A subgraph $\mathcal{G}' = \{\mathcal{V}', \mathcal{E}'\}$ is a connected component if:
  - There is at least one path between every pair of nodes within the subgraph.
  - The nodes in $\mathcal{V}'$ are not adjacent to any nodes in the remaining part of the graph, $\mathcal{V} \setminus \mathcal{V}'$.
- **Connected Graph (Definition 2.19):** A graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ is considered connected if it consists of **exactly one** connected component.

2.3 Properties and Measures

Graphs can possess multiple paths of varying lengths between any given pair of nodes. This variability necessitates metrics to quantify distance and compactness.

Shortest Path

Definition 2.21 (Shortest Path): Given a pair of nodes $v_s, v_t \in \mathcal{V}$ in a graph $\mathcal{G}$, let $\mathcal{P}_{st}$ denote the complete set of paths connecting node v_s to node v_t. The shortest path p_{st}^{sp} between these two nodes is defined as:

$$p_{st}^{sp} = \arg\min_{p \in \mathcal{P}_{st}} |p|$$

Where:

- p represents a path within the set $\mathcal{P}_{st}$.
- $|p|$ denotes the length (number of edges) of that path.
- **Note:** The shortest path between any given pair of nodes is not necessarily unique; multiple distinct paths can share the same minimum length.

Graph Diameter

The collective set of all shortest paths provides critical information about the structural characteristics and overall compactness of a graph.

Definition 2.22 (Diameter): Given a connected graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, its diameter is defined as the length of the longest shortest path between any pair of nodes in the graph:

$$\text{diameter}(\mathcal{G}) = \max_{v_s, v_t \in \mathcal{V}} \min_{p \in \mathcal{P}_{st}} |p|$$

2.3.3 Centrality

In a graph, the centrality of a node measures the importance of the node in the graph. There are different ways to measure node importance. In this section, we introduce various definitions of centrality.

Degree Centrality

Intuitively, a node can be considered as important if there are many other nodes connected to it. Hence, we can measure the centrality of a given node based on its degree. In particular, for node v_i, its degree centrality can be defined as follows:

$$c_d(v_i) = d(v_i) = \sum_{j=1}^{N} \mathbf{A}_{i,j}$$

- **Example 2.24:** For the graph shown in Figure 2.1, the degree centrality for nodes v_1 and v_5 is 3, while the degree centrality for nodes v_2, v_3 and v_4 is 2.

Eigenvector Centrality

While the degree-based centrality considers a node with many neighbors as important, it treats all the neighbors equally. However, the neighbors themselves can have different importance; thus they could affect the importance of the central node differently. The eigenvector centrality (Bonacich, 1972, 2007) defines the centrality score of a given node v_i by considering the centrality scores of its neighboring nodes as:

$$c_e(v_i) = \frac{1}{\lambda} \sum_{j=1}^{N} \mathbf{A}_{i,j} \cdot c_e(v_j)$$

Which can be rewritten in a matrix form as:

$$\mathbf{c}_e = \frac{1}{\lambda} \mathbf{A} \cdot \mathbf{c}_e \tag{2.3}$$

Where $\mathbf{c}_e \in \mathbb{R}^N$ is a vector containing the centrality scores of all nodes in the graph. We can reform Eq. (2.3) as:

$$\lambda \cdot \mathbf{c}_e = \mathbf{A} \cdot \mathbf{c}_e$$

Clearly, $\mathbf{c}_e$ is an eigenvector of the matrix $\mathbf{A}$ with its corresponding eigenvalue λ. However, given an adjacency matrix $\mathbf{A}$, there exist multiple pairs of eigenvectors and eigenvalues. Usually, we want the centrality scores to be positive. Hence, we wish to choose an eigenvector with all positive elements.

According to the **Perron–Frobenius theorem** (Perron, 1907; Frobenius et al., 1912; Pillai et al., 2005), a real squared matrix with positive elements has a unique largest eigenvalue and its corresponding eigenvector has all positive elements. Thus, we can choose λ as the largest eigenvalue and its corresponding eigenvector as the centrality score vector.

- **Example 2.25:** For the graph shown in Figure 2.1, its largest eigenvalue is 2.481 and its corresponding eigenvector is $[1, 0.675, 0.675, 0.806, 1]$. Hence, the eigenvector centrality scores for the nodes v_1, v_2, v_3, v_4, v_5 are $1, 0.675, 0.675, 0.806$, and 1, respectively. Note that the degrees of nodes v_2, v_3 and v_4 are 2; however, the eigenvector centrality of node v_4 is higher than that of the other two nodes as it directly connects to nodes v_1 and v_5 whose eigenvector centrality is high.

Katz Centrality

The Katz centrality is a variant of the eigenvector centrality, which not only considers the centrality scores of the neighbors but also includes a small constant for the central node itself. Specifically, the Katz centrality for a node v_i can be defined as:

$$c_k(v_i) = \alpha \sum_{j=1}^{N} \mathbf{A}_{i,j} c_k(v_j) + \beta \tag{2.4}$$

Where β is a constant. The Katz centrality scores for all nodes can be expressed in the matrix form as:

$$\mathbf{c}_k = \alpha \mathbf{A} \mathbf{c}_k + \boldsymbol{\beta}$$

$$(\mathbf{I} - \alpha \cdot \mathbf{A}) \mathbf{c}_k = \boldsymbol{\beta} \tag{2.5}$$

Where $\mathbf{c}_k \in \mathbb{R}^N$ denotes the Katz centrality score vector for all nodes while $\boldsymbol{\beta}$ is the vector containing the constant term β for all nodes.

Note on Parameter Relations:
The Katz centrality is equivalent to the eigenvector centrality if we set $\alpha = \frac{1}{\lambda_{max}}$ and $\beta = 0$, with λ_{max} being the largest eigenvalue of the adjacency matrix $\mathbf{A}$.

The choice of α is important:

- A large α may make the matrix $\mathbf{I} - \alpha \cdot \mathbf{A}$ ill-conditioned.
- A small α may make the centrality scores useless since it will assign very similar scores close to β to all nodes.

In practice, $\alpha < \frac{1}{\lambda_{max}}$ is often selected, which ensures that the matrix $\mathbf{I} - \alpha \cdot \mathbf{A}$ is invertible and $\mathbf{c}_k$ can be calculated as:

$$\mathbf{c}_k = (\mathbf{I} - \alpha \cdot \mathbf{A})^{-1}\boldsymbol{\beta}$$

- **Example 2.26:** For the graph shown in Figure 2.1, if we set $\beta = 1$ and $\alpha = \frac{1}{5}$, the Katz centrality score for nodes v_1 and v_5 is 2.16, for nodes v_2 and v_3 is 1.79 and for node v_4 is 1.87.

Betweenness Centrality

The aforementioned centrality scores are based on connections to neighboring nodes. Another way to measure the importance of a node is to check whether it is at an important position in the graph. Specifically, if there are many paths passing through a node, it is at an important position in the graph. Formally, we define the betweenness centrality score for a node v_i as:

$$c_b(v_i) = \sum_{v_s \neq v_i \neq v_t} \frac{\sigma_{st}(v_i)}{\sigma_{st}} \tag{2.6}$$

Where:

- σ_{st} denotes the total number of shortest paths from node v_s to node v_t.
- $\sigma_{st}(v_i)$ indicates the number of these paths passing through the node v_i.

As suggested by Eq. (2.6), we need to compute the summation over all possible pairs of nodes for the betweenness centrality score. Therefore, the magnitude of the betweenness centrality score scales as the size of the graph scales. Hence, to make the betweenness centrality score comparable across different graphs, we need to normalize it.

One effective way is to divide the betweenness score by the largest possible betweenness centrality score given a graph. In Eq. (2.6), the maximum of the betweenness score can be reached when all the shortest paths between any pair of nodes pass through the node v_i. That is $\frac{\sigma_{st}(v_i)}{\sigma_{st}} = 1,\ \forall v_s \neq v_i \neq v_t$. There are, in total, $\frac{(N-1)(N-2)}{2}$ pairs of nodes in an undirected graph. Hence, the maximum betweenness centrality score is $\frac{(N-1)(N-2)}{2}$.

We then define the normalized betweenness centrality score $c_{nb}(v_i)$ for the node v_i as:

$$c_{nb}(v_i) = \frac{2\sum_{v_s \neq v_i \neq v_t} \frac{\sigma_{st}(v_i)}{\sigma_{st}}}{(N-1)(N-2)}$$

- **Example 2.27:** For the graph shown in Figure 2.1, the betweenness centrality score for nodes v_1 and v_5 is $\frac{3}{2}$, and their normalized betweenness score is $\frac{1}{4}$. The betweenness centrality score for nodes v_2 and v_3 is $\frac{1}{2}$, and their normalized betweenness score is $\frac{1}{12}$. The betweenness centrality score for node v_4 is 0 and its normalized score is also 0.

2.4 Spectral Graph Theory

16 July 2026 07:50

2.4 Spectral Graph Theory

Spectral graph theory studies the properties of a graph by analyzing the eigenvalues and eigenvectors of its **Laplacian matrix**. It provides an alternative framework to study graph topologies beyond the standard adjacency matrix.

2.4.1 Laplacian Matrix

The Laplacian matrix serves as a fundamental matrix representation for graphs alongside the adjacency matrix.

Definition 2.28 (Laplacian Matrix)

For a given graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ with $\mathbf{A}$ as its adjacency matrix, its **unnormalized Laplacian matrix** $\mathbf{L}$ is defined as:

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

Where:

- $\mathbf{D}$ is the diagonal degree matrix defined as $\mathbf{D} = \mathrm{diag}(d(v_1), \ldots, d(v_N))$.

Definition 2.29 (Normalized Laplacian Matrix)

A normalized alternative version of the Laplacian matrix is defined as:

$$\mathbf{L} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{D} - \mathbf{A})\mathbf{D}^{-\frac{1}{2}} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$$

Note: Unless specified otherwise, "Laplacian matrix" typically refers to the unnormalized version defined in Definition 2.28.

Key Matrix Operations & Vector Multiplications

Since both the degree matrix $\mathbf{D}$ and the adjacency matrix $\mathbf{A}$ are symmetric, the Laplacian matrix $\mathbf{L}$ is also **symmetric**.

Let $\mathbf{f}$ denote a real-valued vector where its i-th element $\mathbf{f}[i]$ is associated with node v_i. Multiplying $\mathbf{L}$ by $\mathbf{f}$ yields a new vector $\mathbf{h}$:

$$\mathbf{h} = \mathbf{L}\mathbf{f} = (\mathbf{D} - \mathbf{A})\mathbf{f} = \mathbf{D}\mathbf{f} - \mathbf{A}\mathbf{f}$$

The i-th element of $\mathbf{h}$ can be broken down explicitly as:

$$\mathbf{h}[i] = d(v_i) \cdot \mathbf{f}[i] - \sum_{j=1}^{N} \mathbf{A}_{i,j} \cdot \mathbf{f}[j]$$

$$\mathbf{h}[i] = d(v_i) \cdot \mathbf{f}[i] - \sum_{v_j \in \mathcal{N}(v_i)} \mathbf{A}_{i,j} \cdot \mathbf{f}[j]$$

$$\mathbf{h}[i] = \sum_{v_j \in \mathcal{N}(v_i)} (\mathbf{f}[i] - \mathbf{f}[j])$$

Thus, $\mathbf{h}[i]$ computes the **summation of the differences** between node v_i and its immediate local neighborhood $\mathcal{N}(v_i)$.

Quadratic Form: $\mathbf{f}^T\mathbf{L}\mathbf{f}$

Evaluating the inner product quadratic form yields:

$$\mathbf{f}^T\mathbf{L}\mathbf{f} = \sum_{v_i \in \mathcal{V}} \mathbf{f}[i] \sum_{v_j \in \mathcal{N}(v_i)} (\mathbf{f}[i] - \mathbf{f}[j])$$

$$\mathbf{f}^T\mathbf{L}\mathbf{f} = \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{N}(v_i)} (\mathbf{f}[i] \cdot \mathbf{f}[i] - \mathbf{f}[i] \cdot \mathbf{f}[j])$$

$$\mathbf{f}^T\mathbf{L}\mathbf{f} = \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{N}(v_i)} \left(\frac{1}{2}\mathbf{f}[i] \cdot \mathbf{f}[i] - \mathbf{f}[i] \cdot \mathbf{f}[j] + \frac{1}{2}\mathbf{f}[j] \cdot \mathbf{f}[j] \right)$$

$$\mathbf{f}^T\mathbf{L}\mathbf{f} = \frac{1}{2} \sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{N}(v_i)} (\mathbf{f}[i] - \mathbf{f}[j])^2$$

Implications:

- **Smoothness Measure:** $\mathbf{f}^T\mathbf{L}\mathbf{f}$ calculates the **sum of the squares of the differences** between adjacent nodes. It measures how much the node values change across connected neighbors.
- **Positive Semi-Definiteness:** Because it is a sum of squares, $\mathbf{f}^T\mathbf{L}\mathbf{f} \geq 0$ for any real vector $\mathbf{f}$. Consequently, the Laplacian matrix is **positive semi-definite**.

2.4.2 The Eigenvalues and Eigenvectors of the Laplacian Matrix

This section establishes key spectral properties based on the eigenvalues and eigenvectors of $\mathbf{L}$.

Theorem 2.30

For a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, the eigenvalues of its Laplacian matrix $\mathbf{L}$ are non-negative.

- **Proof:**
  Let λ be an eigenvalue of $\mathbf{L}$ and $\mathbf{u}$ be its corresponding normalized unit eigenvector ($\mathbf{u}^T\mathbf{u} = 1$). By definition, $\lambda\mathbf{u} = \mathbf{L}\mathbf{u}$.
  Left-multiplying by $\mathbf{u}^T$:

$$\lambda = \lambda\mathbf{u}^T\mathbf{u} = \mathbf{u}^T\lambda\mathbf{u} = \mathbf{u}^T\mathbf{L}\mathbf{u} \geq 0$$

  Since the quadratic form is non-negative, $\lambda \geq 0$. ■

Properties of Eigenvalues & Eigenvectors:

- For a graph with N nodes, there are N total eigenvalues (accounting for multiplicity).
- **The Smallest Eigenvalue:** There always exists at least one eigenvalue equal to 0. Let's test a constant unit vector $\mathbf{u}_1 = \frac{1}{\sqrt{N}}(1, \ldots, 1)$. Applying the neighborhood variation formula shows $\mathbf{L}\mathbf{u}_1 = \mathbf{0} = 0\mathbf{u}_1$, verifying that 0 is an eigenvalue with $\mathbf{u}_1$ as its normalized eigenvector.
- **Ordering:** Eigenvalues are conventionally sorted in non-decreasing order:

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$$

  Their corresponding normalized eigenvectors are denoted as $\mathbf{u}_1, \ldots, \mathbf{u}_N$.

Theorem 2.31

Given a graph $\mathcal{G}$, the number of 0 eigenvalues of its Laplacian matrix $\mathbf{L}$ (the algebraic multiplicity of the 0 eigenvalue) equals the **number of connected components** in the graph.

- **Proof:**
  Assume there are K connected components in graph $\mathcal{G}$. We can partition the node set $\mathcal{V}$ into K disjoint subsets $\mathcal{V}_1, \ldots, \mathcal{V}_K$.

  1. **Lower Bound ($\geq K$):** We construct K distinct vectors $\mathbf{u}_1, \ldots, \mathbf{u}_K$ such that for each component:

  $$\mathbf{u}_i[j] = \frac{1}{\sqrt{|\mathcal{V}_i|}} \text{ if } v_j \in \mathcal{V}_i, \text{ and } 0 \text{ otherwise}$$

  Because there are no edges bridging distinct components, evaluating $\mathbf{L}\mathbf{u}_i = \mathbf{0}$ holds true for all $i = 1, \ldots, K$. Additionally, these vectors are mutually orthogonal ($\mathbf{u}_i^T \mathbf{u}_j = 0$ for $i \neq j$). Thus, there are at least K orthogonal eigenvectors for the eigenvalue 0.

  2. **Upper Bound ($\leq K$):** Suppose there exists another non-zero eigenvector $\mathbf{u}^*$ corresponding to eigenvalue 0 that is orthogonal to all K constructed vectors. Since $\mathbf{u}^*$ is non-zero, let some element $\mathbf{u}^*[d] \neq 0$ belong to a node $v_d \in \mathcal{V}_i$. Using the quadratic formulation:

  $$\mathbf{u}^{*T}\mathbf{L}\mathbf{u}^* = \frac{1}{2}\sum_{v_i \in \mathcal{V}} \sum_{v_j \in \mathcal{N}(v_i)} (\mathbf{u}^*[i] - \mathbf{u}^*[j])^2$$

  To satisfy $\mathbf{u}^{*T}\mathbf{L}\mathbf{u}^* = 0$, the values of all nodes within the same connected component must be identical. Thus, all nodes in component $\mathcal{V}_i$ share the exact same constant value $\mathbf{u}^*[d]$ as node v_d.
  This means $\mathbf{u}_i^T \mathbf{u}^* > 0$, directly contradicting the initial assumption that $\mathbf{u}^*$ is orthogonal to $\mathbf{u}_i$. Therefore, no additional orthogonal eigenvectors for eigenvalue 0 can exist beyond the initial K vectors. ■

2.5 Graph Signal Processing

16 July 2026 07:55

2.5 Graph Signal Processing

In many real-world graphs, features or attributes are associated with nodes. This graph-structured data is treated as **graph signals**, capturing both:

- **Structure Information:** The connectivity between nodes.
- **Data / Attributes:** The values present at the nodes.

Mathematical Formulation

A graph signal consists of a graph $G = \{\mathcal{V}, \mathcal{E}\}$ and a mapping function f defined in the graph domain that maps nodes to real values:

$$f : \mathcal{V} \to \mathbb{R}^{N \times d}$$

Where d is the dimension of the value vector associated with each node.

Simplification ($d = 1$): Without loss of generality, setting $d = 1$ denotes the mapped values for all nodes as $\mathbf{f} \in \mathbb{R}^N$, where $\mathbf{f}[i]$ corresponds to node v_i.

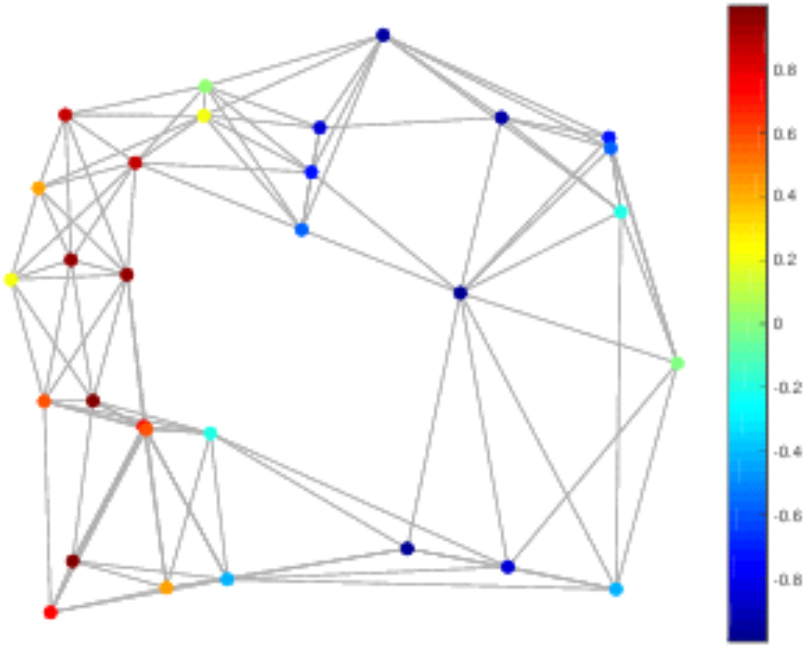


Figure 2.3 A one-dimensional graph signal

Graph Smoothness & Frequency

- **Smooth Graph:** A graph is considered smooth if the values at connected nodes are similar.
- **Low Frequency:** A smooth graph signal corresponds to a low frequency because values change slowly across the graph via the edges.
- **Measurement via Laplacian Matrix:** The quadratic form of the Laplacian matrix $\mathbf{L}$ is utilized to measure the smoothness (or frequency) of a graph signal $\mathbf{f}$:
  - The value $\mathbf{f}^T\mathbf{L}\mathbf{f}$ represents the **smoothness** (or frequency) of the signal $\mathbf{f}$.
  - When a graph signal $\mathbf{f}$ is smooth, $\mathbf{f}^T\mathbf{L}\mathbf{f}$ is small. It is the summation of the squared differences between all pairs of connected nodes.

Signal Domains

A graph signal can be represented in two domains:

1. **Spatial Domain:** The structural representation mapping values directly onto nodes.
2. **Spectral Domain (Frequency Domain):** Based on the **Graph Fourier Transform**, built upon spectral graph theory.

2.5.1 Graph Fourier Transform

Classical vs. Graph Fourier Transform

- **Classical Fourier Transform:** Decomposes a continuous signal $f(t)$ into a series of complex exponentials $\exp(-2\pi it\xi)$ for any real number ξ (representing frequency):

$$\hat{f}(\xi) = \langle f(t), \exp(-2\pi it\xi) \rangle = \int_{-\infty}^{\infty} f(t)\exp(-2\pi it\xi)dt$$

  These exponentials act as the eigenfunctions of the one-dimensional Laplace operator (second-order differential operator):

$$\begin{aligned}
\nabla\left(e^{-2\pi it\xi}\right) &= \frac{\partial^2}{\partial t^2} e^{-2\pi it\xi} \\
&= \frac{\partial}{\partial t}\left[(-2\pi i\xi)\, e^{-2\pi it\xi}\right] \\
&= (-2\pi i\xi)^2 e^{-2\pi it\xi} \\
&= -(2\pi\xi)^2 e^{-2\pi it\xi}.
\end{aligned}$$

$$(-2\pi i\xi)^2 = (-2\pi\xi)^2 i^2 = -(2\pi\xi)^2,$$

the final result is

$$\boxed{\frac{\partial^2}{\partial t^2} e^{-2\pi i t\xi} = -(2\pi\xi)^2 e^{-2\pi i t\xi}.}$$

This is the standard second derivative of the complex exponential used in Fourier analysis.

- **Graph Fourier Transform (GFT):** Analogously, the GFT for a graph signal $\mathbf{f}$ on graph G is defined as:

$$\hat{\mathbf{f}}[l] = \langle \mathbf{f}, \mathbf{u}_l \rangle = \sum_{i=1}^{N} \mathbf{f}[i]\mathbf{u}_l[i] \quad -(2.11)$$

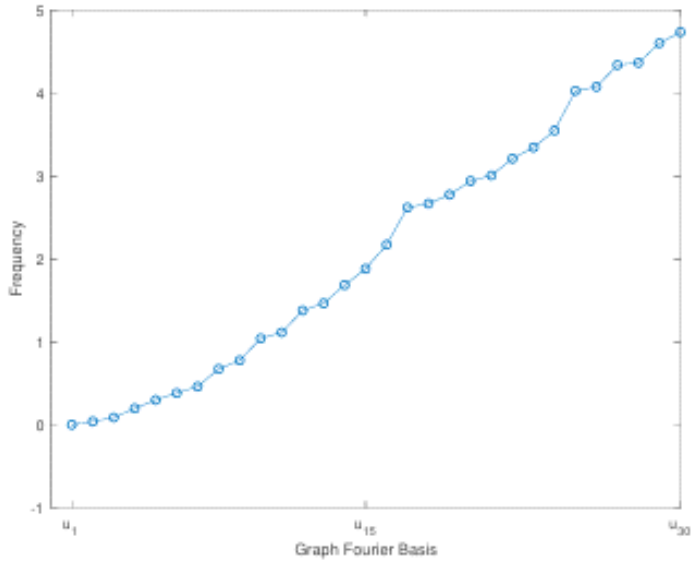


Figure 2.4 Frequencies of graph Fourier basis

Key Components of GFT

- **Graph Fourier Basis:** The eigenvectors $\mathbf{u}_l$ (l-th eigenvector) of the Laplacian matrix $\mathbf{L}$ of the graph.
- **Graph Fourier Coefficients:** The vector $\hat{\mathbf{f}}$ with $\hat{\mathbf{f}}[l]$ as its l-th element.
- **Matrix Form:** The GFT can be written compactly as:

$$\hat{\mathbf{f}} = \mathbf{U}^T\mathbf{f} \quad -(2.12)$$

  Where the l-th column of the matrix $\mathbf{U}$ is $\mathbf{u}_l$.

Understanding Eigenvalues as Frequencies

The eigenvalue λ_l measures the smoothness of its corresponding eigenvector $\mathbf{u}_l$:

$$\mathbf{u}_l^T\mathbf{L}\mathbf{u}_l = \lambda_l \cdot \mathbf{u}_l^T\mathbf{u}_l = \lambda_l$$

- **Small Eigenvalues (λ_l):** The associated eigenvectors vary slowly across the graph (values at connected nodes are similar). These eigenvectors are smooth and change with **low frequency**.
- **Large Eigenvalues (λ_l):** The associated eigenvectors can have very different values on two connected nodes, corresponding to **high frequency**.
- **The First Eigenvector ($\mathbf{u}_1$):** Associated with the eigenvalue $\lambda_1 = 0$. It is constant over all nodes, meaning its value does not change across the graph. It is extremely smooth and has a frequency of 0.

Summary of Decomposition: The Graph Fourier Transform decomposes an input signal $\mathbf{f}$ into a graph Fourier basis with different frequencies. The obtained coefficients $\hat{\mathbf{f}}$ denote how much each corresponding graph Fourier basis contributes to the input signal.

Inverse Graph Fourier Transform (IGFT)

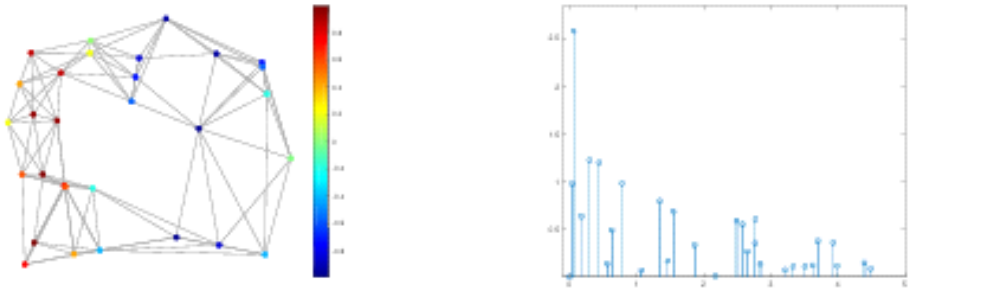
The Inverse Graph Fourier Transform converts the spectral representation $\hat{\mathbf{f}}$ back into the spatial representation $\mathbf{f}$:

- **Element-wise Form:**

$$\mathbf{f}[i] = \sum_{l=1}^{N} \hat{\mathbf{f}}[l]\mathbf{u}_l[i]$$

- **Matrix Form:**

$$\mathbf{f} = \mathbf{U}\hat{\mathbf{f}}$$



(a) A graph signal in the spatial domain (b) A graph signal in the spectral domain

Figure 2.5 Representations of a graph signal in both spatial and spectral Domains

Domain Summary Table

Domain	Representation Type	Description
Spatial Domain	$\mathbf{f}$	Signal values mapped directly on the graph structure/nodes.
Spectral Domain	$\hat{\mathbf{f}}$	Contribution coefficients corresponding to the graph Fourier basis (frequencies).

...

Both domains can be seamlessly transformed into each other using the **Graph Fourier Transform** and the **Inverse Graph Fourier Transform**.

2.6 Complex Graphs

16 July 2026 08:00

2.6 Complex Graphs

While simple graphs are homogeneous (consisting of only one type of node and a single type of edge), real-world graph applications are often much more complicated. This section introduces popular complex graphs with formal definitions.

2.6.1 Heterogeneous Graphs

Heterogeneous graphs model environments where there are multiple types of relations between multiple types of nodes.

- **Real-World Example:** An academic network describing publications and citations.
  - **Node types:** Authors, papers, and venues.
  - **Edge types:** Citation relations (between papers) and authorship relations (between authors and papers).

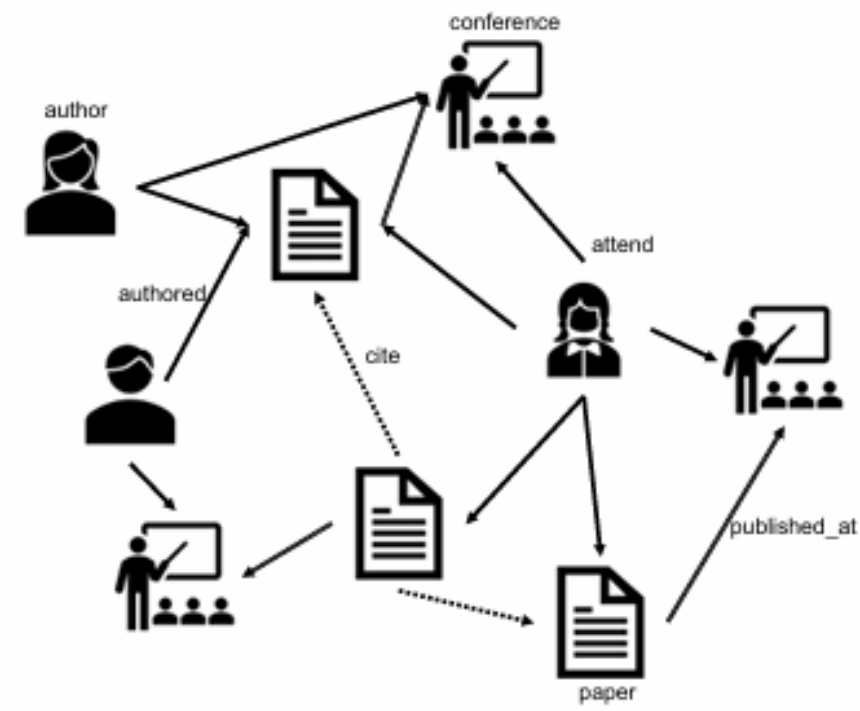


Figure 2.6 A heterogeneous academic graph

Definition 2.35 (Heterogeneous Graphs)

A heterogeneous graph $\mathcal{G}$ consists of a set of nodes $\mathcal{V} = \{v_1, \ldots, v_N\}$ and a set of edges $\mathcal{E} = \{e_1, \ldots, e_M\}$ where each node and each edge are associated with a type.

Let $\mathcal{T}_n$ denote the set of node types and $\mathcal{T}_e$ indicate the set of edge types. There are two mapping functions $\phi_n : \mathcal{V} \to \mathcal{T}_n$ and $\phi_e : \mathcal{E} \to \mathcal{T}_e$ that map each node and each edge to their types, respectively.

2.6.2 Bipartite Graphs

Bipartite graphs capture interactions between two distinct, non-overlapping categories of objects. Every edge in a bipartite graph must connect a node from one set to a node in the other set.

- **Real-World Example:** E-commerce platforms like Amazon.
  - **Disjoint Node Sets:** Users and items.
  - **Edges:** Formed by the users' click behaviors or histories.

Definition 2.36 (Bipartite Graph)

Given a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, it is bipartite if and only if:

$$\mathcal{V} = \mathcal{V}_1 \cup \mathcal{V}_2, \quad \mathcal{V}_1 \cap \mathcal{V}_2 = \emptyset$$

and $v_e^1 \in \mathcal{V}_1$ while $v_e^2 \in \mathcal{V}_2$ for all $e = (v_e^1, v_e^2) \in \mathcal{E}$.

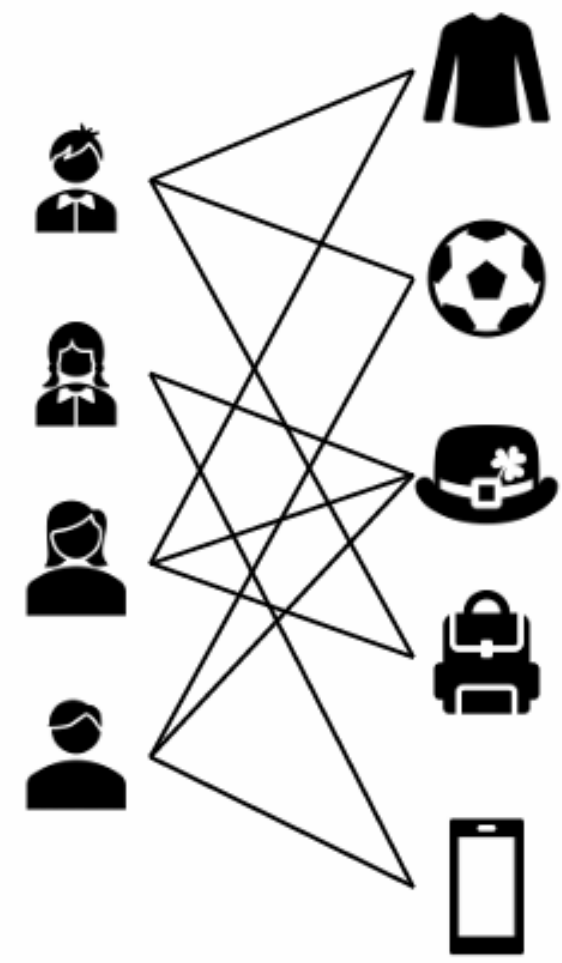


Figure 2.7 An e-commerce bipartite graph

2.6.3 Multi-dimensional Graphs

Multi-dimensional graphs represent networks where multiple distinct types of relations can simultaneously exist between the exact same pair of nodes. Each relation type is modeled as its own dimension.

- **Real-World Examples:**
  - **YouTube:** Users are nodes. One relation dimension is subscribing to each other, while other dimensions include "sharing" or "commenting" on videos.
  - **Amazon:** Users interact with items through various dimensions of behaviors such as "click", "purchase", and "comment".

Definition 2.37 (Multi-dimensional Graph)

A multi-dimensional graph consists of a set of N nodes $\mathcal{V} = \{v_1, \ldots, v_N\}$ and D sets of edges $\{\mathcal{E}_1, \ldots, \mathcal{E}_D\}$. Each edge set $\mathcal{E}_d$ describes the d-th type of relations between the nodes in the corresponding d-th dimension.

These D types of relations can also be expressed by D adjacency matrices $\mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(D)}$. In the dimension d, its corresponding adjacency matrix $\mathbf{A}^{(d)} \in \mathbb{R}^{N \times N}$ describes the edges $\mathcal{E}_d$ between nodes in $\mathcal{V}$. Specifically, the i, j-th element of $\mathbf{A}^{(d)}$, denoted as $\mathbf{A}^{(d)}_{i,j}$, equals to 1 only when there is an edge between nodes v_i and v_j in the dimension d (or $(v_i, v_j) \in \mathcal{E}_d$), otherwise 0.

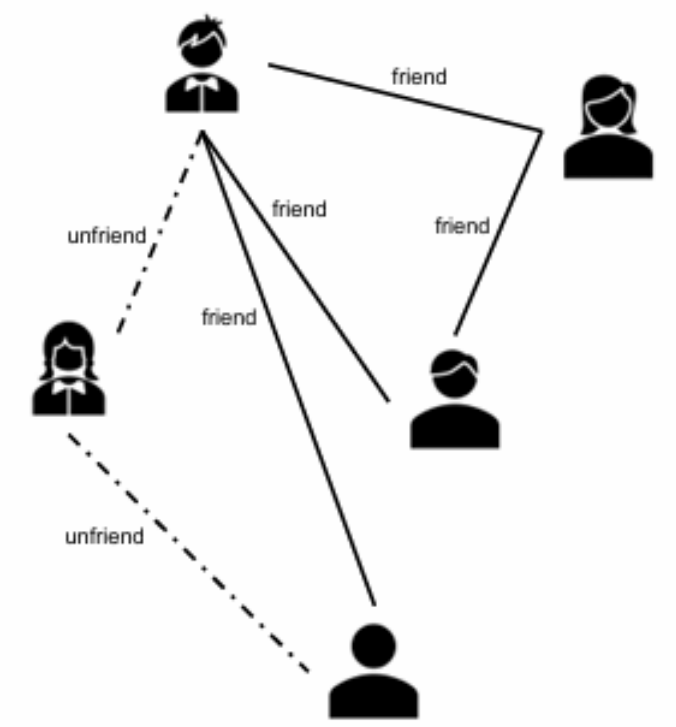


Figure 2.8 An illustrative signed graph

2.6.4 Signed Graphs

Signed graphs contain both positive and negative edges. They have become increasingly ubiquitous due to the growth of online social networks.

- **Real-World Example:** Social networks like Facebook and Twitter.
  - **Positive Edges (+):** Behaviors like adding a "friend" or following a user.
  - **Negative Edges (-):** Behaviors like "unfriend", "block", or "unfollow".

Definition 2.38 (Signed Graphs)

Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}^+, \mathcal{E}^-\}$ be a signed graph, where $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the set of N nodes while $\mathcal{E}^+ \subset \mathcal{V} \times \mathcal{V}$ and $\mathcal{E}^- \subset \mathcal{V} \times \mathcal{V}$ denote the sets of positive and negative edges, respectively. Note that an edge can only be either positive or negative, i.e., $\mathcal{E}^+ \cap \mathcal{E}^- = \emptyset$.

These positive and negative edges between nodes can also be described by a signed adjacency matrix $\mathbf{A}$, where:

- $\mathbf{A}_{i,j} = 1$ only when there is a positive edge between node v_i and node v_j.
- $\mathbf{A}_{i,j} = -1$ denotes a negative edge.
- Otherwise, $\mathbf{A}_{i,j} = 0$.

Figure 2.9 An illustrative hypergraph

2.6.5 Hypergraphs

Core Concept

- **Limitation of Simple Graphs:** Standard graphs only encode pairwise information via traditional edges.
- **Real-World Application:** Many real-world relations involve higher-order associations that go beyond simple pairs (e.g., an author publishing papers with multiple co-authors simultaneously).
- **Hyperedge:** An edge that can connect multiple nodes (or papers, in the provided example) at the same time, thereby capturing higher-order relationships.
- **Hypergraph:** A specialized graph structure containing hyperedges.

Formal Mathematical Representation

Definition 2.39 (Hypergraphs)
Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{W}\}$ be a hypergraph, where:

- $\mathcal{V}$: A set of N nodes.
- $\mathcal{E}$: A set of hyperedges.
- $\mathbf{W} \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{E}|}$: A diagonal matrix where $\mathbf{W}_{j,j}$ denotes the weight of the hyperedge e_j.

Structural Matrices and Node Degrees

- **Incidence Matrix ($\mathbf{H}$):** The hypergraph $\mathcal{G}$ can be formally described by an incidence matrix $\mathbf{H} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{E}|}$.
  - $\mathbf{H}_{i,j} = 1$ only when the node v_i is incident to the hyperedge e_j.
- **Node Degree ($d(v_i)$):** For a given node v_i, its degree is defined as:

$$d(v_i) = \sum_{j=1}^{|\mathcal{E}|} \mathbf{H}_{i,j}$$

- **Hyperedge Degree ($d(e_j)$):** For a given hyperedge e_j, its degree is defined as:

$$d(e_j) = \sum_{i=1}^{|\mathcal{V}|} \mathbf{H}_{i,j}$$

- **Degree Matrices:** $\mathbf{D}_e$ and $\mathbf{D}_v$ denote the diagonal matrices containing the hyperedge degrees and node degrees, respectively.

2.6.6 Dynamic Graphs

Core Concept

- **Static vs. Dynamic:** Static graphs assume fixed connections between nodes when observed. However, in reality, graphs are often constantly evolving as new nodes are introduced and new edges continuously emerge.
- **Real-World Example:** Online social networks (like Facebook), where users constantly establish new friendships over time, and new users join the platform at any given moment.
- **Timestamps:** In an evolving dynamic graph, each node or edge is explicitly associated with a chronological timestamp.
    - **Node Timestamp:** The timestamp of a node corresponds to the exact moment the very first edge involves that node.

Formal Mathematical Representation

Definition 2.40 (Dynamic Graphs)
A dynamic graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ consists of:

- A set of nodes $\mathcal{V} = \{v_1, \ldots, v_N\}$
- A set of edges $\mathcal{E} = \{e_1, \ldots, e_M\}$

Each node and/or each edge is associated with a timestamp indicating the time it emerged. Specifically, two mapping functions ϕ_v and ϕ_e map each node and each edge to their respective emerging timestamps.

Discrete Dynamic Graphs

- **Practical Constraint:** In many actual scenarios, it is impossible to record a continuous, real-time timestamp for every single node and edge.
- **Snapshot Observation:** Instead, the graph's evolution is observed periodically from time to time. At each specific observation timestamp t, a snapshot of the graph (denoted as $\mathcal{G}_t$) is recorded.

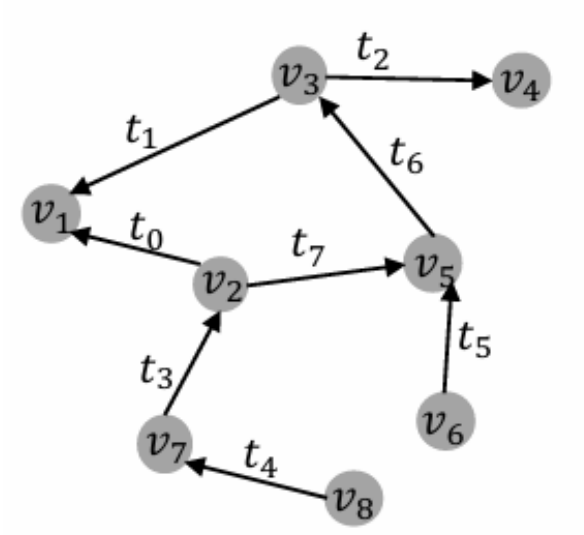


Figure 2.10 An illustrative example of dynamic graphs.

Definition 2.41 (Discrete Dynamic Graphs)
A discrete dynamic graph consists of T graph snapshots observed along with the evolution of a dynamic graph. Specifically, these T graph snapshots can be formally denoted as:

$$\{\mathcal{G}_0, \ldots, \mathcal{G}_T\}$$

Where $\mathcal{G}_0$ represents the base graph observed at time 0.

2.7 Computational Tasks on Graphs

16 July 2026 08:07

2.7 Computational Tasks on Graphs

Computational tasks on graphs can be broadly categorized into two major divisions depending on what constitutes a single data sample:

- **Node-focused tasks:** The entire data is represented as one large single graph where the nodes serve as individual data samples.
- **Graph-focused tasks:** The data consists of a dataset containing multiple independent graphs, where each individual data sample is itself a graph.

2.7.1 Node-focused Tasks

Extensively studied examples include node classification, node ranking, link prediction, and community detection. The two most typical tasks are detailed below:

Node classification

In real-world graphs, nodes contain useful information often treated as **labels** (e.g., demographic properties like age/gender or user interests in social networks). In practice, getting a complete set of labels for all nodes is difficult (e.g., $< 1\%$ of Facebook users provide full demographics). The goal is to infer the missing labels from the partially labeled graph.

Definition 2.42 (Node classification)
Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ denote a graph with $\mathcal{V}$ as the set of nodes and $\mathcal{E}$ as the set of edges.

- $\mathcal{V}_l \subset \mathcal{V}$: The set of labeled nodes.
- $\mathcal{V}_u$: The set of unlabeled nodes.
- $\mathcal{V}_l \cup \mathcal{V}_u = \mathcal{V}$ and $\mathcal{V}_l \cap \mathcal{V}_u = \emptyset$.

The objective of node classification is to learn a mapping function ϕ by leveraging the structure of $\mathcal{G}$ and the labels of $\mathcal{V}_l$ to accurately predict the labels of the unlabeled nodes ($v \in \mathcal{V}_u$).

- **Extensions:** This formulation extends easily to attributed graphs and complex graphs.
- **Example 2.43 (Node Classification in Flickr):** On Flickr, users form a social network graph by following each other. Users subscribe to various interest groups (e.g., *"Black and White"*, *"The Fog and The Rain"*), which act as multiple labels per user. Multi-label node classification is used here to predict potential groups a user might want to join.

Link Prediction

Real-world networks are frequently incomplete (containing missing or unrecorded edges) or are naturally evolving over time (e.g., forming new friendships on Facebook or new academic co-authorships). Link prediction aims to discover these unobserved or future connections.

Definition 2.44 (Link Prediction)
Let $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ denote a graph where $\mathcal{V}$ is the node set and $\mathcal{E}$ is the edge set. Let $\mathcal{M}$ denote the set of all possible edges between nodes in $\mathcal{V}$.
The complementary set of $\mathcal{E}$ with respect to $\mathcal{M}$ is defined as $\mathcal{E}' = \mathcal{M} \setminus \mathcal{E}$, representing all unobserved edges.
The objective of link prediction is to assign a score to each edge in $\mathcal{E}'$ indicating how likely it is to exist or emerge in the future.

- **Extensions:**
  - **Signed graphs:** Predicts both the existence of an edge and its corresponding positive/negative sign.
  - **Hypergraphs:** Aims to infer hyperedges that represent complex relationships between multiple nodes simultaneously.
- **Example 2.45 (Predicting Emerging Collaborations in DBLP):** In a computer science bibliography co-authorship graph (like DBLP), authors are nodes, and an edge exists if they have co-authored at least one paper. Link prediction is utilized to forecast new collaborative relations between authors who have never worked together before.

2.7.2 Graph-focused Tasks

Common examples include graph classification, graph matching, and graph generation. The primary focus is typically on graph classification.

Graph Classification

Unlike node classification where a single node is a sample, graph classification treats an **entire graph** as a single distinct data sample. Traditional machine learning classification models struggle with this due to the structural complexity of graphs.

Definition 2.46 (Graph Classification)
Given a dataset of labeled graphs $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}$, where y_i represents the ground-truth label of the graph $\mathcal{G}_i$.
The objective of the graph classification task is to learn a mapping function ϕ using $\mathcal{D}$ that can successfully predict the structural labels of completely unlabeled test graphs.

- **Additional Information:** Graphs may contain extra node-level or edge-level feature attributes that can be incorporated to enhance classification performance.
- **Example 2.47 (Classifying Proteins into Enzymes or Non-enzymes):** In bioinformatics, proteins are modeled as structural graphs where amino acids are nodes, and an edge is drawn between them if they sit less than 6Å apart in space. Predicting whether a given protein functions as an enzyme (biological catalyst) or a non-enzyme is framed as a classic graph classification problem.

2.8 Conclusion

This chapter provides a foundational overview of graph theory and its core applications in modern graph-based machine learning. The key areas covered include:

- **Graph Fundamentals:** Introduction to basic graph concepts, including foundational data structures and matrix representations of graphs (such as adjacency matrices).
- **Graph Measures and Properties:** Examination of essential structural properties including **degree**, **connectivity**, and **centrality measures**.
- **Graph Signal Processing (GSP):** Discussion on the **Graph Fourier Transform** and fundamental graph signal processing techniques, which establish the theoretical foundations for spectral-based Graph Neural Networks (GNNs).
- **Complex Graphs:** Structural variations and advancements regarding a wide variety of complex graph architectures.
- **Computational Tasks:** Identification of representative computational problems on graphs, broadly categorized into:
  - **Node-focused tasks** (e.g., node classification, node embedding).
  - **Graph-focused tasks** (e.g., graph classification, graph generation).

2.9 Further Reading

For expanding beyond basic graph theory concepts, the following resources, advanced topics, and tools are highlighted:

Advanced Theoretical Concepts & Problems

- **Advanced Properties:** Concepts such as *flow* and *cut*.
- **Classic Graph Problems:**
  - Graph coloring problem
  - Route problem
  - Network flow problem
  - Covering problem
- **Key Literature:**
  - *General Graph Theory:* Bondy *et al.*; Newman (2018).
  - *Spectral Graph Theory:* Detailed spectral properties and theories can be explored in Chung and Graham (1997).
  - *Domain-Specific Applications:* Applications across various areas are covered by Borgatti *et al.* (2009), Nastase *et al.* (2015), and Trinajstic (2018).

Repositories and Datasets

To evaluate graph algorithms on large-scale data, the following primary repositories host extensive graph datasets from multiple domains:

- **Stanford Large Network Dataset Collection (SNAP):** (Leskovec and Krevl, 2014)
- **Network Data Repository:** (Rossi and Ahmed, 2015)

Computational Libraries and Toolboxes

Several software libraries are available to analyze, visualize, and process graph structures efficiently:

- **Graph Analysis & Visualization (Python):**
  - `networkx` (Hagberg *et al.*, 2008)
  - `graph-tool` (Peixoto, 2014)
  - `SNAP` (Leskovec and Sosič, 2016)
- **Graph Signal Processing:**
  - `Graph Signal Processing Toolbox` (Perraudin *et al.*, 2014) can be used to perform spectral operations and signal transformations on graphs.

R2-Chapter -2 Graph Essentials

16 July 2026 08:12

Chapter 2 Graph Essentials

- Networks are heavily intertwined with our daily lives, forming the basis of transportation systems, critical infrastructure, communication channels, and social interactions.
- Social media mining focuses on making sense of individuals embedded within these networks.
- Networks can be conveniently represented using graphs, where individuals are modeled as nodes (depicted as circles) and their connections are modeled as edges (depicted as lines).
- In a network, the node with the maximum *degree* (the number of edges connected to it) is often considered the most influential person, confirming theories that link high connectivity to high influence.

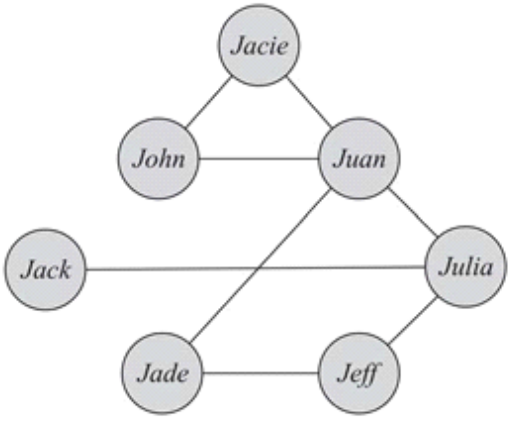


Figure 2.1: A Sample Graph. In this graph, individuals are represented with nodes (circles), and individuals who know each other are connected with edges (lines).

2.1 Graph Basics

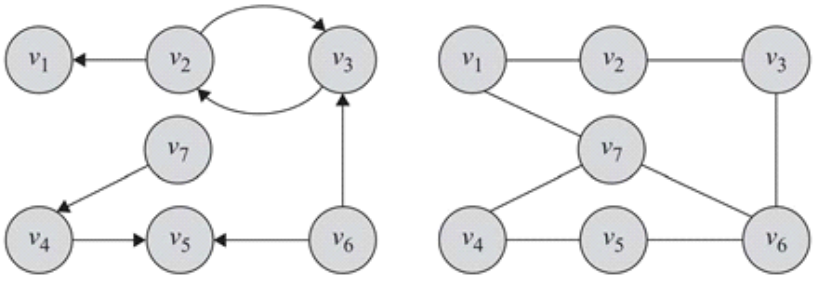
- Every graph contains a set of objects, known as nodes, and the connections between them, known as edges.
- Mathematically, a graph G is defined as a pair $G(V, E)$, where V is the set of nodes and E is the set of edges.

2.1.1 Nodes

- Nodes serve as the fundamental building blocks of any graph.
- Depending on the specific context of the graph, nodes may also be referred to as *vertices* or *actors*.
- Nodes represent the entities in a network, such as people in a social setting or websites in a web graph.
- The mathematical representation for a set of nodes is:

$$V = \{v_1, v_2, \ldots, v_n\}$$

  where v_i (for $1 \leq i \leq n$) represents a single node.
- The total number of nodes is denoted as $|V| = n$, which is referred to as the *size of the graph*.



(a) Directed Graph (b) Undirected Graph

Figure 2.2: A Directed Graph and an Undirected Graph. Circles represent nodes, and lines or arrows connecting nodes are edges.

2.1.2 Edges

- Edges are the elements that connect nodes.
- In social graphs, edges indicate inter-node relationships and are frequently called *relationships* or *(social) ties*.
- The set of edges is usually represented as:

$$E = \{e_1, e_2, \ldots, e_m\}$$

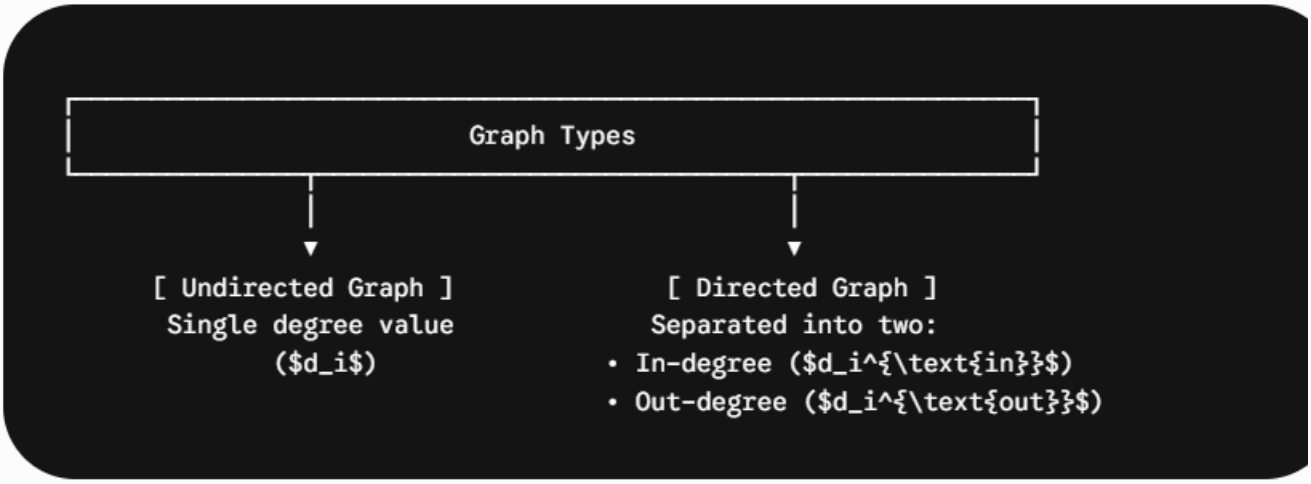
  where e_i (for $1 \leq i \leq m$) represents a single edge.
- The size of the edge set is commonly denoted as $m = |E|$.
- Edges are often represented by their endpoints; for example, $e(v_1, v_2)$ or simply (v_1, v_2) defines an edge between node v_1 and node v_2.
- **Undirected Edges:** These indicate that nodes are connected in both directions. In this case, $e(v_1, v_2)$ is exactly the same as $e(v_2, v_1)$, and a graph comprised of these is called an *undirected graph*.
- **Directed Edges:** These have specific directions, meaning a connection goes from one node to another but not necessarily vice versa. Here, $e(v_1, v_2)$ is not the same as $e(v_2, v_1)$. Directed edges are drawn using arrows starting at v_i and ending at v_j, and they form a *directed graph*.

- **Loops / Self-links:** These are edges that start and end at the exact same node, represented mathematically as $e(v_i, v_i)$.
- **Neighborhoods:**
  - In an undirected graph, the set of nodes connected to a specific node v_i via an edge is called its *neighborhood*, denoted as $N(v_i)$.
  - In a directed graph, neighborhoods are split into *incoming neighbors* $N_{\text{in}}(v_i)$ (nodes that point/connect to v_i) and *outgoing neighbors* $N_{\text{out}}(v_i)$ (nodes that v_i points/connects to).

2.1.3 Degree and Degree Distribution

Core Concepts of Node Degree

- **Degree (d_i):** The number of edges connected to a single node v_i.
- **Social Media Interpretation:**
  - **Facebook:** The degree represents the total number of friends a user has.
  - **Twitter:** Graph interactions are directed, separating relationships into **in-degree** (number of followers) and **out-degree** (number of followees).



Directed Edges

In directed graphs, edges have directionality, leading to two types of degrees:

- **In-degree (d_i^{in}):** Edges pointing toward the node.
- **Out-degree (d_i^{out}):** Edges pointing away from the node.

Social Media Analogy:

- **Facebook (Undirected):** Degree represents a user's total number of friends.
- **Twitter (Directed):** In-degree represents followers; out-degree represents followees.

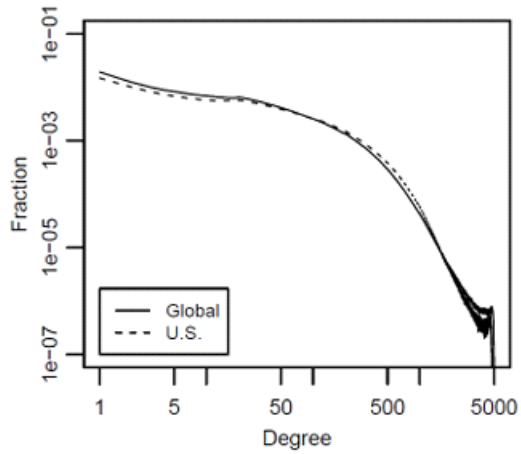


Figure 2.3: Facebook Degree Distribution for the US and Global Users. There exist many users with few friends and a few users with many friends. This is due to a power-law degree distribution.

Key Theorems & Lemmas

Theorem 2.1

In any undirected graph, the summation of all node degrees is equal to twice the number of edges.

$$\sum_i d_i = 2|E|$$

- **Proof:** Every edge has two endpoints. When calculating degrees d_i and d_j for connected nodes v_i and v_j, the shared edge contributes 1 to both d_i and d_j. Removing an edge reduces d_i and d_j by 1, decreasing the overall sum $\sum_k d_k$ by 2. Total removal of all m edges reduces the sum by $2m$, bringing it to zero. Thus, the degree summation is $2 \times m = 2|E|$.

Lemma 2.1

In an undirected graph, there is an even number of nodes having an odd degree.

- **Proof:** Because the total sum of degrees is even ($2|E|$), subtracting the sum of even-degree nodes from this total must yield an even value. Therefore, the sum of degrees for nodes with odd degrees must be even, implying an even number of odd-degree nodes exist.

Lemma 2.2

In any directed graph, the summation of in-degrees is equal to the summation of out-degrees.

$$\sum_i d_i^{\text{out}} = \sum_j d_j^{\text{in}}$$

Degree Distribution

In massive graphs, analyzing the distribution of node degrees (**degree distribution**) is critical for understanding network topologies.

Let p_d (also written as $P(d)$ or $P(d_v = d)$) represent the probability that a randomly selected node v has a degree of exactly d. Since it is a probability distribution, it satisfies:

$$\sum_{d=0}^{\infty} p_d = 1$$

In a graph containing n total nodes, p_d is calculated as:

$$p_d = \frac{n_d}{n}$$

Where n_d equals the number of nodes with degree d.

Visualizing Degree Distribution

A common procedure is to plot a histogram of the degree distribution using the following axes:

- **X-axis:** The degree (d).
- **Y-axis:** Can represent either the absolute number of nodes having that degree (n_d) or the fraction/probability of nodes having that degree (p_d).

Concrete Examples

Example 2.1 (Small Network Application)

For a sample graph with $n = 7$ nodes, where four nodes have a degree of 2, and degrees 1, 3, and 4 are each observed exactly once, the degree distribution p_d for $d = \{1, 2, 3, 4\}$ is calculated as:

$$p_1 = \frac{1}{7}, \quad p_2 = \frac{4}{7}, \quad p_3 = \frac{1}{7}, \quad p_4 = \frac{1}{7}$$

Example 2.2 (Real-World Social Network Application)

On real-world social networking sites, friendship networks exhibit a specific trend known as a **power-law degree distribution**. As seen in historical datasets (such as Facebook user data from May 2012):

- There exists a massive number of users with very few friends (low degree).
- There exists only a small, handful of users with a massive number of friends (very high degree).

Graph Representation and Subgraphs

Any graph G can be mathematically represented as an ordered pair:

$$G(V, E)$$

Where:

- V is the node set.
- E is the edge set.

Since all edges are structurally bounded between existing nodes, the edge set satisfies the relation:

$$E \subseteq V \times V$$

Subgraphs

For any base graph $G(V, E)$, an altered graph $G'(V', E')$ is defined as a formal **subgraph** of $G(V, E)$ if and only if the following subset properties hold true:

1. The subgraph's nodes must be a subset of the original nodes:

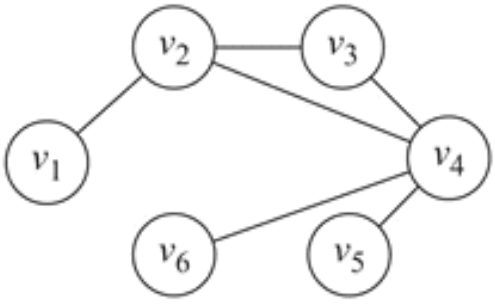
$$V' \subseteq V$$

2. The subgraph's edges must be a subset of the original edges that exist strictly between the chosen subset of nodes V':

$$E' \subseteq (V' \times V') \cap E$$

2.2 Graph Representation

16 July 2026 08:21



(a) Graph

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	1	0	0	0	0
v_2	1	0	1	1	0	0
v_3	0	1	0	1	0	0
v_4	0	1	1	0	1	1
v_5	0	0	0	1	0	0
v_6	0	0	0	1	0	0

(b) Adjacency Matrix

Figure 2.4: A Graph and Its Corresponding Adjacency Matrix.

2.2 Graph Representation

While visual representations of graphs are intuitive for humans, they cannot be directly utilized or manipulated by computers or standard mathematical tools. To resolve this, we require graph representations that satisfy three key criteria:

1. **Preserve Information:** The representation must store the full node and edge sets without losing any information.
2. **Computational Efficiency:** It must be easily stored and manipulated by computer systems.
3. **Mathematical Operability:** It should allow mathematical methods and formulas to be applied directly and seamlessly.

Adjacency Matrix

An **adjacency matrix** (also widely referred to as a **sociomatrix**) offers a natural and direct mathematical representation of a graph.

- **Binary Representation:** In its basic form, the presence or absence of an edge between two nodes is represented as follows:
  - A value of **1** indicates a connection between nodes v_i and v_j.
  - A value of **0** denotes that there is no connection between the two nodes.
- **Generalized Weights:** The matrix can be generalized by replacing binary entries with any real number to indicate the **strength or weight** of the connection between two nodes.
- **Diagonal Entries:** The elements along the main diagonal of the matrix represent **self-links** or **loops** (where a node connects directly back to itself).
- **Sparsity:** In many real-world applications, such as social networks, individuals interact with only a fraction of the total network. As a result, most cells in the matrix remain zero. This forms a **sparse matrix**—a matrix populated primarily with zeros.

Formalization

The adjacency matrix A can be mathematically formalized as:

$$A_{i,j} = \begin{cases} 1 & \text{if } v_i \text{ is connected to } v_j, \\ 0 & \text{otherwise.} \end{cases}$$

Adjacency List

In an **adjacency list**, every node in the graph is associated with a dedicated list containing all the other nodes to which it is directly connected.

- **Ordering:** The elements within each neighbor list are often sorted based on node order or another preferred structural baseline.
- **Space Savings:** Because it completely skips storing non-existent connections (zeros), this representation is significantly more space-efficient than an adjacency matrix when working with sparse networks.

Example Adjacency List

Node	Connected To
v_1	v_2
v_2	v_1, v_3, v_4
v_3	v_2, v_4
v_4	v_2, v_3, v_5, v_6
v_5	v_4
v_6	v_4

Edge List

The **edge list** is another straightforward, common approach for storing large graphs. Instead of mapping out a full matrix or keeping an array of linked lists, it simply records an explicit collection of all edges present in the graph.

- **Structure:** Each element in the list represents a single distinct edge and is written as a coordinate pair: (v_i, v_j), which explicitly denotes that node v_i is connected to node v_j.

Example Edge List

- (v_1, v_2)
- (v_2, v_3)
- (v_2, v_4)
- (v_3, v_4)
- (v_4, v_5)
- (v_4, v_6)

💡 Key Takeaway: Space Efficiency in Real-World Networks

Large-scale social media networks are characteristically **sparse**. When using an **adjacency matrix**, an enormous amount of computational memory is wasted storing zeros for non-existent connections. Both the **adjacency list** and **edge list** representations bypass this issue entirely by only tracking active relationships, saving a significant amount of memory space.

2.3 Types of Graphs

16 July 2026 08:23

2.3 Types of Graphs

In general, there are many basic types of graphs. This section outlines the primary classifications based on their structural properties, edge directions, weights, and signs.

Null Graph

A **null graph** is a graph where the node (vertex) set is entirely empty. Because there are no nodes, there are consequently no edges.

- **Formal Definition:**

$$G(V, E), \quad V = E = \emptyset$$

Empty Graph

An **empty graph** (also known as an *edgeless graph*) is a graph where the edge set is empty, but the node set can be non-empty.

- **Formal Definition:**

$$G(V, E), \quad E = \emptyset$$

- **Key Distinction:** A null graph is always an empty graph, but an empty graph is not necessarily a null graph.

Directed / Undirected / Mixed Graphs

Graphs are often classified by whether their edges possess a specific direction.

- **Directed Graphs:** Consist exclusively of directed edges.
  - Between any two nodes i and j, two distinct edges can exist: one pointing from i to j and another from j to i.
  - **Adjacency Matrix:** Generally asymmetric, meaning a connection from i to j does not imply a connection from j to i ($A_{i,j} \neq A_{j,i}$).
  - *Social Media Example:* **Twitter**, where follower relationships are unidirectional (following someone does not mean they follow you back).
- **Undirected Graphs:** Consist exclusively of undirected edges.
  - Only one edge can exist between any pair of nodes i and j.
  - **Adjacency Matrix:** Always symmetric ($A = A^T$).
  - *Social Media Example:* **Facebook**, where friendship is mutual and bidirectional.
- **Mixed Graphs:** Contain a combination of both directed and undirected edges.

Simple Graphs and Multigraphs

Graphs can also be categorized by the maximum number of allowable edges connecting the same pair of nodes.

- **Simple Graphs:** Graphs where only a single edge can exist between any pair of nodes.
- **Multigraphs:** Graphs where multiple parallel edges can exist between the same two nodes.
  - **Adjacency Matrix:** Can include values greater than 1 to represent the exact number of edges between node pairs.
  - *Social Media Example:* Two individuals who share multiple distinct types of relationships simultaneously (e.g., being friends, colleagues, and group members at the same time), where each relationship type adds a new edge.

Weighted Graphs

A **weighted graph** associates a numerical value (weight) with each edge to represent properties like distance, cost, or capacity.

- **Formal Definition:** Represented as $G(V, E, W)$, where W is the set of weights associated with each edge, satisfying:

$$|W| = |E|$$

- **Adjacency Matrix:** Instead of binary $1/0$ values, the matrix entries store the actual weight values. It combines E and W into a single matrix A, assuming an edge exists between v_i and v_j if and only if $W_{ij} \neq 0$.
- **Notation:** The weight can be written as W_{ij}, w_{ij}, or $w(i, j)$.
- *Example (Web Graph):* A directed representation of internet sites where nodes are websites and edge weights denote the number of hyperlinks linking them. These graphs can contain loops (links pointing back to the self) and multiple links.

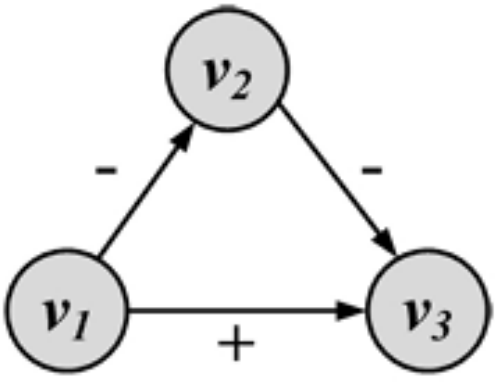


Figure 2.5: A Signed Graph Example.

Signed Edges and Signed Graphs

A **signed graph** is a specialized type of weighted graph that utilizes binary weights, typically represented as positive ($+$) or negative ($-$) signs.

- **Applications:** Commonly used to model contradictory behaviors or dualistic relationships.
  - **Friends and Foes:** A positive ($+$) edge denotes friendship or alliance, while a negative ($-$) edge denotes animosity or conflict.
  - **Social Status:** A positive ($+$) edge can signify higher social status or rank relative to the target node. For example, a school principal pointing to a student with a directed $+$ edge reflects their higher institutional rank.
- **Directionality in Signed Graphs:**
  - *Directed:* One node may view the other as a friend or foe unilaterally (non-reciprocal).
  - *Undirected:* The relationship of alliance or animosity is mutually shared between both endpoints.

Visual Structural Example:
A network can form a triad using signed, directed edges. For instance:

- Node v_1 points to v_3 with a positive ($+$) edge.
- Node v_1 points to v_2 with a negative ($-$) edge.
- Node v_2 points to v_3 with a negative ($-$) edge.

2.4 Connectivity in Graphs

16 July 2026 08:26

2.4 Connectivity in Graphs

Connectivity defines how nodes are connected via a sequence of edges in a graph. Before detailing connectivity, several preliminary concepts must be defined.

Adjacent Nodes and Incident Edges

- **Adjacent Nodes:** Two nodes v_1 and v_2 in a graph $G(V, E)$ are *adjacent* when they are directly connected via an edge:

$$v_1 \text{ is adjacent to } v_2 \equiv e(v_1, v_2) \in E$$

- **Incident Edges:** Two edges $e_1(a, b)$ and $e_2(c, d)$ are *incident* when they share exactly one endpoint (i.e., they are connected via a common node):

$$e_1(a, b) \text{ is incident to } e_2(c, d) \equiv (a = c) \vee (a = d) \vee (b = c) \vee (b = d)$$

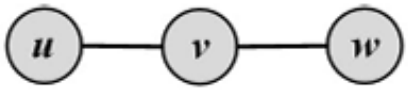


Figure 2.6: Adjacent Nodes and Incident Edges. In this graph u and v, as well as v and w, are adjacent nodes, and edges (u, v) and (v, w) are incident edges.

Directed Graph Constraint: In a directed graph, two edges are incident only if the ending node of one edge matches the beginning node of the other edge (the edge directions must match consecutively).

Traversing an Edge

An edge in a graph can be *traversed* by starting at one of its end-nodes, moving along the edge, and stopping at its other end-node.

- For an edge $e(a, b)$ connecting nodes a and b, visiting e can start at a and end at b.
- In an **undirected graph**, traversal can alternately start at b and end at a.

Walk, Path, Trail, Tour, and Cycle

- **Walk:** A sequence of incident edges traversed one after another. It can be represented as a sequence of nodes $v_1, v_2, v_3, \ldots, v_n$, traversing the edges $e_1(v_1, v_2), e_2(v_2, v_3), \ldots, e_{n-1}(v_{n-1}, v_n)$.
  - **Length of a walk:** The total number of edges traversed during the walk (in this case, $n - 1$).
  - **Open Walk:** A walk where the starting node and ending node are different ($v_1 \neq v_{n+1}$).
  - **Closed Walk:** A walk that returns to its starting point ($v_1 = v_{n+1}$).
- **Trail:** A walk where **no edge** is traversed more than once (all edges in the walk are unique).
  - **Tour (or Circuit):** A closed trail (starts and ends at the same node with no repeated edges).

- **Path:** A walk where **both nodes and edges are distinct** (no repetitions allowed).
  - **Cycle:** A closed path.
  - **Directed Path:** In directed graphs, edge traversal is strictly restricted to follow the defined direction of the edges.

Special Global Structures

- **Hamiltonian Cycle:** A cycle that visits **every node** in the graph exactly once and returns to the starting node.
- **Eulerian Tour:** A tour that traverses **every edge** in the graph exactly once. (Note: A single node can be visited multiple times).
- **Random Walk:** A walk performed on a weighted graph where subsequent nodes are chosen randomly based on transition probabilities. The weight of an edge defines its traversal probability. For a random walk to work correctly, the weights of all outbound edges starting at node v_i must satisfy:

$$\sum_x w_{i,x} = 1, \quad \forall i, j \text{ where } w_{i,j} \geq 0$$

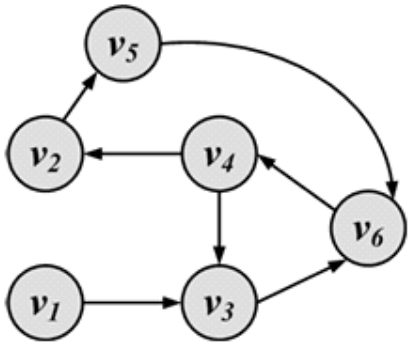
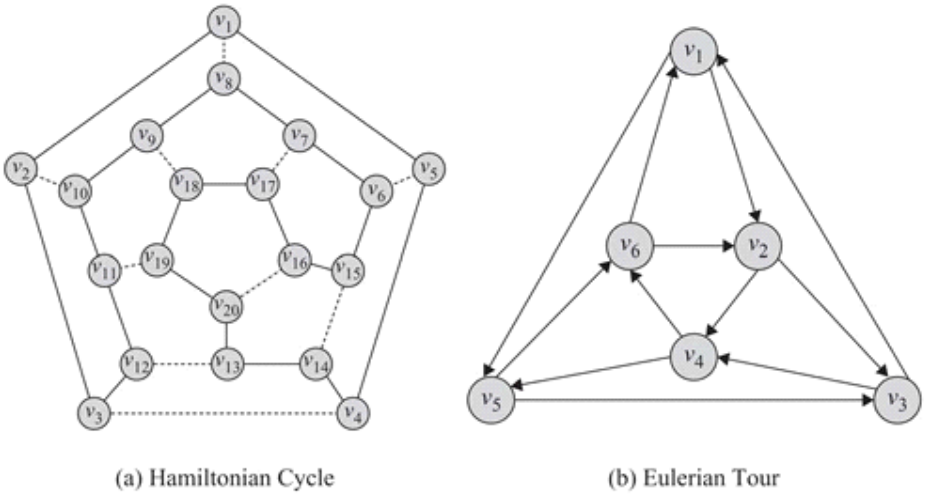


Figure 2.7: Walk, Path, Trail, Tour, and Cycle. In this figure, v_4, v_3, v_6, v_4, v_2 is a walk; v_4, v_3 is a path; v_4, v_3, v_6, v_4, v_2 is a trail; and v_4, v_3, v_6, v_4 is both a tour and a cycle.



(a) Hamiltonian Cycle (b) Eulerian Tour

Figure 2.8: Hamiltonian Cycle and Eulerian Tour. In a Hamiltonian cycle we start at one node, visit all other nodes only once, and return to our start node. In an Eulerian tour, we traverse all edges only once and return to our start point. In an Eulerian tour, we can visit a single node multiple times. In this figure, $v_1, v_5, v_3, v_1, v_2, v_4, v_6, v_2, v_3, v_4, v_5, v_6, v_1$ is an Eulerian tour.

Algorithm 2.1 Random Walk

Require: Initial Node v_0, Weighted Graph $G(V, E, W)$, Steps t

return Random Walk P
state = 0;
$v_t = v_0$;
$P = \{v_0\}$;
while state < t **do**
    state = state + 1;

    select a random node v_j adjacent to v_t with probability $w_{t,j}$;
    $v_t = v_j$;
    $P = P \cup \{v_j\}$;
end while
Return P

Connectivity

- **Connected Nodes:** A node v_i is *connected* to (or *reachable* from) v_j if it is adjacent to it or if there exists a valid path spanning from v_i to v_j.
- **Connected Graph:** An undirected graph is connected if a valid path exists between *any* pair of nodes within it.
- **Weakly Connected Graph:** A directed graph where a path exists between any pair of nodes *only after* replacing all directed edges with undirected edges (ignoring edge directions).
- **Strongly Connected Graph:** A directed graph where a directed path exists between *any* pair of nodes, strictly respecting edge directions.

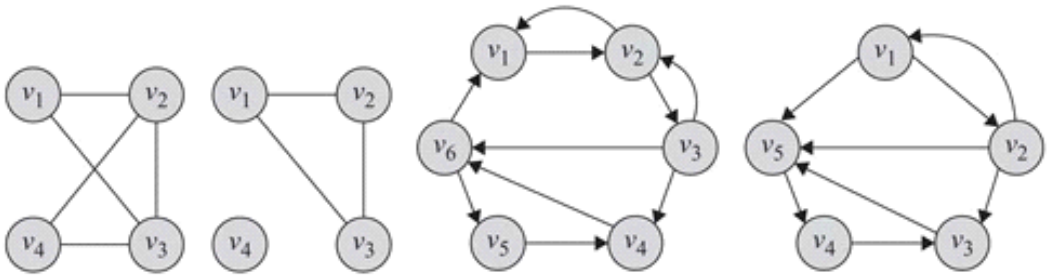


Figure 2.9: Connectivity in Graphs.

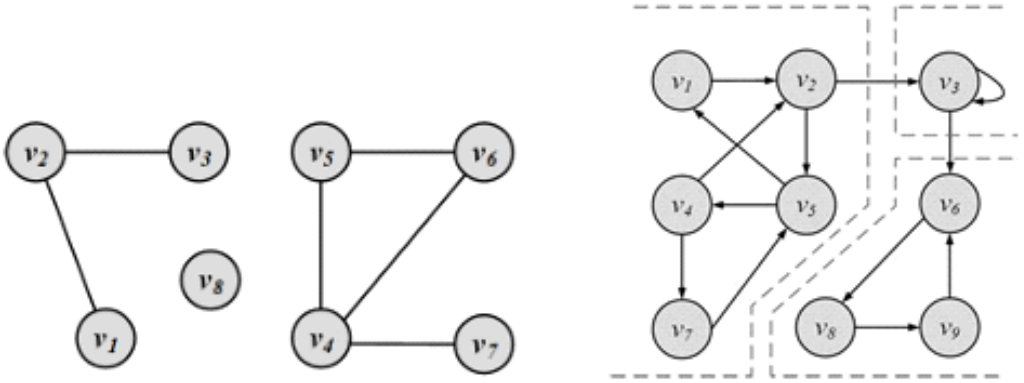


Figure 2.10: Components in Undirected and Directed Graphs.

Components

A **component** is an isolated subgraph that maintains a specific degree of connectivity:

Graph Type	Component Definition
Undirected Graph	A subgraph where a path exists between every pair of nodes inside it.
Directed Graph (Strongly Connected Component)	A subgraph where, for every pair of nodes u and v, there exists a directed path from v to u **and** a directed path from u to v.
Directed Graph (Weakly Connected Component)	A subgraph that becomes a single connected component only after replacing all its directed edges with undirected ones.

Shortest Path

When a graph is connected, multiple paths can exist between a given pair of nodes. The path that possesses the minimum length is termed the **shortest path**.

- **Applications:** Common in optimization systems like GPS routing.
- **Notation:** The length of the shortest path between nodes v_i and v_j is denoted as $l_{i,j}$.
- **n-hop Neighborhood:** A generalization using shortest paths. The n-hop neighborhood of a node v_i is the exact set of nodes whose shortest path distance to v_i is less than or equal to n:

$$\text{Distance} \leq n$$

Diameter

The **diameter** of a graph is the length of the longest shortest path between any pair of nodes within that graph.

- **Constraint:** This metric is defined **only** for connected graphs, as the shortest path between nodes in a disconnected graph does not exist.
- **Formal Definition:** For a graph G, the diameter is formulated as:

$$\text{diameter}_G = \max_{(v_i, v_j) \in V \times V} l_{i,j}$$

2.5 Special Graphs

16 July 2026 08:34

2.5 Special Graphs

Using general graph concepts, many special categories of graphs can be defined to model specific computational and mathematical problems.

2.5.1 Trees and Forests

- **Tree:** A special case of an undirected graph structure that contains **no cycles**. In a tree, there is exactly one unique path between any pair of nodes.
- **Forest:** A graph structure consisting of a set of disconnected trees.
- **Edge Property:** For any tree containing $|V|$ nodes, the number of edges $|E|$ is always:

$$|E| = |V| - 1$$

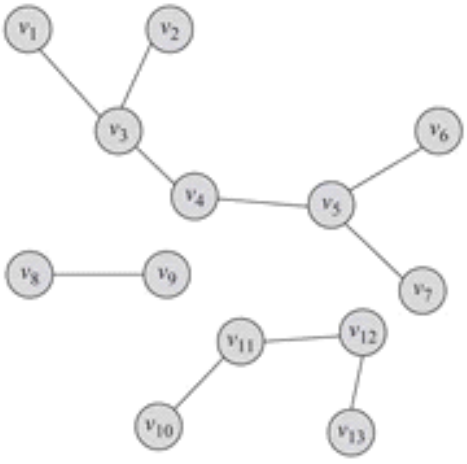


Figure 2.11: A Forest Containing Three Trees.

2.5.2 Special Subgraphs

Certain subgraphs are frequently utilized in network analysis due to their distinct structural properties.

Spanning Tree

- For any connected graph, a **spanning tree** is a subgraph that forms a tree structure and includes **all** the nodes of the original graph.
- If the original graph is not already a tree, its spanning tree will include all nodes but only a subset of the edges.
- A single graph can contain multiple distinct spanning trees.
- For a weighted graph, the total weight of a spanning tree is the summation of all its edge weights.
- **Minimum Spanning Tree (MST):** Out of all possible spanning trees for a weighted graph, the one that yields the minimum total edge weight is the MST.
    - *Example Application:* Optimizing a road network between cities. If nodes represent cities and edge weights represent geographical distances or construction costs, finding the MST provides a solution that minimizes total road mileage while guaranteeing a connected path exists between every pair of cities.

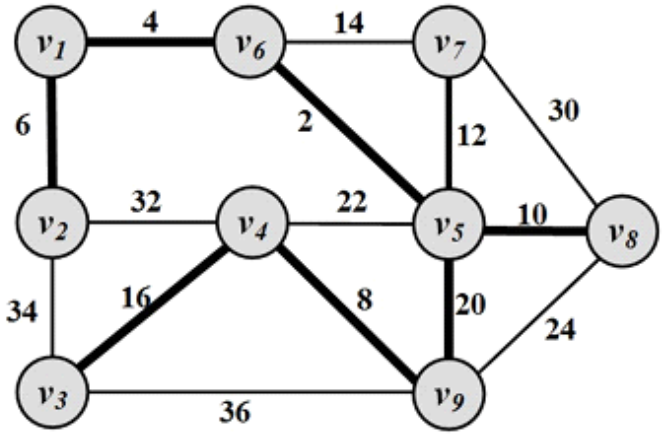


Figure 2.12: Minimum Spanning Tree. Nodes represent cities and values assigned to edges represent geographical distance between cities. Highlighted edges are roads that are built in a way that minimizes their total length.

Steiner Tree

- Given a weighted graph $G(V, E, W)$ and a specific subset of nodes $V' \subseteq V$ (known as **terminal nodes**), the Steiner tree problem aims to find a tree that spans all nodes in V' such that the total weight of the tree is minimized.
- **Key Difference from MST:** Unlike the MST problem, a Steiner tree **does not** need to span all nodes (V) in the entire graph; it only needs to span the specified subset of terminal nodes (V'). It may include other non-terminal nodes (called Steiner vertices) if they help minimize the overall path length.

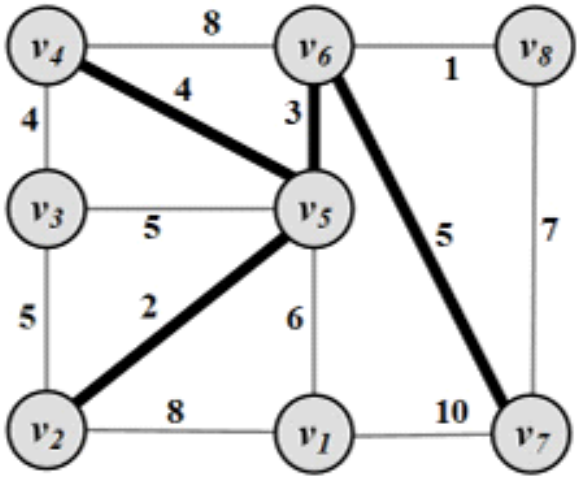


Figure 2.13: Steiner tree for $V' = \{v_2, v_4, v_7\}$.

2.5.3 Complete Graphs

- A **complete graph** is a graph where all possible edges exist for a given set of nodes V. Every distinct pair of nodes is directly connected by an edge.
- The total number of edges $|E|$ in a complete graph is calculated using the combination formula:

$$|E| = \binom{|V|}{2}$$

- Complete graphs containing n nodes are formally denoted by the notation K_n (e.g., K_1, K_2, K_3, K_4).

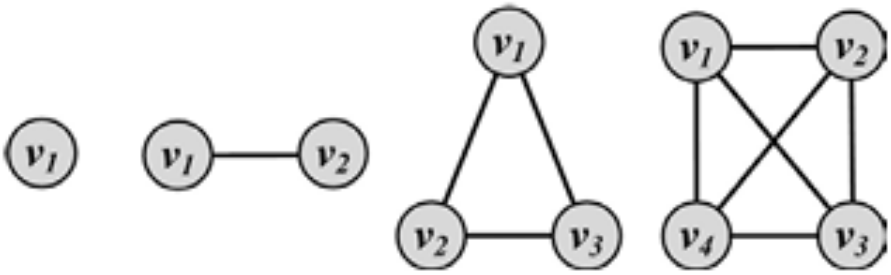
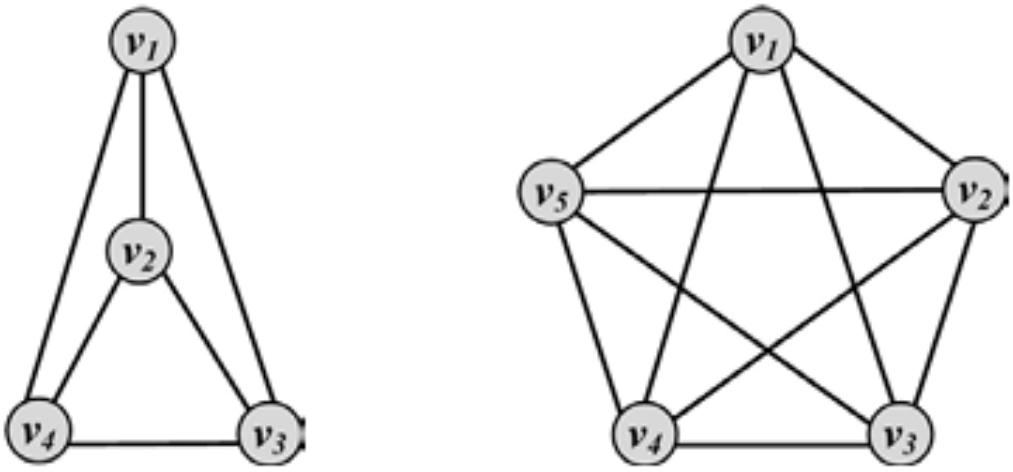


Figure 2.14: Complete Graphs K_i for $1 \leq i \leq 4$.



(a) Planar Graph (b) Non-planar Graph

Figure 2.15: Planar and Nonplanar Graphs.

2.5.4 Planar Graphs

- **Planar Graph:** A graph that can be drawn on a two-dimensional plane in such a way that no two edges cross each other (edges intersect only at their endpoints).
- **Nonplanar Graph:** A graph that cannot be drawn without at least two of its edges crossing over one another.

2.5.5 Bipartite Graphs

A **bipartite graph** $G(V, E)$ is a graph where the total vertex set V can be partitioned into two distinct, independent sets such that every edge connects a node in one set to a node in the other set. No edges are allowed between nodes belonging to the same set.

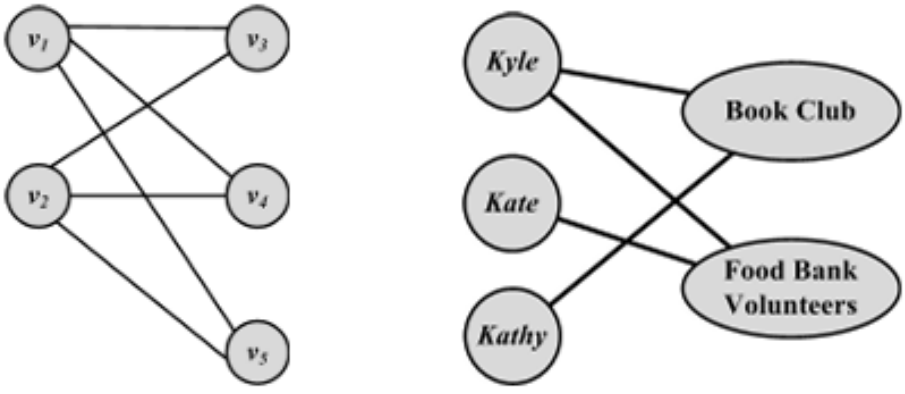
Formally, a bipartite graph satisfies the following conditions:

$$V = V_L \cup V_R$$

$$V_L \cap V_R = \emptyset$$

$$E \subseteq V_L \times V_R$$

(Where V_L and V_R represent the left and right node partitions, respectively).



(a) Bipartite Graph (b) Affiliation Network

Figure 2.16: Bipartite Graphs and Affiliation Networks.

Affiliation Networks

- In social media analysis, **affiliation networks** serve as common real-world examples of bipartite graphs.
- Nodes in one partition (e.g., V_L) represent **individuals**, while nodes in the opposing partition (e.g., V_R) represent **affiliations** or group categories (e.g., a Book Club, Food Bank Volunteers).
- An edge is drawn between an individual node and an affiliation node if and only if that individual is associated with or participates in that specific group.

2.5.6 Regular Graphs

- **Definition:** A **regular graph** is a graph in which all nodes have the exact same degree.
- **General Classification:**
  - A regular graph where all nodes have a degree of 2 is specifically called a **2-regular graph**.
  - More generally, any graph where all nodes have a degree of k is designated as a **k-regular graph**.
- **Key Properties & Examples:**
  - **Connectivity:** Regular graphs can be either connected or disconnected.
  - **Complete Graphs:** These serve as prime examples of regular graphs. In a complete graph containing n nodes, every single node has a degree of $n - 1$ (expressed as $k = n - 1$).
  - **Cycles:** Cycle graphs are inherently regular graphs where the degree of each node is $k = 2$.

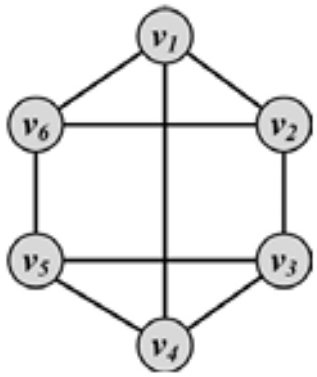


Figure 2.17: Regular Graph with $k = 3$.

2.5.7 Bridges

- **Definition:** In a graph that contains one or more connected components, **bridges** are the specific edges whose removal will directly increase the total number of connected components.
- **Core Characteristics:**
  - **Functional Role:** As the name implies, these edges act as critical links connecting different components or sections within a graph.
  - **Impact of Removal:** Eliminating a bridge edge results in the immediate disconnection of formerly linked components, thereby dividing the graph into a higher number of isolated components.

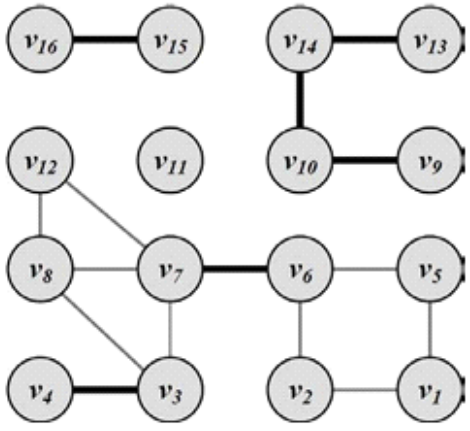


Figure 2.18: Bridges. Highlighted edges represent bridges; their removal will increase the number of components in the graph.

2.6 Graph Algorithms

16 July 2026 08:38

2.6 Graph Algorithms

This section reviews foundational graph algorithms, which represent a vital subset of the extensive methods applied to graph structures.

2.6.1 Graph/Tree Traversal

Traversal algorithms are designed to systematically visit nodes within graphs or special subgraphs, such as trees.

- **Real-World Application:** Consider surveying a social media platform to compute the average age of its users. The typical approach starts at a single user and applies a traversal technique to browse their friends, their friends' friends, and so on.
- **Core Guarantees:**
  1. All reachable nodes (users) are visited.
  2. No node (user) is visited more than once.

Depth-First Search (DFS)

Depth-First Search (DFS) explores as deep as possible along each branch before backtracking to explore alternative paths.

- **Mechanism:** It starts at a node v_i, selects an unvisited neighbor $v_j \in N(v_i)$, and recursively performs DFS on v_j before visiting the remaining neighbors in $N(v_i)$.
- **Formal Priority:** Let a node v_i have neighbors $v_j, v_k \in N(v_i)$, and let $v_{j(1)}, v_{j(2)} \in N(v_j)$ be neighbors of v_j such that $v_i \neq v_{j(1)} \neq v_{j(2)}$. DFS visits v_j next, followed by its deeper neighbors $v_{j(1)}$ and $v_{j(2)}$, before backtracking to visit the closer neighbor v_k.
- **Data Structure:** Utilizes a **Stack (S)** to manage unvisited nodes in a depth-first manner.

Algorithm 2.2 Depth-First Search (DFS)

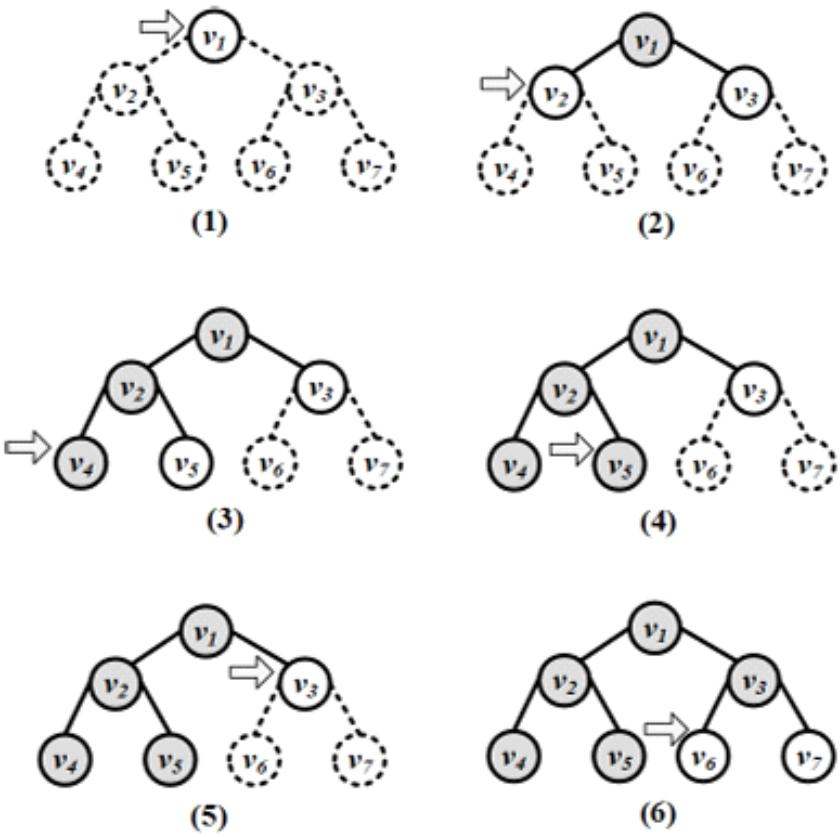
Require: Initial node v, graph/tree $G(V, E)$, stack S

return An ordering on how nodes in G are visited
Push v into S;
$visitOrder = 0$;
while S not empty **do**
  $node$ = pop from S;
  if $node$ not visited **then**
    $visitOrder = visitOrder + 1$;
    Mark $node$ as visited with order $visitOrder$; `//or print` $node$
    Push all neighbors/children of $node$ into S;
  end if
end while
Return all nodes with their visit order.

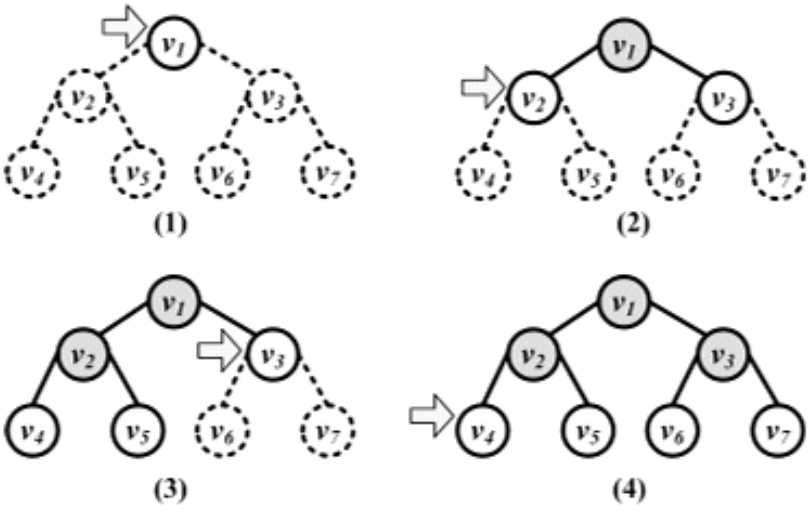
Breadth-First Search (BFS)

Breadth-First Search (BFS) explores a graph level by level, analyzing immediate proximity before expanding outward.

- **Mechanism:** Starts from a designated node, visits all of its immediate neighbors first, and then moves to the second level by traversing the neighbors of those neighbors.
- **Data Structure:** Utilizes a **Queue (Q)** to achieve breadth-based traversal.
- **Social Media Context:** BFS is typically preferred over DFS when analyzing local networks because immediate neighbors (e.g., direct friends) are usually more relevant to visit first.



(a) Depth-First Search (DFS)



(b) Breadth-First Search (BFS)

Figure 2.19: Graph Traversal Example.

Algorithm 2.3 Breadth-First Search (BFS)

Require: Initial node v, graph/tree $G(V, E)$, queue Q

```
return An ordering on how nodes are visited
Enqueue v into queue Q;
visitOrder = 0;
while Q not empty do
    node = dequeue from Q;
    if node not visited then
        visitOrder = visitOrder +1;
        Mark node as visited with order visitOrder; //or print node
        Enqueue all neighbors/children of node into Q;
    end if
end while
```

2.6.2 Shortest Path Algorithms

Shortest path algorithms find the path minimizing total edge cost between nodes.

- **Navigation Systems:** Used to compute the optimal route from a source city to a destination city over a weighted network of roads.
- **Social Media Mining:** Used to measure network diameter (the longest shortest path between any two nodes) to evaluate how tightly connected a social network is.

Dijkstra's Algorithm

Developed in 1959 by Edsger Dijkstra, this single-source shortest path algorithm is designed for **weighted graphs with non-negative edges**. It finds the shortest paths and corresponding path lengths from a starting node s to all other nodes.

Algorithm 2.4 Dijkstra's Shortest Path Algorithm

Require: Start node s, weighted graph/tree $G(V, E, W)$

return Shortest paths and distances from s to all other nodes.
for $v \in V$ **do**
  $distance[v] = \infty$;
  $predecessor[v] = -1$;
end for
$distance[s] = 0$;
$unvisited = V$;
while $unvisited \neq \emptyset$ **do**
  $smallest = arg\,min_{v \in unvisited}\, distance(v)$;
  if $distance(smallest) == \infty$ **then**
    break;
  end if
  $unvisited = unvisited \setminus \{smallest\}$;
  $currentDistance = distance(smallest)$;
  for adjacent node to $smallest$: $neighbor \in unvisited$ **do**
    $newPath = currentDistance + w(smallest, neighbor)$;
    if $newPath < distance(neighbor)$ **then**
      $distance(neighbor) = newPath$;
      $predecessor(neighbor) = smallest$;
    end if
  end for
end while
Return $distance[]$ and $predecessor[]$ arrays

Step-by-Step Execution Logic:

1. **Node Selection:** All nodes begin as unvisited. The unvisited node with the absolute minimum tentative shortest path value is selected and designated as $smallest$.
2. **Neighbor Relaxation:** The algorithm examines all unvisited neighbors of $smallest$. It checks if the shortest path to each neighbor can be improved by routing through $smallest$ by evaluating the condition:

$$newPath < distance(neighbor)$$

   where $newPath = distance(smallest) + w(smallest, neighbor)$.
3. **Path and Predecessor Updates:** If an improvement is found, the value in $distance[neighbor]$ is updated, and $smallest$ is saved into $predecessor[neighbor]$. The `predecessor` array allows the shortest path sequence to be reconstructed recursively.
4. **Finalizing Visited Status:** A node is marked visited once all its neighbors are fully processed. Its minimum distance and optimal incoming path are locked and will not change.

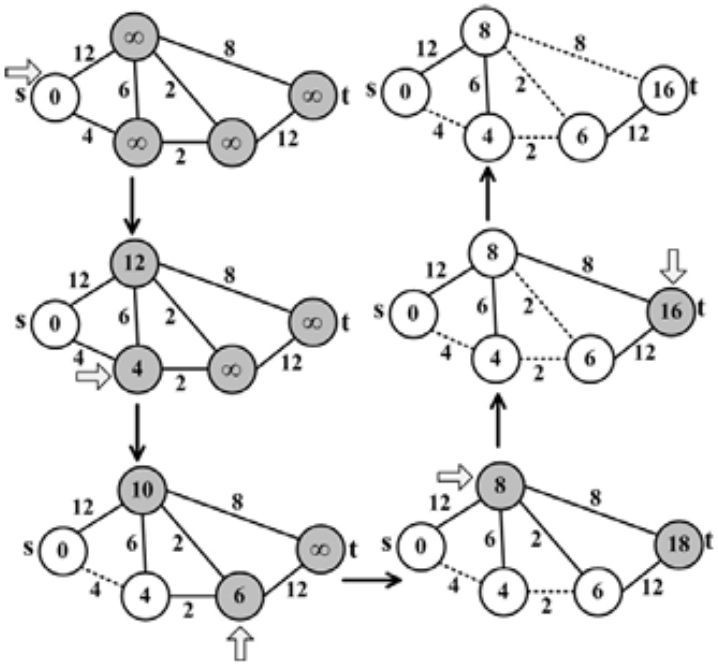


Figure 2.20: Dijkstra's Algorithm Execution Example. The shortest path between node s and t is calculated. The values inside nodes at each step show the best shortest path distance computed up to that step. An arrow denotes the node being analyzed.

2.6.3 Minimum Spanning Trees

A **spanning tree** of a connected undirected graph is a tree that includes all the nodes in the original graph and a subset of its edges. Spanning trees are highly useful in real-life applications, such as:

- **Infrastructure Layout:** A cable company wanting to lay wires to cover all designated areas (nodes) while minimizing the overall cost of wiring (the summation of edge weights).
- **Social Media Mining:** A network of individuals who need to receive a piece of information. The information spreads via friends with an associated cost to pass it between any two nodes; a minimum spanning tree provides the minimum total cost needed to inform everyone in the network.

There are various algorithms available to find a Minimum Spanning Tree (MST) in a weighted graph, with **Prim's algorithm** being one of the most famous.

Prim's Algorithm

Prim's algorithm is a greedy algorithm used to construct an MST for a connected, weighted neighborhood network.

- **Mechanism:** It begins with a single random node in the spanning tree tree structure. It then iteratively expands the tree by evaluating all valid edges that have exactly one endpoint inside the current tree and one endpoint outside it. Among these boundary edges, the edge with the absolute minimum weight is selected and appended to the tree along with its external node. This step is repeated until all nodes are spanned.

Prim's Algorithm

Prim's algorithm is a greedy approach used to discover the MST of a weighted graph. It works through the following iterative steps:

1. **Initialization:** Select a random node from the graph and add it to the spanning tree.
2. **Growth:** Expand the spanning tree by evaluating all edges that have exactly one endpoint in the existing spanning tree and the other endpoint among the unselected nodes.
3. **Selection:** Out of these candidate edges, choose the one with the **minimum weight** and add it (along with its external endpoint) to the spanning tree.
4. **Termination:** Repeat this process iteratively until every node in the graph is fully spanned.

Algorithm 2.5 Prim's Algorithm

Require: Connected weighted graph $G(V, E, W)$

1: **return** Spanning tree $T(V_s, E_s)$
2: $V_s = \{$a random node from $V\}$;
3: $E_s = \{\}$;
4: **while** $V \neq V_s$ **do**
5: $\quad e(u, v) = argmin_{(u,v),u \in V_s, v \in V - V_s} w(u, v)$
6: $\quad V_s = V_s \cup \{v\}$;
7: $\quad E_s = E_s \cup e(u, v)$;
8: **end while**
9: Return tree $T(V_s, E_s)$ as the minimum spanning tree;

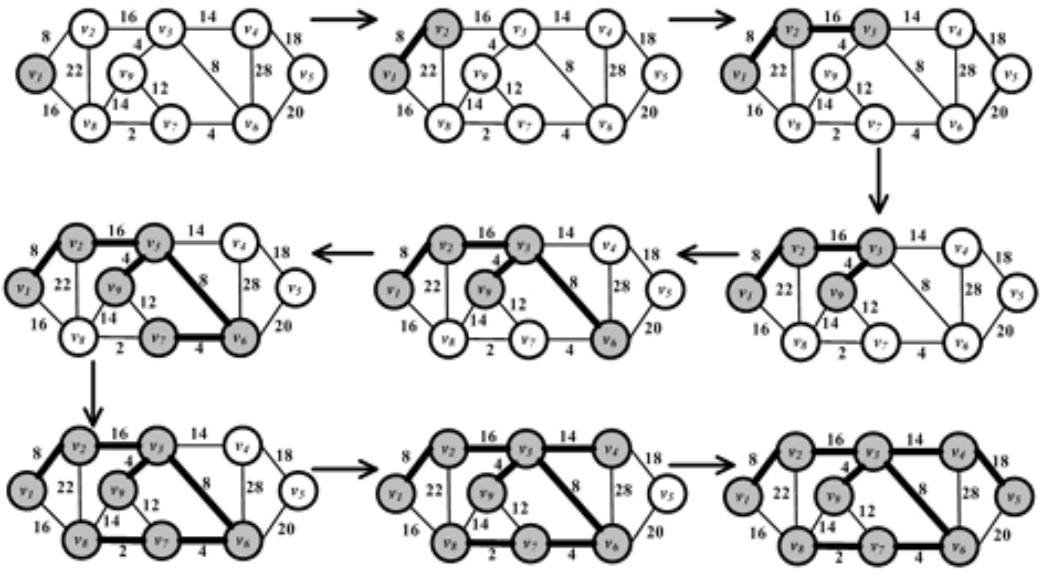


Figure 2.21: Prim's Algorithm Execution Example.

2.6.4 Network Flow Algorithms

Network flow algorithms model the movement of resources through a network. The classic analogy is a system of pipes connecting an infinite water source to a water sink. Given the specific volume capacities of these pipes, the fundamental question is: **What is the maximum flow that can be sent from the source to the sink?**

This concept translates to many fields outside of fluid dynamics, such as:

- **Social Media Limits:** On platforms where users have daily limits (**capacity**) on sending messages (**flow**) to others, network flow algorithms help determine the maximum throughput of messages the entire network must be prepared to handle at any given moment.

Flow Network

A flow network denoted as $G(V, E, C)$ is a directed, weighted graph where the capacity C replaces the traditional weight designation W. It is defined by the following characteristics:

- $\forall e(u, v) \in E, c(u, v) \geq 0$ defines the **edge capacity**.
- When $(u, v) \in E$, then $(v, u) \notin E$ (meaning opposite flow along the same edge is impossible).
- The network contains a designated **source node** (s), which has an infinite supply of flow connected to it.
- The network contains a designated **sink node** (t), where the flow terminates.

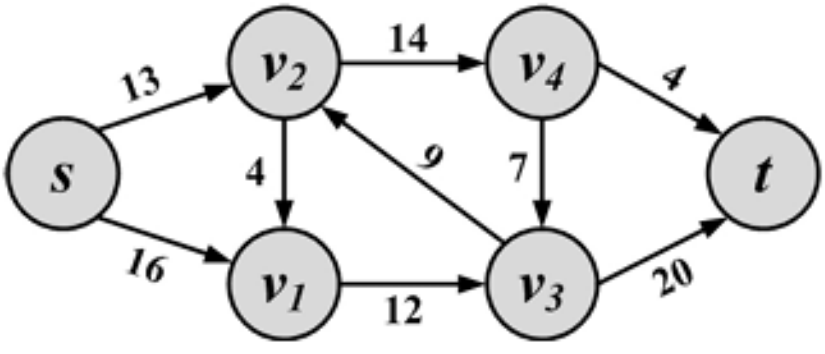


Figure 2.22: A Sample Flow Network.

Flow

Given a network with predefined edge capacities, these edges can be filled with flow up to their maximum capacity thresholds.

- **Visual Notation:** To clearly visualize a network graph, an edge with capacity c and current flow f is commonly written using the notation f/c.

Mathematically, a valid flow must satisfy the following formal properties and constraints:

- **Flow Definition:** $\forall (u,v) \in E, f(u,v) \geq 0$ defines the actual flow passing through that edge.
- **Capacity Constraint:** The flow traversing an edge can never exceed its defined limit:

$$\forall (u,v) \in E, \quad 0 \leq f(u,v) \leq c(u,v)$$

- **Flow Conservation Constraint:** To ensure no flow is lost, the total flow entering any intermediate node must perfectly equal the total flow exiting it. This applies to all nodes except the source (s) and the sink (t):

$$\forall v \in V, v \notin \{s,t\}, \quad \sum_{k:(k,v)\in E} f(k,v) = \sum_{l:(v,l)\in E} f(v,l)$$

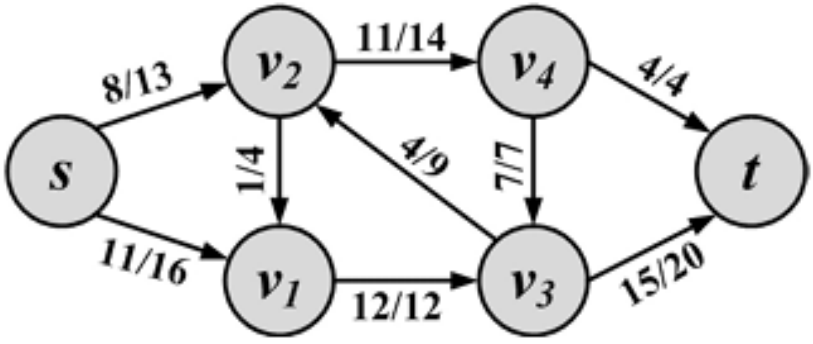


Figure 2.23: A Sample Flow Network with Flows and Capacities.

Flow Quantity

The **flow quantity** (or value of the flow) in any network represents the net amount of flow moving from the source to the sink. It is defined as the total outgoing flow from the source minus any incoming flow to the source.

Alternatively, it can be computed by subtracting the outgoing flow from the sink from its total incoming flow value:

$$flow = \sum_v f(s,v) - \sum_v f(v,s) = \sum_v f(v,t) - \sum_v f(t,v)$$

Example:
For a sample flow network with an outgoing source flow of $11 + 8$ and 0 incoming flow, the calculation is:

$$flow = \sum_v f(s,v) - \sum_v f(v,s) = (11+8) - 0 = 19$$

The ultimate objective in a flow network is to find the flow assignments to each edge that yield the **maximum flow quantity**. This is solved using a maximum flow algorithm, such as the well-established **Ford-Fulkerson algorithm**.

Ford-Fulkerson Algorithm

Core Intuition

1. Find a valid path from the source (s) to the sink (t) such that every edge along the path has unused capacity.
2. Identify the **minimum unused capacity** among all edges on this path.
3. Use that minimum capacity value to increase (augment) the overall flow.
4. Iterate this process until no other paths with available capacity can be found.

Algorithm 2.6 Ford-Fulkerson Algorithm

Require: Connected weighted graph $G(V, E, W)$, Source s, Sink t

1: **return** A Maximum flow graph
2: $\forall (u, v) \in E, f(u, v) = 0$
3: **while** there exists an augmenting path p in the residual graph G_R **do**
4: Augment flows by p
5: **end while**
6: Return flow value and flow graph;

The Residual Network

Before formalizing the process, a few key concepts must be established. Given a flow network $G(V, E, C)$, we can define a corresponding network called the **residual network**, denoted as $G_R(V, E_R, C_R)$. This network keeps track of how much capacity remains in the original network.

- **Edge Existence:** The residual network contains an edge between nodes u and v if and only if either (u, v) or (v, u) exists in the original graph. If one of these edges exists originally, we obtain **two** edges in the residual network: one from (u, v) and one from (v, u).
- **Forward Edges:** If there is no flow currently passing through an edge in the original network, a residual capacity equal to the full original capacity of the edge remains.
- **Backward Edges:** The residual network introduces the ability to send flow in the opposite direction. This effectively allows the algorithm to "undo" or cancel out a certain amount of flow previously sent through the original network.

The **residual capacity** $c_R(u, v)$ for any edge (u, v) within the residual graph is formally computed as:

$$c_R(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E \\ f(v, u) & \text{if } (u, v) \notin E \end{cases}$$

Note: In visual representations of a residual network, edges that possess a residual capacity of zero are typically omitted.

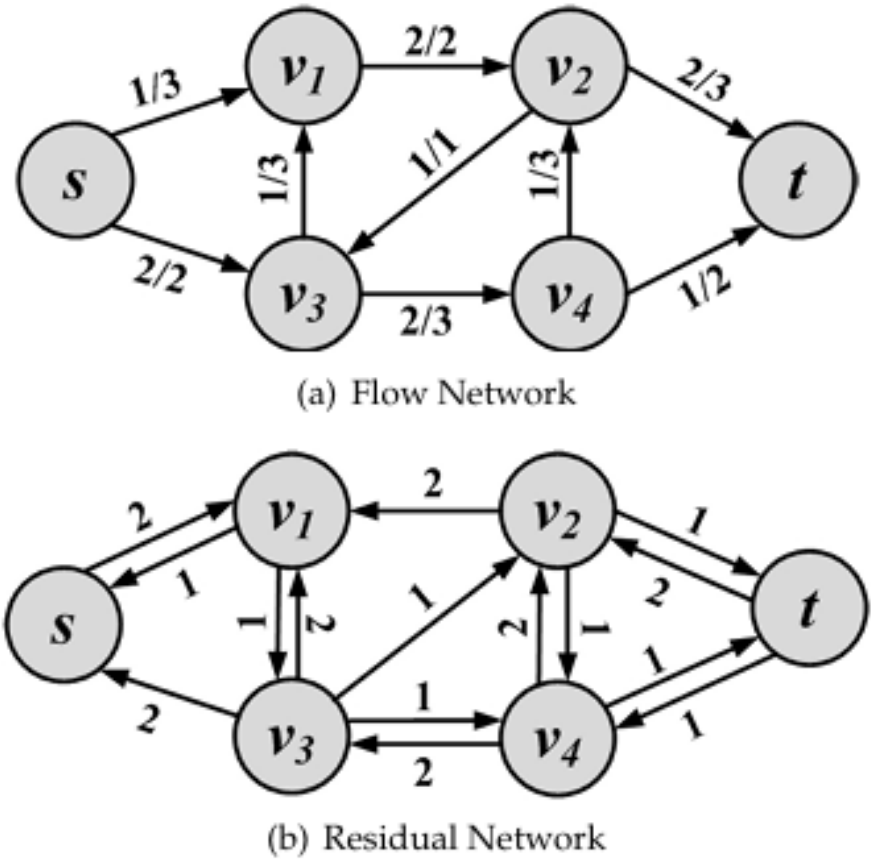


Figure 2.24: A Flow Network and Its Residual.

Augmentation and Augmenting Paths

The residual graph alters how capacity is interpreted depending on the direction of its edges relative to the original graph:

- **Same Direction:** Shows how much **more** flow can be pushed forward along that edge in the original network.
- **Opposite Direction:** Shows how much flow can be **pushed back** (canceled out) on the original edge.

By finding a valid path through the residual network, we can **augment** (increase) the total flow in the original graph.

Key Definitions

- **Augmenting Path:** Any simple path from the source s to the sink t within the residual graph. Because all active capacities in the residual graph are strictly positive, these paths are guaranteed to increase the original flow.
- **Bottleneck Edge:** The maximum amount of flow that can be pushed along an augmenting path is strictly bounded by the **minimum capacity** found along that path. This limiting edge is often referred to as the *weak link*.

Flow Augmentation Formula

Given a flow $f(u, v)$ in the original graph, and residual flows $f_R(u, v)$ and $f_R(v, u)$, the flow can be updated using the following formula:

$$f_{\text{augmented}}(u, v) = f(u, v) + f_R(u, v) - f_R(v, u)$$

Example:
Consider an tracking path through a residual network: s, v_1, v_3, v_4, v_2, t. If the minimum capacity along this path is 1, the overall flow quantity will increase by 1. Once the original graph is augmented with this path, a new residual graph is generated. This cycle continues until no further augmenting paths can be located in the residual graph.

Summary of execution

The Ford-Fulkerson algorithm systematically locates these augmenting paths in the residual network and continuously updates the flows in the original network to achieve maximum flow. Finding these paths during execution can be successfully achieved using standard graph traversal algorithms, such as **Breadth-First Search (BFS)**.

2.6.5 Maximum Bipartite Matching

Suppose we are trying to solve the following problem in social media:

Given n products and m users such that some users are only interested in certain products, find the maximum number of products that can be bought by users.

Graph Formulation

This problem can be reformulated as a **bipartite graph problem** $G(V, E)$:

- **Nodes (V):** Divided into two distinct sets:
  - V_L: Left node set representing **products**.
  - V_R: Right node set representing **users**.
  - Total vertices: $V = V_L \cup V_R$
- **Edges (E):** Represent the interest of users in specific products.
- **Matching (M):** A subset of edges ($M \subset E$) such that each node in V appears in at most one edge in M. A node is either matched (appears in an edge $e \in M$) or not.
- **Maximum Bipartite Matching (M_{Max}):** A matching such that, for any other matching M' in the graph:

$$|M_{\text{Max}}| \geq |M'|$$

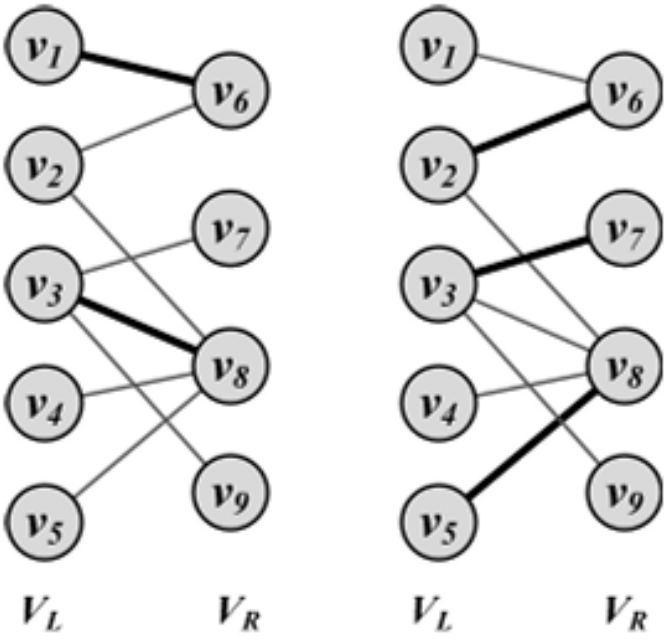


Figure 2.26: Maximum Bipartite Matching.

Algorithm 2.6 Ford-Fulkerson Algorithm

Require: Connected weighted graph $G(V, E, W)$, Source s, Sink t

1: **return** A Maximum flow graph
2: $\forall (u, v) \in E, f(u, v) = 0$
3: **while** there exists an augmenting path p in the residual graph G_R **do**
4: Augment flows by p
5: **end while**
6: Return flow value and flow graph;

Solving via Ford-Fulkerson Maximum Flow

The maximum bipartite matching problem can be solved by converting the bipartite graph $G(V, E)$ into a flow graph $G'(V', E', C)$ using the following procedure:

1. **Define New Vertices:** Add a source node s and a sink node t:

$$V' = V \cup \{s\} \cup \{t\}$$

2. **Establish Directed Edges:** Connect the source to the left partition and the right partition to the sink:

$$E' = E \cup \{(s, v) \mid v \in V_L\} \cup \{(v, t) \mid v \in V_R\}$$

3. **Assign Capacities:** Set a uniform unit capacity for all edges in the network:

$$c(u, v) = 1, \quad \forall (u, v) \in E'$$

By solving the maximum flow for this constructed network, the maximum matching is obtained. This is because maximizing the network flow directly maximizes the number of bottleneck unit edges utilized between V_L and V_R.

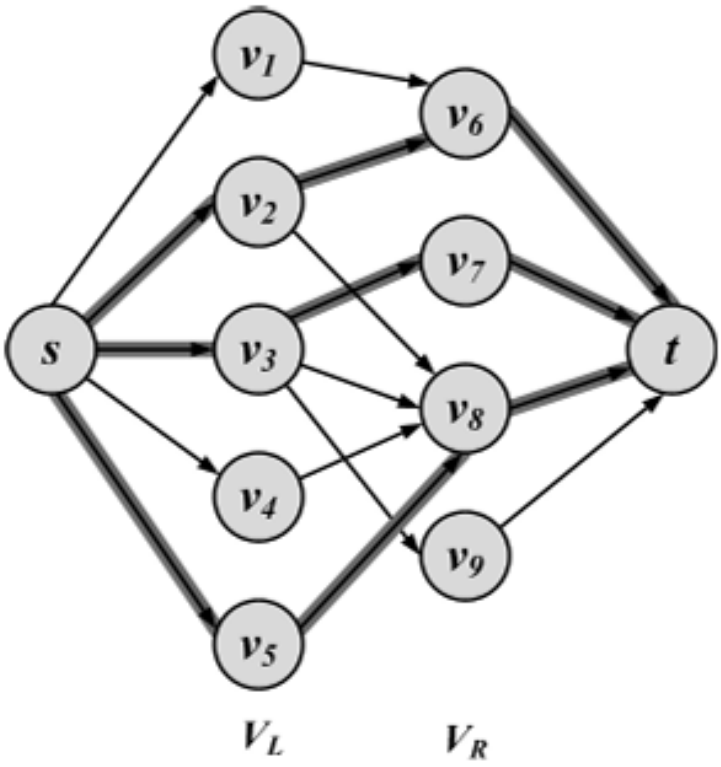


Figure 2.27: Maximum Bipartite Matching Using Max Flow.

2.6.6 Bridge Detection

Bridges (also known as **cut edges**) are edges within a graph whose removal changes a formerly connected component into two disconnected components.

Implementation Concept

While highly efficient algorithms exist for this task, a direct and intuitive brute-force approach involves testing edges sequentially:

1. Remove edges from the graph one by one.
2. Verify whether the removal causes the connected components to become disconnected.
3. Analyze connectedness using standard graph traversal algorithms like **Breadth-First Search (BFS)** or **Depth-First Search (DFS)**. If an edge $e(u, v)$ is removed and a traversal starting at node u fails to reach node v, the component has been split, identifying e as a bridge.

Algorithm 2.7 Bridge Detection Algorithm

Require: Connected graph $G(V, E)$

```
return Bridge Edges
bridgeSet = {}
for e(u, v) ∈ E do
    G' = Remove e from G
    Disconnected = False;
    if BFS in G' starting at u does not visit v then
        Disconnected = True;
    end if
    if Disconnected then
        bridgeSet = bridgeSet ∪ {e}
    end if
end for
Return bridgeSet
```

2.7 Summary

This summary provides a comprehensive overview of the fundamental concepts, structures, types, connectivity characteristics, and algorithms associated with graph theory as covered in the chapter.

Graph Fundamentals & Representations

Graphs are built using fundamental building blocks: **nodes** (vertices) and **edges** (links). Key basic properties include the **degree** of a node (the number of edges connected to it) and the **degree distribution** across the entire graph.

To compute and process graphs, they must be represented using data structures. The three well-established techniques are:

- **Adjacency Matrix:** A 2D array representation.
- **Adjacency List:** An array of lists tracking adjacent vertices.
- **Edge List:** A simple list containing all the edges in the graph.

Note on Social Networks: Social networks are typically sparse. Because of this sparsity, both **adjacency lists** and **edge lists** are significantly more efficient and save substantial space compared to an adjacency matrix.

Types of Graphs

Graphs can be categorized based on their structural rules and edge properties:

- **Null and Empty Graphs:** Graphs containing no edges (or no nodes and edges).
- **Directed, Undirected, and Mixed Graphs:** Depending on whether edges have a specific direction, no direction, or a combination of both.
- **Simple Graphs vs. Multigraphs:** Simple graphs contain no self-loops or multiple edges between the same two nodes, whereas multigraphs can have multiple parallel edges.
- **Weighted Graphs:** Graphs where edges are assigned numerical values.
    - *Example:* **Signed graphs** are a specific type of weighted graph used to represent contradictory behaviors (using positive or negative weights).

Connectivity, Components, and Special Graphs

Understanding how nodes are linked is crucial to analyzing graph behavior:

1. Paths and Movements

- **Walks, Trails, Paths, Tours, and Cycles:** Various ways of moving through a graph depending on whether nodes or edges are allowed to repeat.
- **Diameter:** The longest shortest path between any pair of nodes in the graph.

2. Structural Components

- **Components:** Connected subgraphs within a larger graph. These can be categorized into **strongly connected** and **weakly connected** components depending on the reachability in directed graphs.
- **Bridges:** Single-point-of-failure edges. Removing a bridge will turn a previously connected graph into a disconnected one.

3. Special Graph Structures

- **Complete Graphs:** Graphs where every single node is directly connected to all other nodes.
- **Regular Graphs:** Graphs where all nodes share the exact same degree.
- **Trees:** A specific type of graph containing no cycles.
  - *Spanning Tree:* A subgraph that includes all the nodes of the original graph but contains no cycles.
  - *Steiner Tree:* A tree that connects a designated subset of vertices, potentially utilizing additional intermediate vertices.
- **Bipartite Graphs:** Graphs whose nodes can be completely partitioned into two distinct sets such that edges only exist *between* the two sets, never *inside* the individual sets.
  - *Example:* **Affiliation networks** are a primary real-world example of bipartite graphs.

Key Graph Algorithms

Graph algorithms are specialized techniques utilized to query, manipulate, and extract optimal paths or configurations from a graph:

Algorithm Category	Purpose / Function	Key Examples Mentioned
Traversal Algorithms	Provides a structured ordering of the nodes. Useful for checking graph connectivity or generating paths.	General traversal techniques
Shortest Path Algorithms	Finds the path containing the absolute minimum edge length/weight between two nodes.	**Dijkstra's algorithm**
Spanning Tree Algorithms	Isolates subgraphs that connect all nodes while selecting edges that sum up to a minimum value.	**Prim's algorithm**
Maximum Flow Algorithms	Evaluates the maximum flow capacity passing through a weighted capacity network.	**Ford-Fulkerson algorithm**
Bipartite Matching	Solves matching problems within partitioned networks; directly applied via maximum flow concepts.	**Maximum bipartite matching**
Bridge Detection	Identifies critical single-point-of-failure edges within a network.	Simple bridge detection solutions

2.9 Exercises

16 July 2026 08:56

Question 1

Given a directed graph $G(V, E)$ and its adjacency matrix A, we propose two methods to make G undirected,

$$A'_{ij} = \min(1, A_{ij} + A_{ji})$$

$$A'_{ij} = A_{ij} \times A_{ji}$$

where $A'_{i,j}$ is the (i, j) entry of the undirected adjacency matrix. Discuss the advantages and disadvantages of each method.

Answer

Method 1: $A'_{ij} = \min(1, A_{ij} + A_{ji})$

- **Description:** An undirected edge is established between nodes i and j if there is a directed edge in **at least one** direction ($i \to j$ OR $j \to i$).
- **Advantages:**
  - **Preserves Connectivity:** It retains all existing relational paths from the original directed graph, ensuring that weakly connected structures remain intact.
  - **Maximizes Information Retention:** No structural connection is completely lost.
- **Disadvantages:**
  - **Loss of Reciprocity Details:** It treats a one-way connection identically to a mutual, two-way connection.
  - **Noise Amplification:** If unidirectional edges represent accidental or weak interactions, this method artificially promotes them to full undirected partnerships.

Method 2: $A'_{ij} = A_{ij} \times A_{ji}$

- **Description:** An undirected edge is established between nodes i and j if and only if there is a **mutual/reciprocal** directed edge ($i \to j$ AND $j \to i$).
- **Advantages:**
  - **High-Confidence Criteria:** It filters out unidirectional noise and ensures only strong, confirmed, reciprocal relationships are kept.
  - **Symmetry Emphasis:** Highly suitable for applications where true interactions must be mutual (e.g., symmetric communications or strong social bonds).
- **Disadvantages:**
  - **Severe Information Loss:** It discards all one-way relationships. If the original graph contains few reciprocal paths, the resulting undirected graph will become extremely sparse or completely disconnected, losing vital structural context.

Question 2

Is it possible to have the following degrees in a graph with 7 nodes?

$$\{4, 4, 4, 3, 5, 7, 2\}$$

Answer

No, it is **not possible** for two distinct reasons:

1. **Handshaking Lemma Contradiction:**
   According to the Handshaking Lemma, the sum of all node degrees in any graph must be an **even** number because each edge contributes exactly 2 to the total sum.

   $$\text{Sum of degrees} = 4 + 4 + 4 + 3 + 5 + 7 + 2 = 29$$

   Since 29 is an **odd** number, such a graph cannot exist structurally.
2. **Maximum Degree Violation (for Simple Graphs):**
   In a simple graph (no loops or multiple edges) containing 7 nodes, the maximum possible degree for any single node is $7 - 1 = 6$ (a node can connect to at most the remaining 6 nodes). The given sequence includes a degree of **7**, which is impossible.

Question 3

Given the following adjacency matrix, compute the adjacency list and the edge list.

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Answer

Assuming standard **1-based indexing** (where nodes are labeled 1 through 9):

1. Adjacency List

An adjacency list maps each node to a list of its direct neighbors:

- **Node 1:** `[2, 3]`
- **Node 2:** `[1, 3]`
- **Node 3:** `[1, 2, 4, 5]`
- **Node 4:** `[3, 5, 6, 7]`
- **Node 5:** `[3, 4, 6, 7]`
- **Node 6:** `[4, 5, 7, 8]`
- **Node 7:** `[4, 5, 6, 8]`
- **Node 8:** `[6, 7, 9]`
- **Node 9:** `[8]`

(Note: For 0-based indexing, subtract 1 from each node identifier).

2. Edge List

An edge list contains all unique undirected pairs of connected nodes (formatted as `(u, v)` where `u < v`):

Edge Number	Connected Nodes
1	`(1, 2)`
2	`(1, 3)`
3	`(2, 3)`
4	`(3, 4)`
5	`(3, 5)`
6	`(4, 5)`
7	`(4, 6)`
8	`(4, 7)`
9	`(5, 6)`
10	`(5, 7)`
11	`(6, 7)`
12	`(6, 8)`
13	`(7, 8)`
14	`(8, 9)`

Question 4

Prove that $|E| = |V| - 1$ in trees.

Answer

We can prove this property by mathematical induction on the number of vertices, $|V| = n$.

- **Base Case ($n = 1$):**
  A tree with 1 vertex ($|V| = 1$) has 0 edges ($|E| = 0$).

  $$|E| = 1 - 1 = 0$$

  The base case holds true.

- **Inductive Hypothesis:**
  Assume that the statement holds for all trees with fewer than n vertices, where $n > 1$. That is, any tree with k vertices (where $k < n$) has exactly $k - 1$ edges.

- **Inductive Step:**
  Let T be a tree with n vertices. By definition, a tree is a connected graph with no cycles. Every finite tree with $n \geq 2$ vertices contains at least one **leaf** (a vertex with a degree of exactly 1).
  Let v be a leaf in T, and let e be the single edge connecting v to the rest of the tree. If we remove v and its incident edge e from T, we obtain a new graph $T' = T \setminus \{v\}$.
  Because we only removed a leaf, T' remains connected and acyclic, meaning T' is also a tree. The dimensions of T' are:

  $$|V'| = n - 1$$

  By applying the inductive hypothesis to T', the number of edges in T' is:

  $$|E'| = |V'| - 1 = (n - 1) - 1 = n - 2$$

  Since the original tree T has exactly one more edge than T' (the edge e that was removed), the total number of edges in T is:

  $$|E| = |E'| + 1 = (n - 2) + 1 = n - 1 = |V| - 1$$

Thus, by mathematical induction, the relation $|E| = |V| - 1$ is proven for all trees.

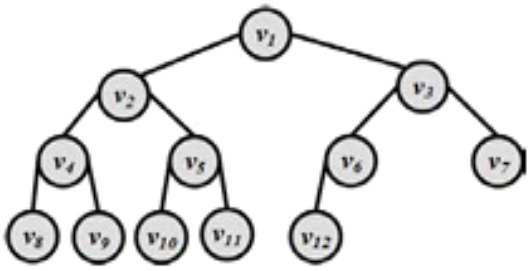


Figure 2.28: A Sample (Binary) Tree

Question 5

Consider the tree shown in Figure 2.28. Traverse the graph using both BFS and DFS and list the order in which nodes are visited in each algorithm.

Answer

Assuming conventional left-to-right processing of child nodes starting from the root node v_1:

- **Breadth-First Search (BFS) Order:**
  BFS explores the tree level by level, from top to bottom and left to right.

$$\mathbf{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9, v_{10}, v_{11}, v_{12}}$$

- **Depth-First Search (DFS) Order (Pre-order):**
  DFS explores as deep as possible along each branch before backtracking.

$$\mathbf{v_1, v_2, v_4, v_8, v_9, v_5, v_{10}, v_{11}, v_3, v_6, v_{12}, v_7}$$

Question 6

For a tree and a node v, under what condition is v visited sooner by BFS than DFS? Provide details.

Answer

A node v is visited sooner by BFS than DFS under the following condition:

The node v is located at a shallow depth (close to the root) but resides in a later-explored branch (e.g., a right subtree), while the earlier-explored branches (e.g., left subtrees) are very deep.

Details

- **BFS Behavior:** BFS explores nodes strictly by their depth level. It guarantees that every single node at depth d is visited before any node at depth $d+1$ is touched, regardless of which branch it belongs to.
- **DFS Behavior:** DFS dives completely down the leftmost available paths to maximum depth before backtracking. If the left subtrees are deep and complex, DFS spends significant time exploring deep nodes at levels $d+1, d+2, \ldots$ before it ever backtracks to visit a shallow node located in a right-hand branch.

Example from Figure 2.28: Look at node v_3 or v_7. BFS visits v_3 as the 3^{rd} node because it is at level 1. Conversely, DFS must completely exhaust the deep left subtrees of v_2 ($v_4, v_8, v_9, v_5, v_{10}, v_{11}$) before finally backtracking to v_3, making it the 9^{th} visited node.

Question 7

For a real-world social network, is BFS or DFS more desirable? Provide details.

Answer

BFS (Breadth-First Search) is significantly more desirable for real-world social networks.

Details

- **Shortest Paths & Degrees of Separation:** Social networks are primarily concerned with connections, such as finding mutual friends or determining degrees of separation (e.g., 1st, 2nd, or 3rd-degree connections on LinkedIn). Because BFS searches layer by layer, it naturally finds the shortest path in terms of hops in an unweighted social graph. DFS could wander deep down an endless chain of one person's distant followers without finding a close mutual connection.
- **Small-World Phenomenon:** Most social networks exhibit the "small-world" property, meaning the path length between any two random users is relatively small (often around 6 hops or fewer). BFS is highly efficient at discovering these tight local communities.
- **Neighborhood & Recommendation Queries:** Friend recommendation systems look at immediate local neighborhoods (friends of friends). With BFS, you can easily implement a **bounded-depth search** (e.g., terminate after depth 2 or 3) to find recommendations, which is difficult to restrict effectively using DFS.

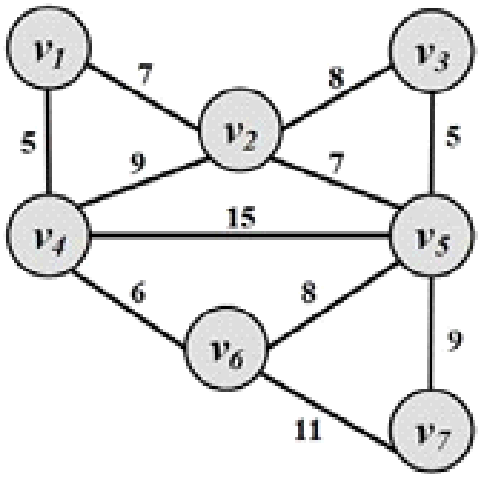


Figure 2.29: Weighted Graph.

Question 8

Compute the shortest path between any pair of nodes using Dijkstra's algorithm for the graph in Figure 2.29.

Answer

Running Dijkstra's algorithm from each vertex yields the following **Shortest Distance Matrix** between all pairs of nodes:

Source \ Destination	v_1	v_2	v_3
v_1	0	7	15
v_2	7	0	8
v_3	15	8	0
v_4	5	9	17
v_5	14	7	5
v_6	11	15	13
v_7	22	16	14

Exact Shortest Paths Table

- v_1 to v_2: Distance = **7** | Path: $v_1 \rightarrow v_2$
- v_1 to v_3: Distance = **15** | Path: $v_1 \rightarrow v_2 \rightarrow v_3$
- v_1 to v_4: Distance = **5** | Path: $v_1 \rightarrow v_4$
- v_1 to v_5: Distance = **14** | Path: $v_1 \rightarrow v_2 \rightarrow v_5$
- v_1 to v_6: Distance = **11** | Path: $v_1 \rightarrow v_4 \rightarrow v_6$
- v_1 to v_7: Distance = **22** | Path: $v_1 \rightarrow v_4 \rightarrow v_6 \rightarrow v_7$
- v_2 to v_3: Distance = **8** | Path: $v_2 \rightarrow v_3$
- v_2 to v_4: Distance = **9** | Path: $v_2 \rightarrow v_4$
- v_2 to v_5: Distance = **7** | Path: $v_2 \rightarrow v_5$
- v_2 to v_6: Distance = **15** | Path: $v_2 \rightarrow v_5 \rightarrow v_6$
- v_2 to v_7: Distance = **16** | Path: $v_2 \rightarrow v_5 \rightarrow v_7$
- v_3 to v_4: Distance = **17** | Path: $v_3 \rightarrow v_2 \rightarrow v_4$
- v_3 to v_5: Distance = **5** | Path: $v_3 \rightarrow v_5$
- v_3 to v_6: Distance = **13** | Path: $v_3 \rightarrow v_5 \rightarrow v_6$
- v_3 to v_7: Distance = **14** | Path: $v_3 \rightarrow v_5 \rightarrow v_7$
- v_4 to v_5: Distance = **14** | Path: $v_4 \rightarrow v_6 \rightarrow v_5$
- v_4 to v_6: Distance = **6** | Path: $v_4 \rightarrow v_6$
- v_4 to v_7: Distance = **17** | Path: $v_4 \rightarrow v_6 \rightarrow v_7$
- v_5 to v_6: Distance = **8** | Path: $v_5 \rightarrow v_6$
- v_5 to v_7: Distance = **9** | Path: $v_5 \rightarrow v_7$
- v_6 to v_7: Distance = **11** | Path: $v_6 \rightarrow v_7$

Question 9

Detail why edges with negative weights are not desirable for computing shortest paths using Dijkstra's algorithm.

Answer

Dijkstra's algorithm is a **greedy algorithm** that relies fundamentally on the assumption that adding an edge to a path can *never* decrease the total cost of that path. This assumption holds true only if all edge weights are non-negative (≥ 0).

When negative edge weights are introduced, the algorithm fails for two main reasons:

1. **Premature Finalization (Greedy Failure):**
   Dijkstra's algorithm permanently marks a node as "visited" once its current minimum distance is drawn from the priority queue. It assumes this distance can never be reduced further. If a negative edge exists later down a different branch, it could reveal a path that lowers the cost of an already-visited node. Because standard Dijkstra does not re-evaluate visited nodes, it will miss this update, yielding mathematically incorrect paths.
2. **Negative Cycles:**
   If the graph contains a loop whose total cumulative weight is negative, a path can traverse this loop infinitely to produce an infinitely small ($-\infty$) total cost. Dijkstra's algorithm lacks the capacity to detect or escape negative cycles, leading to faulty behavior or infinite loops.

Note: For graphs containing negative weights, alternative algorithms like the **Bellman-Ford algorithm** must be used instead.

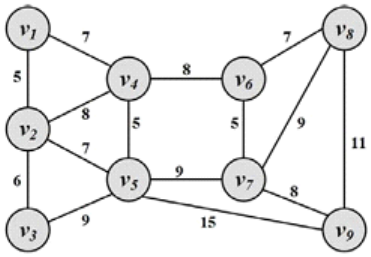


Figure 2.30: Weighted Graph

Question 10

Compute the minimal spanning tree using Prim's algorithm in the graph provided in Figure 2.30.

Solution:

Prim's algorithm builds the Minimum Spanning Tree (MST) by starting from an arbitrary vertex and successively adding the minimum-weight edge that connects a vertex in the tree to a vertex outside the tree.

Step	Connected Vertices (Vmst)	Available Edges to Unvisited Vertices	Chosen Edge	Edge Weight
1	{v1}	(v1,v2)=5, (v1,v4)=7	(v1,v2)	5
2	{v1,v2}	(v1,v4)=7, (v2,v3)=6, (v2,v4)=8, (v2,v5)=7	(v2,v3)	6
3	{v1,v2,v3}	(v1,v4)=7, (v2,v4)=8, (v2,v5)=7, (v3,v5)=9	(v1,v4)	7
4	{v1,v2,v3,v4}	(v4,v5)=5, (v2,v5)=7, (v4,v6)=8	(v4,v5)	5
5	{v1,v2,v3,v4,v5}	(v4,v6)=8, (v5,v7)=9, (v5,v9)=15	(v4,v6)	8
6	{v1,v2,v3,v4,v5,v6}	(v6,v7)=5, (v6,v8)=7, (v5,v7)=9	(v6,v7)	5
7	{v1,v2,v3,v4,v5,v6,v7}	(v6,v8)=7, (v7,v8)=9, (v7,v9)=8	(v6,v8)	7
8	{v1,v2,v3,v4,v5,v6,v7,v8}	(v7,v9)=8, (v8,v9)=11, (v5,v9)=15	(v7,v9)	8

Final MST Weight:

$$\text{Total Weight} = 5 + 6 + 7 + 5 + 8 + 5 + 7 + 8 = 51$$

Question 11

Compute the maximum flow in Figure 2.31.

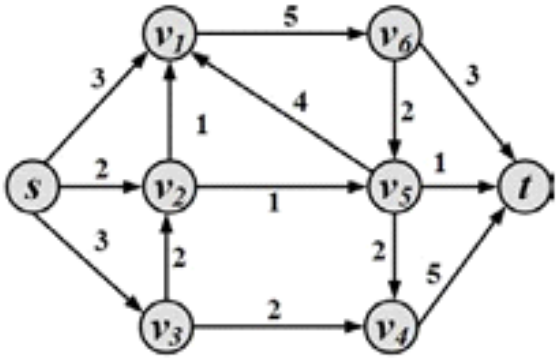


Figure 2.31: Flow Graph

Solution:

We can find the maximum flow from the source (s) to the sink (t) by identifying augmenting paths using the Ford-Fulkerson method:

1. **Path 1:** $s \to v_1 \to v_6 \to t$
   - $\text{Capacity} = \min(3, 5, 3) = 3$
2. **Path 2:** $s \to v_3 \to v_4 \to t$
   - $\text{Capacity} = \min(3, 2, 5) = 2$
3. **Path 3:** $s \to v_2 \to v_5 \to t$
   - $\text{Capacity} = \min(2, 1, 1) = 1$
4. **Path 4:** $s \to v_2 \to v_1 \to v_6 \to v_5 \to v_4 \to t$
   - $\text{Capacity} = \min(1, 1, 2, 2, 2, 3) = 1$

At this point, all paths out of s are fully saturated.

Verification via Min-Cut:

We can verify this result using the **Max-Flow Min-Cut Theorem**. Consider the cut separating the network into two sets: $S = \{s, v_2, v_3\}$ and $T = \{v_1, v_4, v_5, v_6, t\}$.
The directed edges crossing from S to T are:

- (s, v_1) with capacity 3
- (v_2, v_1) with capacity 1
- (v_2, v_5) with capacity 1
- (v_3, v_4) with capacity 2

$$\text{Capacity of Min-Cut} = 3 + 1 + 1 + 2 = 7$$

Thus, the **maximum flow is 7**.

Question 12

Given a flow network, you are allowed to change one edge's capacity. Can this increase the flow? How can we find the correct edge to change?

Solution:

- **Can this increase the flow?**
  Yes. If the total max flow of a network is limited by a bottleneck, increasing the capacity of a critical edge forming that bottleneck can increase the overall maximum flow.
- **How to find the correct edge to change:**
  1. **Find the Minimum Cut:** First, compute the maximum flow of the network. Identify the minimum cut (S, T) that partitions the vertices such that the source $s \in S$ and the sink $t \in T$.
  2. **Target Saturated Edges:** The only edges whose capacity increases can alter the max flow are the edges crossing from the S side to the T side. These edges are fully saturated (their current flow equals their capacity). Increasing the capacity of any edge *not* in a minimum cut yields zero increase in total flow.
  3. **Select the Proper Edge:** Choose a saturated edge (u, v) where $u \in S$ and $v \in T$ such that there is still an available path with remaining capacity from s to u and from v to t in the residual graph.

Question 13

How many bridges are in a bipartite graph? Provide details.

Solution:

A **bridge** (or cut-edge) is an edge whose removal increases the number of connected components in a graph. The number of bridges in a bipartite graph is not a fixed constant; it depends entirely on the graph's structure and number of vertices (n).

- **Minimum Number of Bridges = 0**
  - A bipartite graph contains no bridges if it contains cycles or is 2-edge-connected. For example, a simple even cycle (like C_4) or a complete bipartite graph $K_{m,n}$ (where $m, n \geq 2$) are bipartite and have exactly **0 bridges**.
- **Maximum Number of Bridges = $n - 1$**
  - By definition, every tree is an acyclic graph, meaning all trees are inherently bipartite. In any tree containing n vertices, there are exactly $n - 1$ edges, and **every single edge is a bridge**.

Conclusion: The number of bridges in a bipartite graph with n vertices can range anywhere from 0 **to** $n - 1$.

R2-Chapter 3 Network Measures

16 July 2026 09:14

Chapter 3: Network Measures

Introduction & Context

When analyzing social media, measuring different structural properties of a social network allows us to better understand the individuals embedded within it.

- **Real-World Example**: In February 2012, basketball star Kobe Bryant joined the Chinese microblogging site Sina Weibo, gaining over 100,000 followers within a few hours. In this scenario, the total number of followers served as a metric to *measure* his popularity and influence among Chinese social media users.
- **The Role of Metrics**: To systematically evaluate characteristics like popularity or influence, specific measures must be designed to quantify the structural properties of social networks.

Graph Representation in Social Media Mining

Social media networks are most frequently modeled using a **graph representation**. This graph captures:

- **Friendships** between users.
- **User interactions** within the platform.

These structural inputs are often mathematically represented via an **adjacency matrix**, from which various network measure values are computed.

Key Questions in Network Mining

By studying the graph representations of social media networks, data mining aims to answer three primary questions:

- **Identifying Influencers**: Who are the central figures (influential individuals) in the network?
- **Interaction Dynamics**: What interaction patterns are common among friends?
- **User Similarity**: Who are the *like-minded* users, and how can we find these similar individuals?

Categorization of Network Measures

To answer the core questions of network analysis, specific quantitative measures are defined:

Target Analysis	Purpose / Function
Centrality Measures	Quantifies and identifies the various types of central, highly influential nodes in a network.
Interaction Measures	Quantifies common interaction patterns between connected individuals.
Similarity Measures	Evaluates network features to identify like-minded or structurally similar users.

3.1 Centrality

16 July 2026 09:18

3.1 Centrality

Centrality defines how important a node is within a network.

3.1.1 Degree Centrality

In real-world interactions, individuals with many connections are often considered important. **Degree centrality** applies this exact concept as a structural metric, ranking nodes with more connections higher.

Undirected Graphs

The degree centrality C_d for a node v_i in an undirected graph is defined as:

$$C_d(v_i) = d_i$$

where d_i is the degree (the total number of adjacent edges) of node v_i.

Directed Graphs

In directed networks, edge direction yields different interpretations of importance. Centrality can be measured using in-degree, out-degree, or a combination:

- **Prestige / Prominence (In-degree):** Measures how popular a node is based on incoming links.

$$C_d(v_i) = d_i^{\text{in}}$$

- **Gregariousness (Out-degree):** Measures the outgoing activity or social explicit outreach of a node.

$$C_d(v_i) = d_i^{\text{out}}$$

- **Combination:** Fully ignores edge directions by combining both components. When edge directions are removed, this becomes equivalent to the undirected graph measure.

$$C_d(v_i) = d_i^{\text{in}} + d_i^{\text{out}}$$

Limitation: Raw degree centrality metrics cannot be compared across different networks (e.g., comparing a user's absolute connection count on Facebook versus Twitter). To overcome this limitation, the values must be normalized.

Normalizing Degree Centrality

To standardize scores across networks of varying sizes, three simple normalization techniques are commonly utilized:

1. **Normalize by the maximum possible degree:**

$$C_d^{\text{norm}}(v_i) = \frac{d_i}{n-1}$$

where n is the total number of nodes in the network.

2. **Normalize by the maximum observed degree within the network:**

$$C_d^{\max}(v_i) = \frac{d_i}{\max_j d_j}$$

3. **Normalize by the degree sum:**

$$C_d^{\text{sum}}(v_i) = \frac{d_i}{\sum_j d_j} = \frac{d_i}{2|E|} = \frac{d_i}{2m}$$

where $|E| = m$ represents the total number of edges in the graph.

Example: In a star graph consisting of 9 nodes (where one central node v_1 connects to 8 peripheral leaf nodes):

- For the central node: $C_d(v_1) = d_1 = 8$
- For all other peripheral nodes: $C_d(v_j) = d_j = 1$ (where $j \neq 1$)

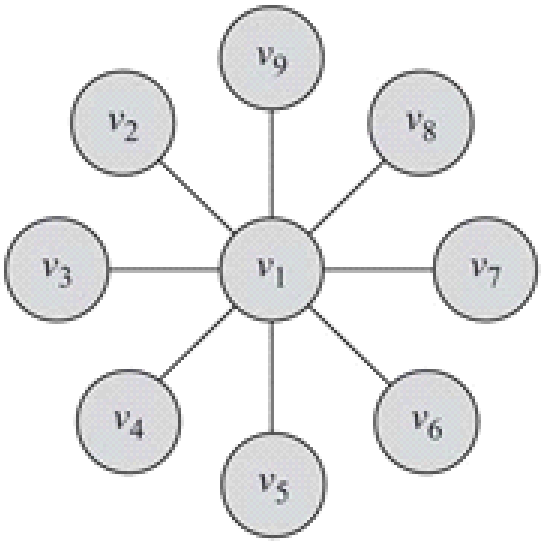


Figure 3.1: Degree Centrality Example.

3.1.2 Eigenvector Centrality

While degree centrality treats all connections equally, real-world scenarios show that having many connections does not automatically guarantee importance. Instead, **having connections to other important nodes** provides a much stronger signal of importance. Eigenvector centrality generalizes degree centrality by incorporating the structural importance of a node's neighbors.

Mathematical Formulation

Let A be the adjacency matrix of a graph, and $c_e(v_i)$ denote the eigenvector centrality of node v_i. The centrality of v_i is proportional to the summation of its neighbors' centralities:

$$c_e(v_i) = \frac{1}{\lambda} \sum_{j=1}^{n} A_{j,i} c_e(v_j)$$

where λ is a fixed constant.

Assuming $\mathbf{C}_e = (c_e(v_1), c_e(v_2), \ldots, c_e(v_n))^T$ represents the centrality vector for all nodes, the system can be rewritten in matrix form as:

$$\lambda \mathbf{C}_e = A^T \mathbf{C}_e$$

This demonstrates that $\mathbf{C}_e$ is an **eigenvector** of the transposed adjacency matrix A^T (or simply A in undirected networks, since $A = A^T$), and λ is its corresponding **eigenvalue**.

Matrix Selection & The Perron-Frobenius Theorem

Because a matrix can yield multiple eigenvalue-eigenvector pairs, a criterion is needed to choose the correct pair. For meaningful structural comparison, centrality metrics must be positive numbers ($\forall v_i > 0$). This constraint is resolved using the following theorem:

Theorem 3.1 (Perron-Frobenius Theorem)
Let $A \in \mathbb{R}^{n \times n}$ represent the adjacency matrix for a [strongly] connected graph or a positive n by n matrix ($A_{i,j} > 0$). There exists a positive real number (Perron-Frobenius eigenvalue) $\lambda_{\max}$, such that $\lambda_{\max}$ is an eigenvalue of A and any other eigenvalue of A is strictly smaller than $\lambda_{\max}$. Furthermore, there exists a corresponding eigenvector $\mathbf{v} = (v_1, v_2, \ldots, v_n)$ of A with eigenvalue $\lambda_{\max}$ such that $\forall v_i > 0$.

Conclusion: To ensure valid, positive centrality values, compute the eigenvalues of A, select the largest eigenvalue ($\lambda_{\max}$), and compute its corresponding eigenvector $\mathbf{C}_e$.

Example 3.2: Three-Node Path Graph ($v_1 - v_2 - v_3$)

1. **Adjacency Matrix (A):**

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

2. **Setting up the Eigensystem:**
We must solve $\lambda \mathbf{C}_e = A\mathbf{C}_e$, which is equivalent to $(A - \lambda I)\mathbf{C}_e = 0$. Assuming $\mathbf{C}_e = [u_1\ u_2\ u_3]^T$:

$$\begin{bmatrix} 0-\lambda & 1 & 0 \\ 1 & 0-\lambda & 1 \\ 0 & 1 & 0-\lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

3. **Characteristic Equation:**
Since $\mathbf{C}_e \neq [0\ 0\ 0]^T$, the determinant must equal zero:

$$\det(A - \lambda I) = \begin{vmatrix} 0-\lambda & 1 & 0 \\ 1 & 0-\lambda & 1 \\ 0 & 1 & 0-\lambda \end{vmatrix} = 0$$

$$\implies (-\lambda)(\lambda^2 - 1) - 1(-\lambda) = 2\lambda - \lambda^3 = \lambda(2 - \lambda^2) = 0$$

4. **Eigenvalue Selection:**
The calculated eigenvalues are $(-\sqrt{2}, 0, +\sqrt{2})$. Selecting the largest eigenvalue yields $\lambda = \sqrt{2}$.

5. **Eigenvector Computation:**

$$\begin{bmatrix} 0-\sqrt{2} & 1 & 0 \\ 1 & 0-\sqrt{2} & 1 \\ 0 & 1 & 0-\sqrt{2} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Assuming the vector $\mathbf{C}_e$ has a norm of 1, the definitive solution is:

$$\mathbf{C}_e = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} 1/2 \\ \sqrt{2}/2 \\ 1/2 \end{bmatrix}$$

Analysis: Node v_2 is the most central node in the network, while nodes v_1 and v_3 share identical, lower centrality scores.

Example 3.3: Five-Node Graph

1. **Adjacency Matrix (A):**

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

2. **Eigenvalues:**
The complete set of eigenvalues computed for A is $(-1.74, -1.27, 0.00, +0.33, +2.68)$.

3. **Eigenvector Selection:**
Isolating the largest eigenvalue gives $\lambda_{\max} = 2.68$. The corresponding eigenvector centrality vector is:

$$\mathbf{C}_e = \begin{bmatrix} 0.4119 \\ 0.5825 \\ 0.4119 \\ 0.5237 \\ 0.2169 \end{bmatrix}$$

Analysis: Based on the final components of the eigenvector centrality vector, node v_2 has the highest value (0.5825) and is confirmed as the most structurally central node.

3.1.3 Katz Centrality

Motivation & Core Problem

- **Limitation of Eigenvector Centrality:** A major issue arises when eigenvector centrality is applied to **directed graphs**.
- **The Zero-Centrality Issue:** Centrality is only passed along through outgoing edges. In special cases—such as a node within a **directed acyclic graph (DAG)**—a node's centrality can become zero even if it has many connected edges.
- **The Solution:** This issue is corrected by introducing a **bias term** (β) to the centrality calculation. The bias term β is added to the centrality values of **all nodes** uniformly, regardless of their position or the overall network topology.

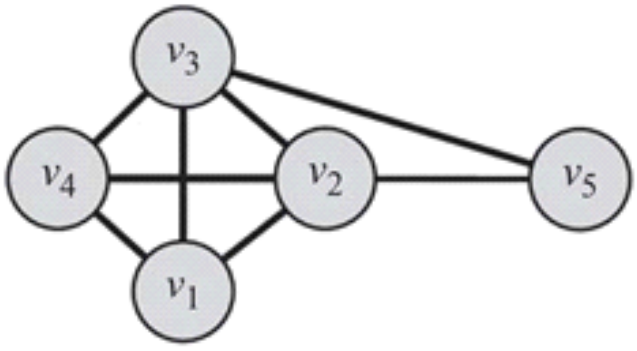


Figure 3.3: Katz Centrality Example.

Mathematical Formulation

1. Scalar Form

The Katz centrality of a specific node v_i is defined as:

$$C_{\text{Katz}}(v_i) = \alpha \sum_{j=1}^{n} A_{j,i} C_{\text{Katz}}(v_j) + \beta$$

- **First Term ($\alpha \sum A_{j,i} C_{\text{Katz}}(v_j)$):** Similar to eigenvector centrality, where the influence of neighboring nodes is passed along. Its impact is governed by the attenuation constant α.
- **Second Term (β):** The constant bias term that ensures no node drops to a zero centrality value.

2. Vector Form

To compute this across the entire network, Equation 3.19 can be rewritten in matrix-vector form:

$$\mathbf{C}_{\text{Katz}} = \alpha A^T \mathbf{C}_{\text{Katz}} + \beta \mathbf{1}$$

Where $\mathbf{1}$ represents a vector composed entirely of 1s.

3. Rearranged Matrix Form

By isolating $\mathbf{C}_{\text{Katz}}$ on the left-hand side and factoring it out, the equation becomes directly solvable via matrix inversion:

$$\mathbf{C}_{\text{Katz}} = \beta (\mathbf{I} - \alpha A^T)^{-1} \cdot \mathbf{1}$$

Parameter Selection & Divergence Constraints

Because calculating Katz centrality requires inverting the matrix $(\mathbf{I} - \alpha A^T)$, the choice of the attenuation parameter α is strictly bounded:

- **When $\alpha = 0$:** The eigenvector centrality component is completely removed. As a result, every node receives the exact same baseline centrality value, β.
- **As α increases:** The influence of the network topology grows, reducing the relative impact of the baseline bias β.
- **Divergence Boundary:** If $\det(\mathbf{I} - \alpha A^T) = 0$, the matrix becomes non-invertible, causing the centrality values to diverge towards infinity.

Critical Condition: The determinant first hits 0 when $\alpha = 1/\lambda$, where λ represents the largest eigenvalue of A^T.
Note: $\det(\mathbf{I} - \alpha A^T) = 0$ can be rewritten as the characteristic equation $\det(A^T - \alpha^{-1}\mathbf{I}) = 0$, meaning $\alpha^{-1} = \lambda$.

- **Practical Rule:** To ensure correct and stable centrality computations, always select an α such that:

$$\alpha < \frac{1}{\lambda}$$

Example 3.4

Consider a network sample governed by the following symmetric adjacency matrix A (where $A = A^T$):

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Step-by-Step Calculation:

1. **Find the Eigenvalues:** Calculating the eigenvalues of A yields: $(-1.68, -1.0, -1.0, +0.35, +3.32)$.
2. **Identify the Largest Eigenvalue:** $\lambda = 3.32$.
3. **Set Parameters:**
   - Choose $\alpha = 0.25$ (which satisfies the safety threshold condition, as $0.25 < \frac{1}{3.32} \approx 0.301$).
   - Choose a baseline bias $\beta = 0.2$.
4. **Compute the Centrality Vector:**

$$\mathbf{C}_{\text{Katz}} = \beta(\mathbf{I} - \alpha A^T)^{-1} \cdot \mathbf{1} = \begin{bmatrix} 1.14 \\ 1.31 \\ 1.31 \\ 1.14 \\ 0.85 \end{bmatrix}$$

Conclusion: Nodes v_2 and v_3 share the highest score (1.31), making them the most central nodes in this network configuration according to Katz centrality.

3.1.4 PageRank

Like eigenvector centrality, Katz centrality has certain limitations, particularly in **directed graphs**.

- **The Problem:** Once a node becomes an authority (acquiring high centrality), it passes **all** of its centrality along **all** of its out-links. This is often undesirable because not everyone known by a highly influential or popular person is equally important or well-known.
- **The Solution:** Divide the passed centrality by the number of outgoing links (out-degree) of that node. Consequently, each connected neighbor receives only a fraction of the source node's centrality.

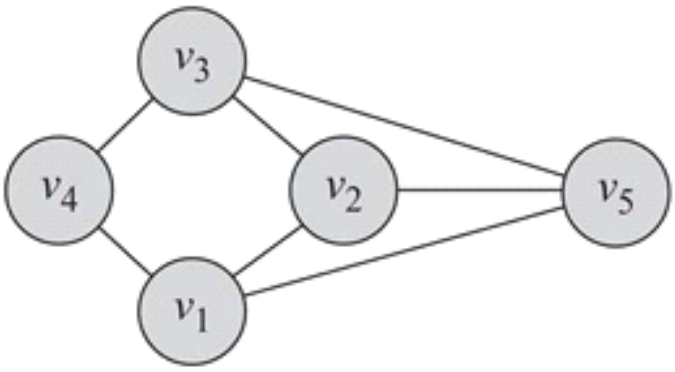


Figure 3.4: PageRank Example.

Mathematical Formulation

To implement this division of centrality, the PageRank centrality $C_p(v_i)$ of a node v_i is defined as:

$$C_p(v_i) = \alpha \sum_{j=1}^{n} A_{j,i} \frac{C_p(v_j)}{d_j^{\text{out}}} + \beta$$

Where:

- $A_{j,i}$ is the entry of the adjacency matrix A.
- d_j^{out} is the out-degree of node v_j.
- α and β are parameters.

Note on Zero Out-Degrees ($d_j^{\text{out}} = 0$):
Equation (3.24) is only defined when d_j^{out} is non-zero. If a node has an out-degree of zero ($d_j^{\text{out}} = 0$), then for all i, $A_{j,i} = 0$, resulting in an indeterminate form $\frac{0}{0}$. We can resolve this problem by setting $d_j^{\text{out}} = 1$ since the node will not contribute any centrality to any other nodes.

Matrix Representation

Assuming all nodes have positive out-degrees ($d_j^{\text{out}} > 0$), Equation (3.24) can be reformulated in matrix format:

$$\mathbf{C}_p = \alpha A^T D^{-1} \mathbf{C}_p + \beta \mathbf{1}$$

We can reorganize this equation to solve directly for the centrality vector $\mathbf{C}_p$:

$$\mathbf{C}_p = \beta (\mathbf{I} - \alpha A^T D^{-1})^{-1} \cdot \mathbf{1}$$

Where:

- $D = \text{diag}(d_1^{\text{out}}, d_2^{\text{out}}, \ldots, d_n^{\text{out}})$ is the diagonal matrix of out-degrees.
- $\mathbf{I}$ is the identity matrix.
- $\mathbf{1}$ is a column vector of all ones.

Application and Key Characteristics

- **Google Web Search:** PageRank centrality is used by the Google search engine to order search results. Webpages and their hyperlinks represent a massive web-graph. When a user executes a search query, matching webpages with higher PageRank values are displayed first.
- **Parameter Selection (α):**
  - Similar to Katz centrality, in practice, we choose a value of $\alpha < \frac{1}{\lambda}$, where λ is the largest eigenvalue of the matrix $A^T D^{-1}$.
  - In **undirected graphs**, the largest eigenvalue of $A^T D^{-1}$ is always $\lambda = 1$. Consequently, the parameter is selected such that $\alpha < 1$.

Example 3.5

For the example graph, the adjacency matrix A is represented as:

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Assuming parameter values of:

- $\alpha = 0.95$ (where $\alpha < 1$)
- $\beta = 0.1$

The PageRank values are computed as:

$$\mathbf{C}_p = \beta(\mathbf{I} - \alpha A^T D^{-1})^{-1} \cdot \mathbf{1} = \begin{bmatrix} 2.14 \\ 2.13 \\ 2.14 \\ 1.45 \\ 2.13 \end{bmatrix}$$

Conclusion:

Hence, nodes v_1 and v_3 have the highest PageRank values.

3.1.5 Betweenness Centrality

Betweenness centrality evaluates the importance of a node based on its role in connecting other nodes within a network. For any given node v_i, it measures the proportion of all shortest paths between other pairs of nodes that pass through v_i.

The betweenness centrality of a node v_i is defined as:

$$C_b(v_i) = \sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

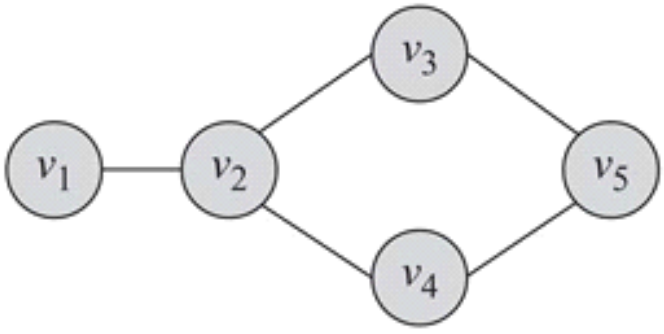


Figure 3.5: Betweenness Centrality Example.

Key Terms

- σ_{st}: The total number of shortest paths from node s to node t (also referred to as *information pathways*).
- $\sigma_{st}(v_i)$: The number of shortest paths from s to t that pass through the node v_i.

Normalization

To compare betweenness centrality across different networks, the values must be normalized by dividing by the maximum possible value the metric can take.

Betweenness centrality reaches its maximum value when a node v_i lies on every single shortest path between all pairs (s, t) in the network. Under this condition, $\frac{\sigma_{st}(v_i)}{\sigma_{st}} = 1$ for all pairs where $s \neq t \neq v_i$.

For an interconnected star-like network layout where a central node v_1 sits on the shortest path between all other pairs of nodes, the maximum value is calculated as:

$$C_b(v_i) = \sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}} = \sum_{s \neq t \neq v_i} 1 = 2\binom{n-1}{2} = (n-1)(n-2)$$

The **normalized betweenness centrality** is therefore obtained using the formula:

$$C_b^{\text{norm}}(v_i) = \frac{C_b(v_i)}{2\binom{n-1}{2}}$$

Computing Betweenness

Computing betweenness centrality requires calculating the shortest paths between all pairs of nodes in the network.

- If a single-source shortest path algorithm like **Dijkstra's algorithm** is used, it must be executed $|V| - 1$ times (excluding the target node being evaluated) to find all-pairs shortest paths.
- More efficient specialized algorithms, such as **Brandes' algorithm**, have been developed to optimize this computation.

Comprehensive Examples

Example 3.6 (Star Network Context)
Consider a graph where a single central hub node v_1 connects directly to all other peripheral nodes ($n = 9$ nodes total).
Since all paths connecting any peripheral pair (s, t) must pass through the central node v_1, the calculations are:

- **Raw Centrality:** $C_b(v_1) = 2\binom{8}{2}$
- **Normalized Centrality:** $C_b^{\text{norm}}(v_1) = 1$

For any peripheral node in this configuration, no shortest paths pass through them, resulting in a betweenness centrality of 0.

Example 3.7 (Diamond Network with a Tail)
Consider a sample graph with 5 nodes structured as follows: node v_1 connects to v_2; node v_2 splits into two parallel paths to v_3 and v_4; both v_3 and v_4 reconnect at node v_5 (forming a diamond loop with v_1 as a tail).

- **Node v_1:** $C_b(v_1) = 0$ (No shortest paths between other node pairs pass through it).
- **Node v_2:**

$$C_b(v_2) = 2 \times \left(\underbrace{\frac{1}{1}}_{s=v_1,t=v_3} + \underbrace{\frac{1}{1}}_{s=v_1,t=v_4} + \underbrace{\frac{2}{2}}_{s=v_1,t=v_5} + \underbrace{\frac{1}{2}}_{s=v_3,t=v_4} + \underbrace{0}_{s=v_3,t=v_5} + \underbrace{0}_{s=v_4,t=v_5} \right)$$

$$C_b(v_2) = 2 \times 3.5 = 7$$

- **Node v_3:**

$$C_b(v_3) = 2 \times \left(\underbrace{0}_{s=v_1,t=v_2} + \underbrace{0}_{s=v_1,t=v_4} + \underbrace{\frac{1}{2}}_{s=v_1,t=v_5} + \underbrace{0}_{s=v_2,t=v_4} + \underbrace{\frac{1}{2}}_{s=v_2,t=v_5} + \underbrace{0}_{s=v_4,t=v_5} \right)$$

$$C_b(v_3) = 2 \times 1.0 = 2$$

- **Node v_4:** Due to structural symmetry with node v_3:

$$C_b(v_4) = C_b(v_3) = 2 \times 1.0 = 2$$

- **Node v_5:**

$$C_b(v_5) = 2 \times \left(\underbrace{0}_{s=v_1,t=v_2} + \underbrace{0}_{s=v_1,t=v_3} + \underbrace{0}_{s=v_1,t=v_4} + \underbrace{0}_{s=v_2,t=v_3} + \underbrace{0}_{s=v_2,t=v_4} + \underbrace{\frac{1}{2}}_{s=v_3,t=v_4} \right)$$

$$C_b(v_5) = 2 \times 0.5 = 1$$

Note on Undirected Graphs: The centralities in the calculations above are scaled by a factor of 2. This accounts for the bidirectional symmetry inherent to undirected graphs, meaning:

$$\sum_{s \neq t \neq v_i} \frac{\sigma_{st}(v_i)}{\sigma_{st}} = 2 \sum_{s \neq t \neq v_i, s<t} \frac{\sigma_{st}(v_i)}{\sigma_{st}}$$

3.1.6 Closeness Centrality

Intuition & Definition

In closeness centrality, the core intuition is that **more central nodes can reach other nodes more quickly**. Formally, these central nodes should have a smaller average shortest path length to all other nodes in the network.

Closeness centrality is defined as:

$$C_c(v_i) = \frac{1}{\bar{l}_{v_i}}$$

where the average shortest path length $\bar{l}_{v_i}$ of node v_i to all other nodes is calculated as:

$$\bar{l}_{v_i} = \frac{1}{n-1} \sum_{v_j \neq v_i} l_{i,j}$$

- n: Total number of nodes in the graph.
- $l_{i,j}$: The shortest path length between node v_i and node v_j.
- **Key Principle:** The smaller the average shortest path length ($\bar{l}_{v_i}$), the higher the closeness centrality ($C_c(v_i)$) of the node.

Example 3.8: Closeness Centrality Calculations

For a graph with $n = 5$ nodes, the closeness centralities are calculated as follows (where $n - 1 = 4$):

- **Node 1:**

$$C_c(v_1) = \frac{1}{(1+2+2+3)/4} = 0.5$$

- **Node 2:**

$$C_c(v_2) = \frac{1}{(1+1+1+2)/4} = 0.8$$

- **Nodes 3 & 4:**

$$C_c(v_3) = C_c(v_4) = \frac{1}{(1+1+2+2)/4} \approx 0.66$$

(Note: The textbook source contains a slight typographical error, printing $C_b(v_4)$ instead of $C_c(v_4)$).

- **Node 5:**

$$C_c(v_5) = \frac{1}{(1+1+2+3)/4} \approx 0.57$$

Conclusion: Node v_2 has the smallest average shortest path length, giving it the **highest closeness centrality** (0.8).

Example 3.9: Comparison of Centrality Measures

Different centrality measures represent distinct perspectives on what makes a node "central." Consequently, a node deemed highly important by one measure might be considered insignificant by another.

To illustrate this, we analyze the top three central nodes for the graph below using six different centrality methods.

Example Graph Structure (Figure 3.6)

The network consists of two cohesive 4-node diamond cliques ($\{v_3, v_4, v_5, v_6\}$ and $\{v_7, v_8, v_9, v_{10}\}$) connected by a bridge edge between v_6 and v_7, with a tail structure ($v_1 - v_2 - v_3$) attached to the left side.

Centrality Method	First Node	Second Node	Third Node
Degree Centrality	**v3 or v6**	**v6 or v3**	**v∈{v4,v5,v7,v8,v9}**
Eigenvector Centrality	**v6**	**v3**	**v4 or v5**
Katz Centrality (α=β=0.3)	**v6**	**v3**	**v4 or v5**
PageRank (α=β=0.3)	**v3**	**v6**	**v2**
Betweenness Centrality	**v6**	**v7**	**v3**
Closeness Centrality	**v6**	**v3 or v7**	**v7 or v3**

Key Takeaways & Relationships

- **Spectral & Degree-Based Measures:** There is a high degree of similarity among the top central nodes for **Degree**, **Eigenvector**, **Katz**, and **PageRank** centralities. This similarity occurs because all four of these measures heavily utilize direct degrees or eigenvector-based walk-counting.
- **Shortest Path-Based Measures: Betweenness** and **Closeness** centralities yield highly aligned results (ranking v_6 and v_7 near the top). This is because both algorithms rely on finding and utilizing the **shortest paths** across the network to determine node importance.

3.1.7 Group Centrality

Standard centrality measures focus exclusively on a single node. **Group centrality** generalizes these concepts to evaluate the importance of a *group of nodes* within a network.

Key Notation

- S: The set of nodes in the group being measured for centrality.
- $V - S$: The set of all nodes outside the group (nonmembers).

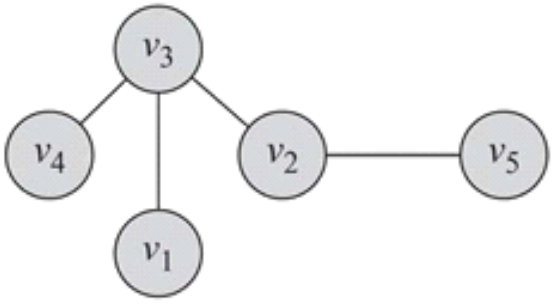


Figure 3.7: Group Centrality Example.

Group Degree Centrality

Group degree centrality is defined as the number of nodes outside the group that are directly connected to at least one member of the group.

Formula

$$C_d^{\text{group}}(S) = |\{v_i \in V - S \mid v_i \text{ is connected to } v_j \in S\}|$$

Key Characteristics & Normalization

- **Directed Graphs:** For directed networks, connections can be specialized in terms of either **in-degrees** or **out-degrees**.
- **Maximum Value:** In the best-case scenario, the group members are connected to all nonmembers. Thus, the maximum possible value is $|V - S|$.
- **Normalization:** To normalize group degree centrality, divide the raw value by the maximum possible value:

$$\text{Normalized } C_d^{\text{group}}(S) = \frac{C_d^{\text{group}}(S)}{|V - S|}$$

Group Betweenness Centrality

Group betweenness centrality measures how often the group acts as an intermediary along the shortest paths between pairs of nodes outside the group.

Formula

$$C_b^{\text{group}}(S) = \sum_{s \neq t, s \notin S, t \notin S} \frac{\sigma_{st}(S)}{\sigma_{st}}$$

Where:

- σ_{st}: The total number of shortest paths between two nonmember nodes s and t.
- $\sigma_{st}(S)$: The number of those shortest paths that pass through *any* member of the group S.

Key Characteristics & Normalization

- **Maximum Value:** In the best case, all shortest paths between all pairs of nonmembers pass through the group. The maximum value is given by $2\binom{|V-S|}{2}$.
- **Normalization:** Group betweenness centrality is normalized by dividing the raw value by this maximum value.

Group Closeness Centrality

Group closeness centrality is defined as the reciprocal of the average shortest path length from the group S to all nonmember nodes.

Formula

$$C_c^{\text{group}}(S) = \frac{1}{\bar{l}_S^{\text{group}}}$$

Where the average path length $\bar{l}_S^{\text{group}}$ is calculated as:

$$\bar{l}_S^{\text{group}} = \frac{1}{|V - S|} \sum_{v_j \notin S} l_{S,v_j}$$

Approaches to Define Group-to-Node Distance (l_{S,v_j})

The distance l_{S,v_j} between the group S and a nonmember $v_j \in V - S$ can be computed using one of three common distance paradigms:

1. **Minimum Distance (Closest Member):** Finds the distance to the nearest group member.

$$l_{S,v_j} = \min_{v_i \in S} l_{v_i,v_j}$$

2. **Maximum Distance:** Finds the distance to the farthest group member.
3. **Average Distance:** Computes the average distance to all members of the group.

Example 3.10

Consider a graph with five nodes $V = \{v_1, v_2, v_3, v_4, v_5\}$ and the following link structure:

- v_3 is connected to v_1, v_2, and v_4.
- v_2 is connected to v_5.

Let the group of interest be $S = \{v_2, v_3\}$. The nonmembers are $V - S = \{v_1, v_4, v_5\}$ (so $|V - S| = 3$).

1. Group Degree Centrality Calculation

- The members of S are connected to all three nonmember nodes (v_1, v_4, and v_5).
- **Raw Value:** $C_d^{\text{group}}(S) = 3$
- **Normalized Value:** $\frac{3}{|V-S|} = \frac{3}{3} = 1$

2. Group Betweenness Centrality Calculation

- There are $2\binom{3}{2} = 6$ ordered pairs of shortest paths between the nonmembers. Every single one of these paths must pass through at least one member of S (v_2 or v_3).
- **Raw Value:** $C_b^{\text{group}}(S) = 6$
- **Normalized Value:** $\frac{6}{2\binom{|V-S|}{2}} = \frac{6}{6} = 1$

3. Group Closeness Centrality Calculation

- *Assumption:* The **minimum function** is used to define group-to-node distance.
- Every nonmember node is directly adjacent to at least one group member:
  - $l_{S,v_1} = 1$ (via v_3)
  - $l_{S,v_4} = 1$ (via v_3)
  - $l_{S,v_5} = 1$ (via v_2)
- The average distance is $\bar{l}_S^{\text{group}} = \frac{1+1+1}{3} = 1$.
- **Centrality Value:** $C_c^{\text{group}}(S) = \frac{1}{1} = 1$

3.2 Transitivity and Reciprocity

16 July 2026 09:40

3.2 Transitivity and Reciprocity

In social media networks, observing specific behaviors is crucial for graph analysis. One fundamental type is **linking behavior**, which determines how links (edges) are formed within a social graph.

Two primary measures are commonly used to analyze linking behavior in directed networks: **transitivity** and **reciprocity**. Among these, transitivity can also be effectively applied to undirected networks.

3.2.1 Transitivity

Transitivity evaluates whether the linking behavior within a network demonstrates a transitive relationship.

- **Mathematical Definition:** For a transitive relation R, the condition is defined as:

$$aRb \wedge bRc \rightarrow aRc$$

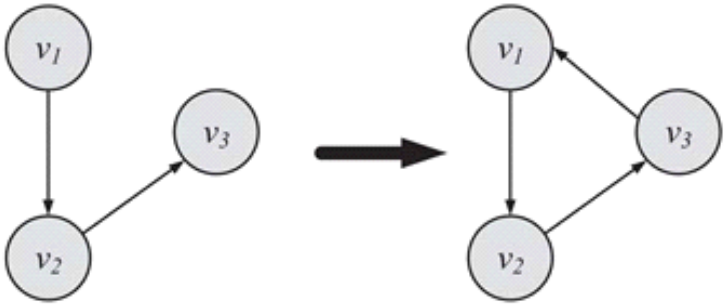


Figure 3.8: Transitive Linking.

Transitive Linking

- **Formal Definition:** Let v_1, v_2, and v_3 denote three distinct nodes. When directed edges (v_1, v_2) and (v_2, v_3) are formed, if the edge (v_3, v_1) is also formed, a transitive linking behavior (transitivity) is established.
- **Informal Definition:**

  "A friend of my friend is my friend."

Network Implications

- **Structure:** Transitive behavior requires a minimum of **three edges**. Together with their participating nodes, these edges form a geometric **triangle**.
- **Density:** A higher level of transitivity results in a denser graph, shifting its structure closer to a **complete graph**. Measuring transitivity allows us to determine how close a given graph is to being complete.
- **Measurement Types:**
  - **Global Clustering Coefficient:** Computed comprehensively for the entire network.
  - **Local Clustering Coefficient:** Computed specifically for an individual node.

Clustering Coefficient

The clustering coefficient is utilized to analyze transitivity within an **undirected graph**. Since transitivity manifests through the formation of triangles, it can be calculated by counting all paths of length 2 (consisting of edges (v_1, v_2) and (v_2, v_3)) and verifying if a third edge (v_3, v_1) exists to close the path.

The standard definition for the clustering coefficient C is:

$$C = \frac{|\text{Closed Paths of Length 2}|}{|\text{Paths of Length 2}|}$$

Alternative Formulae Using Triangles and Triples

Because every single triangle contains exactly six closed paths of length 2, the equation can be rewritten as:

$$C = \frac{(\text{Number of Triangles}) \times 6}{|\text{Paths of Length 2}|}$$

By introducing the concept of node triples, it simplifies further to:

$$C = \frac{(\text{Number of Triangles}) \times 3}{\text{Number of Connected Triples of Nodes}}$$

Understanding Triples

A **triple** is an ordered set of three nodes connected by either two edges (**open triple**) or three edges (**closed triple**). Two triples are considered distinct if:

- Their constituent nodes are different, **or**
- Their constituent nodes are identical, but the triples are missing different edges.

Example: The triples $v_i v_j v_k$ and $v_j v_k v_i$ are distinct because the first is missing edge $e(v_k, v_i)$ while the second is missing edge $e(v_i, v_j)$. Conversely, triples $v_i v_j v_k$ and $v_k v_j v_i$ are identical because both miss the exact same edge $e(v_k, v_i)$.
Because a single triangle misses no edges, **three distinct triples** can be derived from it. This mathematically justifies multiplying the number of triangles by a factor of 3 in the numerator.

Example Calculation

For a graph containing 2 triangles and 2 distinct open triples ($v_2 v_1 v_4$ and $v_2 v_3 v_4$), the global clustering coefficient is evaluated as follows:

$$C = \frac{(\text{Number of Triangles}) \times 3}{\text{Number of Connected Triples of Nodes}}$$

$$C = \frac{2 \times 3}{(2 \times 3) + 2} = 0.75$$

(Note: The denominator accounts for the 6 triples generated by the 2 triangles, plus the 2 unique open triples).

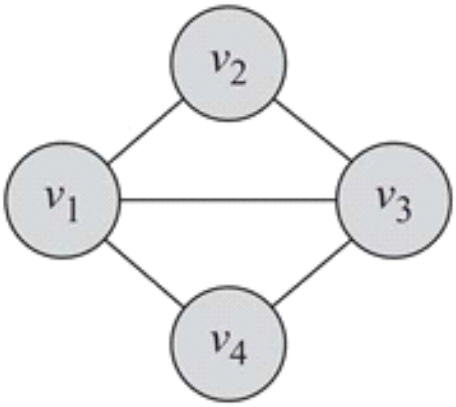


Figure 3.9: A Global Clustering Coefficient Example.

Local Clustering Coefficient

The **local clustering coefficient** measures network transitivity at the individual node level. Primarily utilized for undirected graphs, it quantifies how densely the neighbors of a specific node v (nodes directly adjacent to v) are interconnected with one another.

Definition & Formula

The coefficient for a node v_i is defined as the ratio of actual connections between its neighbors to the total possible connections between them:

$$C(v_i) = \frac{\text{Number of Pairs of Neighbors of } v_i \text{ That Are Connected}}{\text{Number of Pairs of Neighbors of } v_i}$$

For an **undirected graph**, if a node v_i has a degree of d_i (i.e., it has d_i neighbors), the total number of possible pairs among its neighbors (the denominator) can be mathematically written using the combination formula:

$$\binom{d_i}{2} = \frac{d_i(d_i - 1)}{2}$$

Key Properties & Boundaries

- **Maximum Value ($C(v_i) = 1$):** Achieved when all neighbors of the node are interconnected.
- **Minimum Value ($C(v_i) = 0$):** Achieved when none of the neighbors are connected to each other.

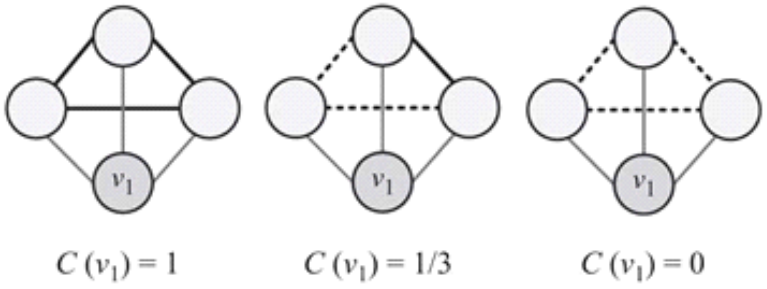


Figure 3.10: Change in Local Clustering Coefficient for Different Graphs. Thin lines depict connections to neighbors. Solid lines indicate connected neighbors, and dashed lines are the missing connections among neighbors.

Visualizing Local Clustering Values

When analyzing graph diagrams, network connections can be categorized to evaluate $C(v_i)$:

- **Target Links:** Thin lines represent the connections from the primary node v_1 to its immediate neighbors.
- **Connected Neighbors:** Solid lines represent existing active edges directly between those neighbors.
- **Missing Links:** Dashed lines denote potential connections between neighbors that do not exist.

3.2.2 Reciprocity

Reciprocity is a simplified version of transitivity restricted to **directed graphs**. Instead of looking at multi-node paths, it focuses exclusively on closed paths/loops of length 2.

Conceptual Overview

- **Formal Definition:** If a directed edge exists from node v to node u, reciprocity occurs if a returning directed edge also exists from node u to node v.
- **Real-World Analogy:** Social media networks (like Tumblr) classify these reciprocal node links as "mutual followers."
- **Informal Expression:**

"If you become my friend, I'll be yours."

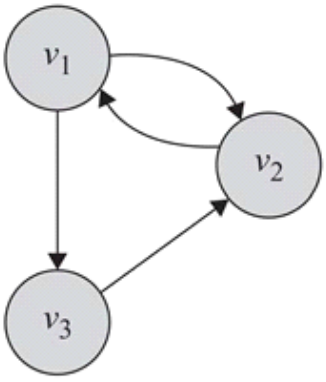


Figure 3.11: A Graph with Reciprocal Edges.

Mathematical Calculation

Reciprocity tracks the total count of mutually pre-existing pairs relative to total edges. Any directed network containing $|E|$ edges can possess a maximum of $\frac{|E|}{2}$ reciprocal pairs. This maximum limit serves as a standard normalization factor.

Using the network's adjacency matrix A, reciprocity (R) is derived systematically as follows:

$$R = \frac{\sum_{i,j,i<j} A_{i,j} A_{j,i}}{|E|/2}$$

$$R = \frac{2}{|E|} \sum_{i,j,i<j} A_{i,j} A_{j,i}$$

$$R = \frac{2}{|E|} \times \frac{1}{2} \mathrm{Tr}(A^2)$$

$$R = \frac{1}{|E|} \mathrm{Tr}(A^2) = \frac{1}{m} \mathrm{Tr}(A^2)$$

Where:

- $\mathrm{Tr}(A) = A_{1,1} + A_{2,2} + \cdots + A_{n,n} = \sum_{i=1}^{n} A_{i,i}$ represents the **matrix trace** (the sum of the elements on the main diagonal).
- m represents the total number of directed edges in the network ($m = |E|$).
- The absolute maximum value for the expression $\sum_{i,j} A_{i,j} A_{j,i}$ is equal to m, occurring only when every directed link is perfectly bidirectional.

Step-by-Step Application Example

Consider a directed network with 3 nodes and 4 total directed edges ($m = 4$).

Step 1: Construct the Adjacency Matrix (A)

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 2: Compute the Squared Matrix (A^2)

$$A^2 = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Step 3: Calculate the Trace of A^2 and Resolve Reciprocity (R)

$$\mathrm{Tr}(A^2) = 1 + 1 + 0 = 2$$

$$R = \frac{1}{m}\mathrm{Tr}(A^2) = \frac{1}{4}(2) = \frac{2}{4} = \frac{1}{2}$$

3.3 Balance and Status

16 July 2026 09:45

3.3 Balance and Status

Signed graphs are highly effective tools for representing structured relationships and social hierarchies within a network:

- **Friends or Foes:** A positive edge from node v_1 to v_2 denotes that v_1 considers v_2 as a friend, whereas a negative edge denotes that v_1 assumes v_2 is an enemy.
- **Social Status:** A positive edge from v_1 to v_2 can also denote that v_1 views v_2's social status as higher than its own.

Consistency in Real-World Scenarios

Real-world networks tend to exhibit behavioral and structural consistency. For instance, a friend of one's friend is naturally more likely to be a friend than an enemy. In a signed graph, this translates to observing specific structural triads (e.g., three positive edges) much more frequently than inconsistent ones (e.g., two positive edges and one negative edge).

To formally measure the consistency of individual attitudes, two primary social science theories are applied depending on the edge context:

1. **Social Balance Theory:** Used when edges represent friend/foe relationships.
2. **Social Status Theory:** Used when edges represent social status.

Social Balance Theory

Also known as **structural balance theory**, this concept evaluates the consistency of friend/foe dynamics among individuals.

Informal Rules of Social Balance

- *The friend of my friend is my friend.*
- *The friend of my enemy is my enemy.*
- *The enemy of my enemy is my friend.*
- *The enemy of my friend is my enemy.*

Mathematical Definition

Let w_{ij} denote the weight value of the edge between nodes v_i and v_j, where:

- **Positive edges (Friendship):** $w_{ij} = 1$
- **Negative edges (Enemyship):** $w_{ij} = -1$

For a structural triangle consisting of nodes v_i, v_j, and v_k, the triad is considered **balanced** if and only if:

$$w_{ij}w_{jk}w_{ki} \geq 0$$

Key Structural Properties

- **Balanced Triangles:** The product of the three edge values is always positive. Visually, a triangle is balanced if it contains an **even number of negative edges** (i.e., zero or two negative edges).
- **Unbalanced Triangles:** The product of the edge values is negative (i.e., containing an odd number of negative edges).
- **Generalization to Cycles:** This theory extends beyond triangles to any closed subgraph. For any cycle, if the product of all edge values is positive, the cycle is considered socially balanced.

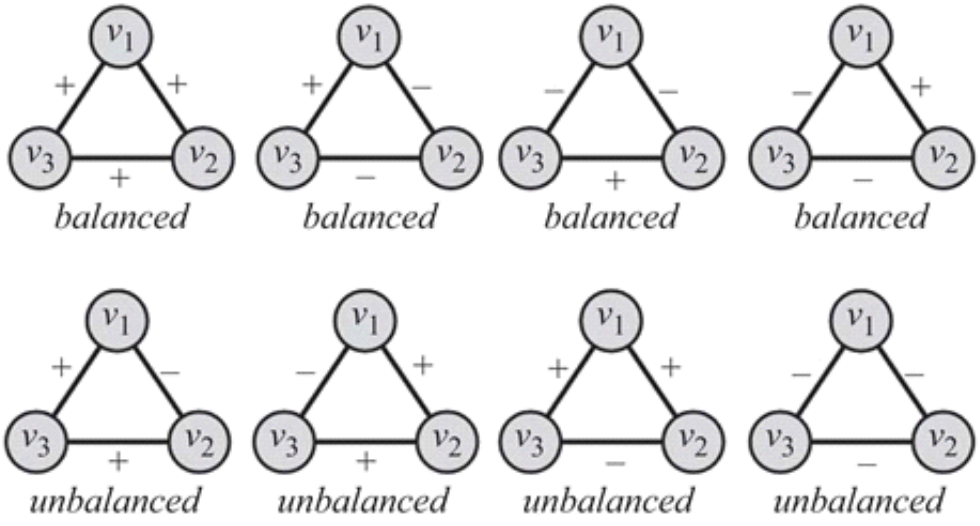


Figure 3.12: Sample Graphs for Social Balance Theory. In balanced triangles, there are an even number of negative edges.

Social Status Theory

Social status theory measures how consistently individuals assign relative social status to their immediate neighbors.

The Axiom of Transitivity

If X has a higher status than Y, and Y has a higher status than Z, then X must have a higher status than Z.

Graph Representation Mechanics

This framework utilizes directed, signed edges to define status directionally:

- A **directed positive edge** from $X \to Y$ $(+)$ indicates that Y has a higher status than X.
- A **directed negative edge** from $X \to Y$ $(-)$ indicates the reverse (X has a higher status than Y).

Examples & Configuration Rules

- **Unbalanced Configuration:** Consider a triad where $v_1 \xrightarrow{+} v_2$ $(v_2 > v_1)$ and $v_2 \xrightarrow{+} v_3$ ($v_3 > v_2$). Status transitivity dictates that v_3 must have a higher status than v_1. If the remaining edge is drawn as $v_1 \xrightarrow{-} v_3$ $(v_1 > v_3)$, this creates a contradiction, making the configuration **unbalanced** and implausible.
- **Balanced Configuration:** A triad where all relative statuses match across every edge path without contradiction (e.g., $v_1 \xrightarrow{-} v_2$, $v_2 \xrightarrow{-} v_3$, and $v_1 \xrightarrow{-} v_3$).

Generalization and Divergence

- **Cycle Generalization:** In a cycle of n nodes, if $n - 1$ consecutive edges are positive and the final concluding edge is negative, social status theory defines the entire cycle as balanced.
- **Theoretical Divergence:** It is entirely possible for an identical graph configuration to be classified as **balanced** under Social Balance Theory, yet categorized as **unbalanced** under Social Status Theory.

3.4 Similarity

16 July 2026 09:48

3.4 Similarity

This section reviews methods used to compute the similarity between two nodes in a network. In social media contexts, nodes frequently represent individuals within a friendship network or items that are related to one another.

The similarity between connected individuals is generally evaluated using two paradigms:

- **Network Similarity:** Computed using network topology information regarding the nodes and the edges connecting them. Network similarity can be analyzed by measuring either **structural equivalence** or **regular equivalence**.
- **Content Similarity:** Computed based on the similarity of the user-generated content (discussed separately in later chapters).

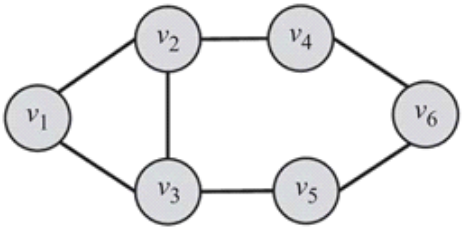


Figure 3.14: Sample Graph for Computing Similarity.

3.4.1 Structural Equivalence

Structural equivalence defines similarity based on the overlap between the direct neighborhoods of two nodes. The larger the shared neighborhood, the more similar the two nodes are considered to be.

Analogy: Two brothers share a high structural equivalence because they have common family members (sisters, mother, father, grandparents, etc.), whereas two random individuals share very little in common.

Let $N(v_i)$ and $N(v_j)$ denote the neighborhoods of nodes v_i and v_j, respectively. A basic metric for node similarity is defined as:

$$\sigma(v_i, v_j) = |N(v_i) \cap N(v_j)|$$

Normalization Techniques

In large networks, the raw intersection value can grow rapidly because nodes may share many neighbors. To control for this, similarity values are normalized to fall within the bounded range $[0, 1]$. Two common normalization methods are:

- **Jaccard Similarity:**

$$\sigma_{\text{Jaccard}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|}$$

- **Cosine Similarity:**

$$\sigma_{\text{Cosine}}(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{\sqrt{|N(v_i)| \cdot |N(v_j)|}}$$

Important Caveat: The standard definition of a neighborhood $N(v_i)$ typically excludes the node v_i itself. This can cause issues: if two nodes are directly connected but share no other mutual neighbors, they will be assigned a similarity score of zero. This limitation can be rectified by explicitly including each node within its own neighborhood set.

Numerical Example

Consider a sample graph containing connected nodes where we evaluate the similarity between node v_2 and node v_5.

- Given neighborhoods: $N(v_2) = \{v_1, v_3, v_4\}$ and $N(v_5) = \{v_3, v_6\}$
- Intersection: $\{v_1, v_3, v_4\} \cap \{v_3, v_6\} = \{v_3\}$ (Size = 1)
- Union: $\{v_1, v_3, v_4\} \cup \{v_3, v_6\} = \{v_1, v_3, v_4, v_6\}$ (Size = 4)

$$\sigma_{\text{Jaccard}}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{|\{v_1, v_3, v_4, v_6\}|} = 0.25$$

$$\sigma_{\text{Cosine}}(v_2, v_5) = \frac{|\{v_1, v_3, v_4\} \cap \{v_3, v_6\}|}{\sqrt{|\{v_1, v_3, v_4\}| \cdot |\{v_3, v_6\}|}} = 0.40$$

Statistical Significance & Random Expectation

A robust way to measure similarity is to compare the observed neighborhood overlap $\sigma(v_i, v_j)$ against the expected value of overlap if nodes chose their neighbors entirely at random. The greater the distance between the observed overlap and the random expectation, the more significant the similarity.

For two nodes v_i and v_j with degrees d_i and d_j in a network of n nodes:

- There is a $\frac{d_i}{n}$ chance of a node becoming v_i's neighbor.
- Since v_j selects d_j neighbors, the expected random overlap is $\frac{d_i d_j}{n}$.

Using the adjacency matrix A, the raw neighborhood overlap can be rewritten as:

$$\sigma(v_i, v_j) = |N(v_i) \cap N(v_j)| = \sum_k A_{i,k} A_{j,k}$$

By subtracting the random expectation from the raw overlap, we define **Significance Similarity** ($\sigma_{\text{significance}}$):

$$\sigma_{\text{significance}}(v_i, v_j) = \sum_k A_{i,k} A_{j,k} - \frac{d_i d_j}{n}$$

$$= \sum_k A_{i,k} A_{j,k} - n \frac{1}{n} \sum_k A_{i,k} \frac{1}{n} \sum_k A_{j,k}$$

$$= \sum_k A_{i,k} A_{j,k} - n \bar{A}_i \bar{A}_j$$

$$= \sum_k (A_{i,k}A_{j,k} - A_{i,k}\bar{A}_j - \bar{A}_i A_{j,k} + \bar{A}_i\bar{A}_j)$$

$$= \sum_k (A_{i,k} - \bar{A}_i)(A_{j,k} - \bar{A}_j)$$

Where $\bar{A}_i = \frac{1}{n}\sum_k A_{i,k}$ is the mean value of the row. The term $\sum_k (A_{i,k} - \bar{A}_i)(A_{j,k} - \bar{A}_j)$ represents the **covariance** between the connectivity profiles of A_i and A_j.

Pearson Correlation Coefficient

Normalizing this covariance by the product of the variances (standard deviations) yields the **Pearson Correlation Coefficient**:

$$\sigma_{\text{pearson}}(v_i, v_j) = \frac{\sigma_{\text{significance}}(v_i, v_j)}{\sqrt{\sum_k (A_{i,k} - \bar{A}_i)^2}\sqrt{\sum_k (A_{j,k} - \bar{A}_j)^2}}$$

$$\sigma_{\text{pearson}}(v_i, v_j) = \frac{\sum_k (A_{i,k} - \bar{A}_i)(A_{j,k} - \bar{A}_j)}{\sqrt{\sum_k (A_{i,k} - \bar{A}_i)^2}\sqrt{\sum_k (A_{j,k} - \bar{A}_j)^2}}$$

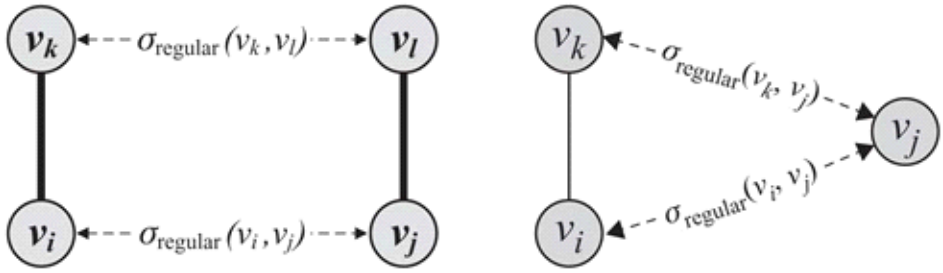
Unlike Jaccard and Cosine similarities, the Pearson correlation coefficient outputs values within the range $[-1, 1]$:

- **Positive Value (> 0):** Indicates that when v_i befriends an individual v_k, v_j is highly likely to also befriend v_k.
- **Negative Value (< 0):** Indicates the opposite; when v_i befriends v_k, v_j is unlikely to befriend v_k.
- **Zero Value (0):** Denotes that there is no linear relationship between the befriending behaviors of v_i and v_j.

3.4.2 Regular Equivalence

In **regular equivalence**, unlike *structural equivalence*, the focus is not on the specific neighborhoods shared between individuals, but on how the neighborhoods themselves are structurally similar.

- **Concept Illustration**: Athletes are considered similar not because they know each other personally, but because they interact with similar types of individuals (e.g., coaches, trainers, and other players). This concept applies broadly across various industries or professions where individuals do not know each other but have contact with highly comparable roles.
- **Core Principle**: Regular equivalence assesses similarity by comparing the **similarity of neighbors** rather than their literal overlap.



(a) Original Formulation (b) Relaxed Formulation

Figure 3.15: Regular Equivalence. Solid lines denote edges, and dashed lines denote similarities between nodes. In regular equivalence, similarity between nodes v_i and v_j is replaced by similarity between (a) their neighbors v_k and v_l or between (b) neighbor v_k and node v_j.

Mathematical Formalization

1. Original Formulation

One way to formalize this concept is to consider two nodes v_i and v_j similar when they have many similar neighbors v_k and v_l.

$$\sigma_{\text{regular}}(v_i, v_j) = \alpha \sum_{k,l} A_{i,k} A_{j,l} \sigma_{\text{regular}}(v_k, v_l)$$

The Problem: This original formulation is strictly **self-referential**. Solving for nodes i and j requires first solving for their neighbors k and l, which in turn requires solving for their respective neighbors, leading to an infinite loop.

2. Relaxed Formulation

To resolve the self-referential problem, the model is relaxed: it assumes that node v_i is similar to node v_j when v_j is similar to v_i's direct neighbors v_k.

$$\sigma_{\text{regular}}(v_i, v_j) = \alpha \sum_k A_{i,k} \sigma_{\text{regular}}(v_k, v_j)$$

Expressed compactly in **vector/matrix format**:

$$\sigma_{\text{regular}} = \alpha A \sigma_{\text{regular}}$$

3. Accounting for Self-Similarity

A node is inherently highly similar to itself. To guarantee this in the mathematical framework, an identity matrix $\mathbf{I}$ is introduced. This adds 1 to all diagonal entries, representing baseline self-similarities $\sigma_{\text{regular}}(v_i, v_i)$:

$$\sigma_{\text{regular}} = \alpha A \sigma_{\text{regular}} + \mathbf{I}$$

By algebraically rearranging the terms, we arrive at the closed-form solution to find the regular equivalence similarity matrix:

$$\sigma_{\text{regular}} = (\mathbf{I} - \alpha A)^{-1}$$

Convergence Constraints

This equation shares a clear structural similarity with **Katz centrality**. To guarantee matrix convergence, the attenuation parameter α must be selected carefully. The standard convention is to pick α such that:

$$\alpha < \frac{1}{\lambda}$$

Where λ represents the **largest eigenvalue** of the adjacency matrix A.

Example 3.15

Given a graph with the following 6×6 adjacency matrix A:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

1. **Eigenvalue Calculation**: The largest eigenvalue of A is calculated to be $\lambda = 2.43$.
2. **Parameter Choice**: We set $\alpha = 0.4$, ensuring the convergence condition is met since $0.4 < \frac{1}{2.43} \approx 0.4115$.
3. **Similarity Matrix Computation**: Computing $\sigma_{\text{regular}} = (\mathbf{I} - 0.4A)^{-1}$ yields:

$$\sigma_{\text{regular}} = \begin{bmatrix} 1.43 & 0.73 & 0.73 & 0.26 & 0.26 & 0.16 \\ 0.73 & 1.63 & 0.80 & 0.56 & 0.32 & 0.26 \\ 0.73 & 0.80 & 1.63 & 0.32 & 0.56 & 0.26 \\ 0.26 & 0.56 & 0.32 & 1.31 & 0.23 & 0.46 \\ 0.26 & 0.32 & 0.56 & 0.23 & 1.31 & 0.46 \\ 0.16 & 0.26 & 0.26 & 0.46 & 0.46 & 1.27 \end{bmatrix}$$

Key Takeaways from the Example Matrix:

- Any single row or column displays the precise relative similarity score of a given node to all other nodes in the network.
- **Node** v_1 is most structurally similar (excluding self-similarity) to nodes v_2 and v_3 (score of 0.73).
- Overall, nodes v_2 and v_3 share the highest cross-node similarity score (0.80) within this specific graph structure.

3.5 Summary

1. Network Centrality Measures

Centrality measures are designed to identify the most critical or "central" nodes within a graph.

- **Degree Centrality:** Assumes that the node with the maximum number of connections (highest degree) is the most central individual.
    - *Directed Graph Variants:* In directed networks, this is split into **prestige** and **gregariousness**.
- **Eigenvector Centrality:** An extension of degree centrality that considers a node important if it is connected to *other important nodes*. Grounded in the **Perron-Frobenius theorem**, it is computed via the eigenvector of the network's adjacency matrix.
- **Katz Centrality:** Resolves specific limitations of eigenvector centrality in directed graphs by introducing a **bias term**.
- **PageRank Centrality:** Represents a normalized version of Katz centrality. It is famously utilized by the Google search engine to rank web pages.

- **Betweenness Centrality:** Assumes central nodes act as **hubs** or bridges that connect distinct groups of other nodes.
- **Closeness Centrality:** Operationalizes the intuition that central nodes are physically close (require fewer hops) to all other nodes in the network.
- **Group Centrality:** Individual node metrics can be generalized to assess sets of nodes via **group degree centrality**, **group betweenness centrality**, and **group closeness centrality**.

2. Linking Behavior and Network Triads

Linking between nodes (e.g., establishing a friendship on a social media platform) is the most commonly observed network phenomenon. Linking behavior is evaluated using two primary characteristics:

- **Transitivity:** Expressed by the adage, *"when a friend of my friend is my friend."*
  - It is commonly evaluated using **closed triads** of edges.
  - It is quantified using the **clustering coefficient**:
    - *Global Clustering Coefficient:* Measures transitivity across the network as a whole.
    - *Local Clustering Coefficient:* Evaluates transitivity specifically for an individual node.
- **Reciprocity:** A simplified form of transitivity occurring in loops of length 2. It embodies the concept: *"if you become my friend, I'll be yours."*

3. Social Theories for Validation

To analyze whether structural relationships within a social media network are consistent, key social network theories are applied to validate outcomes:

- **Social Balance Theory**
- **Social Status Theory**

4. Node Similarity Measures

Node similarity defines how equivalent two nodes are based on their positions within the network topology:

- **Structural Equivalence:** Two nodes are considered identical or similar if they **share neighborhoods** (i.e., they connect to the exact same neighbors).
  - *Key Metrics:* **Cosine similarity** and **Jaccard similarity**.
- **Regular Equivalence:** Nodes are considered similar if their **neighborhoods are structurally similar**, meaning they play the same role or connect to similar types of nodes, even if they do not share the exact same neighbors.

3.7 Exercises

16 July 2026 10:45

Exercise 3.7 – Centrality

Question 1

Come up with an example of a directed connected graph in which eigenvector centrality becomes zero for some nodes. Describe when this happens.

Answer

A simple example is the directed graph

$$v_1 \to v_2 \to v_3$$

The adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

This graph is a **directed acyclic graph (DAG)**. Since there are no directed cycles, repeated multiplication by the adjacency matrix eventually produces the zero matrix, making all eigenvalues equal to zero.

The eigenvector centrality equation is

$$A\mathbf{x} = \lambda \mathbf{x}.$$

Here, the dominant eigenvalue is

$$\lambda = 0,$$

which forces

$$\mathbf{x} = \mathbf{0}.$$

Thus, every node has eigenvector centrality equal to zero.

More generally, eigenvector centrality becomes zero for nodes that are **not part of, or cannot receive influence from, any strongly connected component associated with the dominant eigenvalue**. In directed acyclic graphs, this results in all nodes receiving zero centrality.

Question 2

Does β have any effect on the order of centralities? Can changing β reverse the ranking of two nodes?

Answer

Yes.

The parameter β controls the amount of **baseline importance (teleportation/random jump)** assigned to every node.

- When $\beta = 0$, centrality depends entirely on the graph structure.
- As β increases, every node receives additional uniform importance.
- Nodes with relatively few incoming links benefit proportionally more from increasing β.

Therefore, changing β **can change the ordering of node centralities**. It is possible for a node that originally had lower centrality than another node to become more central after increasing β, particularly in sparse or weakly connected graphs.

Hence, **yes**, changing β can reverse the ranking of two nodes.

Question 3

In PageRank, what α values guarantee that the centrality values are calculated correctly (i.e., values do not diverge)?

Answer

The PageRank equation is

$$\mathbf{r} = \alpha P^T \mathbf{r} + (1 - \alpha)\mathbf{v},$$

where

- P is the transition matrix,
- $\mathbf{v}$ is the teleportation vector.

To guarantee convergence,

$$0 \leq \alpha < 1.$$

Reason:

- If $\alpha < 1$, teleportation ensures that the Markov chain is irreducible and aperiodic, producing a unique stationary distribution.
- If $\alpha = 1$, convergence is guaranteed **only for certain graphs** (strongly connected and aperiodic). In general, multiple stationary distributions or non-convergence may occur.

In practice,

$$\boxed{\alpha = 0.85}$$

is the standard choice.

Question 4

Calculate the PageRank values for the graph under different parameter settings.

Graph structure

The graph contains the following directed edges:

- $v_1 \leftrightarrow v_2$
- $v_1 \leftrightarrow v_3$
- $v_1 \rightarrow v_4$
- $v_2 \rightarrow v_4$
- $v_4 \rightarrow v_3$

(a) $\alpha = 1,\ \beta = 0$

This corresponds to the **pure link structure** with no teleportation.

Observations:

- v_3 receives links from both v_1 and v_4.
- v_1 receives links from both v_2 and v_3.
- v_4 receives links from v_1 and v_2.
- v_2 receives only one incoming link.

Hence the ranking is approximately

$$\boxed{PR(v_1) \approx PR(v_3) > PR(v_4) > PR(v_2)}$$

(b) $\alpha = 0.85,\ \beta = 1$

Teleportation is introduced.

Effects:

- Every node receives a positive baseline probability.
- Differences between highly ranked and weakly ranked nodes become smaller.
- The ordering generally remains the same.

Approximate ranking:

$$\boxed{PR(v_1) \approx PR(v_3) > PR(v_4) > PR(v_2)}$$

with more balanced PageRank values.

(c) $\alpha = 0,\ \beta = 1$

No link information is used.

Every node is equally likely to be visited.

Therefore,

$$\boxed{PR(v_1) = PR(v_2) = PR(v_3) = PR(v_4) = \frac{1}{4}.}$$

Discussion

Parameters	Result
$\alpha = 1, \beta = 0$	Ranking depends entirely on graph structure.
$\alpha = 0.85, \beta = 1$	Standard PageRank. Link structure dominates but teleportation prevents dead ends and guarantees convergence.
$\alpha = 0, \beta = 1$	Graph structure is ignored; all nodes receive equal PageRank.

Question 5

Consider a full n-tree. This is a tree in which every internal node has n children. Calculate the betweenness centrality for the root node, internal nodes, and leaves.

Answer

Betweenness centrality counts the number of shortest paths passing through a node.

Root node

Every shortest path connecting nodes in different branches must pass through the root.

Therefore, the root has the **maximum betweenness centrality**.

If each subtree contains m nodes, then

$$BC(\text{root}) = \binom{n}{2} m^2,$$

since every pair of nodes chosen from different subtrees contributes one shortest path through the root.

Internal node

An internal node lies between its descendants and the remainder of the tree.

Its betweenness is positive but smaller than the root's.

If the node has descendant subtree size s, then

$$BC(\text{internal}) = s\,(N - s - 1),$$

where

- N is the total number of nodes,
- s is the number of descendants below that node.

Thus, deeper internal nodes generally have lower betweenness than those closer to the root.

Leaf node

A leaf is never an intermediate node on any shortest path.

Hence,

$$\boxed{BC(\text{leaf}) = 0.}$$

Summary

Node type	Betweenness Centrality
Root	Highest; all shortest paths between different branches pass through it
Internal node	Positive; depends on the size of its descendant subtree
Leaf	0

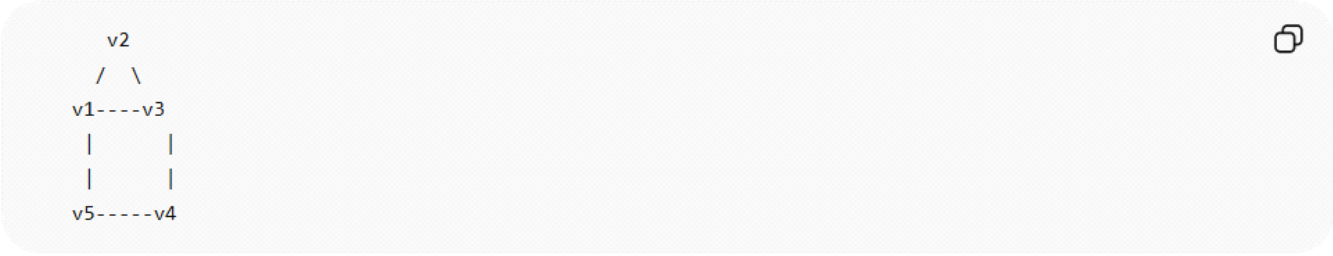
Question 6

Show an example where the eigenvector centrality of all nodes in the graph is the same while betweenness centrality gives different values for different nodes.

Answer

A suitable example is the **cycle graph with an added chord**.

Consider the undirected graph:



Edges:

$$(v_1, v_2),\ (v_2, v_3),\ (v_3, v_4),\ (v_4, v_5),\ (v_5, v_1),\ (v_1, v_3)$$

Eigenvector Centrality

Eigenvector centrality measures how well-connected a node is to other important nodes.

Because every node belongs to the same connected component and the graph is nearly regular, the eigenvector centrality values are approximately equal:

$$EC(v_1) \approx EC(v_2) \approx EC(v_3) \approx EC(v_4) \approx EC(v_5).$$

Betweenness Centrality

Betweenness centrality measures how frequently a node lies on shortest paths.

- Nodes v_1 and v_3 lie on many shortest paths because of the extra edge.
- Nodes v_2, v_4, and v_5 lie on fewer shortest paths.

Thus,

$$BC(v_1) = BC(v_3) > BC(v_2),\ BC(v_4),\ BC(v_5).$$

Conclusion

This example illustrates that:

- Eigenvector centrality can assign nearly identical importance to all nodes.
- Betweenness centrality distinguishes nodes that serve as bridges on shortest paths.

Question 7

(a) Let p be the probability that any node v_i is connected to any node v_j. What is the expected reciprocity of this graph?

Answer

For any ordered pair of distinct nodes,

- the probability that the edge $v_i \to v_j$ exists is

$$p,$$

- the probability that the reverse edge $v_j \to v_i$ also exists is

$$p.$$

Since edge occurrences are independent,

$$P(\text{both edges exist}) = p^2.$$

The expected reciprocity is

$$r = \frac{\text{expected number of reciprocal edges}}{\text{expected number of edges}} = \frac{p^2}{p} = p.$$

Therefore,

$$\boxed{r = p.}$$

(b) Let m be the number of edges and n the number of nodes. What are the maximum and minimum possible reciprocity?

Answer

Reciprocity is defined as

$$r = \frac{\text{number of reciprocal edges}}{\text{total number of edges}}.$$

Maximum Reciprocity

Maximum reciprocity occurs when every edge has its reverse edge.

Hence,

$$\boxed{r_{\max} = 1.}$$

Minimum Reciprocity

Minimum reciprocity occurs when no edge has a reverse edge.

Thus,

$$\boxed{r_{\min} = 0.}$$

Summary

Quantity	Value
Expected reciprocity	$\boxed{p}$
Maximum reciprocity	$\boxed{1}$
Minimum reciprocity	$\boxed{0}$

Question 8(a)

When $m = 15$ and $n = 10$, find a graph with a minimum average clustering coefficient (one is enough).

Answer

The clustering coefficient measures how likely two neighbors of a node are connected.

To minimize the average clustering coefficient, the graph should contain **no triangles**.

One example is a **tree** on 10 nodes (9 edges) with six additional edges added carefully so that no triangles are formed.

For example,

$$\begin{aligned} E = \{&(v_1, v_2), (v_2, v_3), (v_3, v_4), (v_4, v_5), (v_5, v_6), \\ &(v_6, v_7), (v_7, v_8), (v_8, v_9), (v_9, v_{10}), \\ &(v_1, v_6), (v_2, v_7), (v_3, v_8), (v_4, v_9), (v_5, v_{10}), (v_1, v_8)\}. \end{aligned}$$

These edges create a sparse graph without forming triangles.

Therefore,

$$\boxed{\text{Average clustering coefficient} = 0.}$$

Question 8(b)

Can you come up with an algorithm to find such a graph for any m and n?

Answer

Yes.

A simple algorithm is:

1. **Create a spanning tree** containing all n vertices.
   - This guarantees connectivity.
   - Trees contain no triangles.
2. **Count the remaining edges**

$$r = m - (n - 1).$$

3. While $r > 0$:
   - Add an edge between two vertices that are **not already adjacent**.
   - Only add the edge if it **does not complete a triangle** with existing edges.
   - Decrease r by one.
4. Continue until exactly m edges have been added.

If triangle-free edge additions are no longer possible (as m approaches the maximum number of edges in a bipartite graph), construct a **complete bipartite graph** $K_{a,b}$, which is triangle-free and has the largest possible number of edges without increasing the clustering coefficient.

Pseudocode

```
Input: n vertices, m edges

1. Build a spanning tree on n vertices.
2. remaining = m - (n - 1)

3. while remaining > 0:
        choose two non-adjacent vertices u and v
        if adding (u,v) creates no triangle:
             add edge (u,v)
             remaining -= 1

4. return the graph
```

This algorithm maintains an average clustering coefficient as low as possible by avoiding triangle formation whenever feasible.

Question 9

Find all conflicting directed triad configurations for social balance and social status. A conflicting configuration is an assignment of positive/negative edge signs for which only one theory considers the triad balanced and the other considers it unbalanced.

Answer

Background

Two theories evaluate signed directed triads differently:

- **Social Balance Theory** focuses on whether friendships/enmities form a psychologically stable relationship.
- **Social Status Theory** interprets positive edges as pointing from a lower-status person to a higher-status person (equivalently, reversing a negative edge). A triad is balanced if the resulting graph admits a consistent ordering of node status (i.e., it is acyclic).

A triad is **conflicting** when one theory classifies it as balanced while the other classifies it as unbalanced.

Case 1: Balanced under Social Balance, Unbalanced under Social Status

Consider the directed cycle

$$A \to B, \qquad B \to C, \qquad C \to A$$

with all edges **positive**.

- **Balance Theory:** Balanced because all relationships are positive.
- **Status Theory:** Unbalanced because the edges imply

$$A < B < C < A,$$

which is impossible (a directed cycle cannot define a consistent status ordering).

Therefore,

$$\boxed{\text{Balanced (Balance Theory), Unbalanced (Status Theory).}}$$

Case 2: Unbalanced under Social Balance, Balanced under Social Status

Consider

$$A \to B : +, \qquad B \to C : +, \qquad C \to A : -$$

Interpreting the negative edge for status reverses its direction:

$$A \to B, \qquad B \to C, \qquad A \to C,$$

which is acyclic.

Hence,

- **Status Theory:** Balanced.
- **Balance Theory:** Unbalanced because the signed cycle contains an odd number of negative relationships.

Therefore,

$$\boxed{\text{Balanced (Status Theory), Unbalanced (Balance Theory).}}$$

Summary

Edge Signs	Social Balance	Social Status
$+, +, +$ directed cycle	Balanced	Unbalanced
$+, +, -$ directed cycle	Unbalanced	Balanced

These are the standard examples of **conflicting directed triads** between the two theories.

Question 10

In Figure 3.6:

1. **Compute node similarity using Jaccard and cosine similarity for nodes v_5 and v_4.**
2. **Find the most similar node to v_7 using regular equivalence.**

Answer

The information provided is not sufficient to compute these values.

The calculations require **Figure 3.6**, which contains the graph structure (nodes and edges). The current image only shows the exercise questions and **does not include Figure 3.6**.

To answer Question 10, the following information is needed:

- The neighbors of v_4 and v_5 (for Jaccard and cosine similarity).
- The complete graph topology containing v_7 (for regular equivalence).

Once Figure 3.6 is available, the computations are:

(a) Jaccard Similarity
For nodes u and v,

$$\boxed{J(u,v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}}$$

where $N(u)$ is the set of neighbors of node u.

(b) Cosine Similarity
Using adjacency vectors,

$$\boxed{\cos(u,v) = \frac{\mathbf{a}_u \cdot \mathbf{a}_v}{\|\mathbf{a}_u\| \, \|\mathbf{a}_v\|}}$$

where $\mathbf{a}_u$ and $\mathbf{a}_v$ are the adjacency vectors of the two nodes.

(c) Regular Equivalence
Regular equivalence identifies nodes that connect to **similar types of nodes**, even if they are not directly adjacent. The node most similar to v_7 is the one occupying the same structural role in the graph.

Please upload or include Figure 3.6, and I will compute:

- the **Jaccard similarity** between v_5 and v_4,
- the **Cosine similarity** between v_5 and v_4, and
- determine the **most regular-equivalent node to** v_7 with complete calculations.

R2-Chapter 4 - Network Models

16 July 2026 10:51

Chapter 4: Network Models

Introduction: The Scale of Real-World Social Networks

Real-world social networks are massive and highly complex. To understand their scale, consider the state of Facebook in May 2011:

- **Nodes:** 721 million users.
- **Edges:** 68.5 billion total friendships.
- **Average Degree:** An average of 190 friends per user.

This raises two fundamental questions:

1. What are the principal underlying processes that help initiate these friendships?
2. How can these seemingly independent friendships form such a complex network?

The Need for Network Models

In social media, networks contain millions of nodes and billions of edges, making the reasons for the existence of individual connections highly obscure. Because analyzing every friendship independently is incredibly difficult, researchers design **network models** to generate smaller-scale graphs that simulate real-world network properties.

This approach allows for the cost-efficient measurement of simulated networks under a key assumption: *if the model accurately simulates properties observed in the real world, its analysis can explain the behavior of the real-world network itself.*

Key Benefits of Network Models:

- **Mathematical Explanations:** They allow for a better understanding of phenomena observed in real-world networks by providing concrete mathematical explanations.
- **Controlled Experiments:** They enable controlled experiments on synthetic networks when real-world data is unavailable or inaccessible.

Three Principal Network Models

Before diving into their specific properties, this chapter introduces three primary models designed to accurately replicate real-world network structures:

1. **The Random Graph Model**
2. **The Small-World Model**
3. **The Preferential Attachment Model**

4.1 Properties of Real-World Networks

16 July 2026 11:08

4.1 Properties of Real-World Networks

Real-world networks share common characteristics. When designing network models, the objective is to devise frameworks that accurately describe these systems by mimicking their core attributes. To determine these characteristics, measurements for specific network properties must show consistency across different types of networks.

Three specific network attributes exhibit consistent measurements across real-world networks:

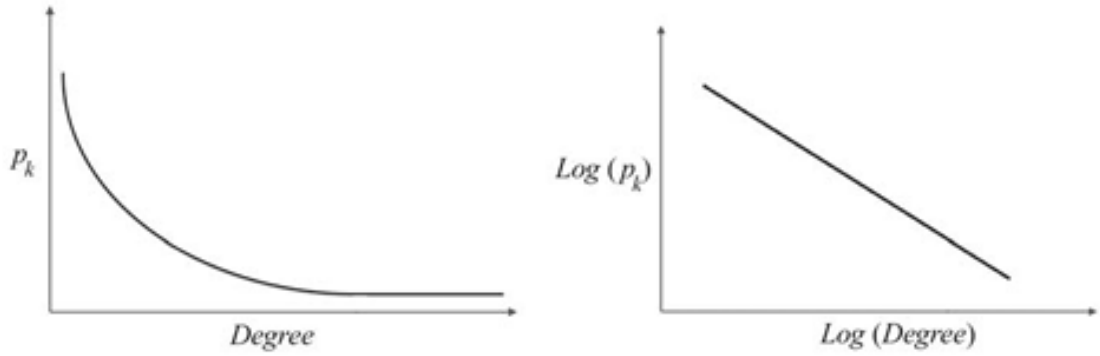
- **Degree Distribution:** Denotes how node degrees are distributed across a network.
- **Clustering Coefficient:** Measures the transitivity of a network.
- **Average Path Length:** Denotes the average distance (shortest path length) between pairs of nodes.

4.1.1 Degree Distribution

The distribution of popularity, importance, or scale in real-world networks typically follows a highly skewed trend, similar to the distribution of wealth among individuals (where most people hold an average amount of capital while a few are exceptionally wealthy) or city populations (where a few metropolitan areas are densely populated compared to average-sized cities).

In social media and online entities, this phenomenon is regularly observed through the following patterns:

- Many websites are visited less than a thousand times a month, whereas a few are visited more than a million times daily.
- Most social media users are active on a few sites, whereas a few individuals are active on hundreds of sites.
- There are exponentially more modestly priced products for sale compared to expensive ones.
- There exist many individuals with a few friends and a handful of users with thousands of friends.



(a) Power-Law Degree Distribution

(b) Log-Log Plot of Power-Law Degree Distribution

Figure 4.1: Power-Law Degree Distribution and Its Log-Log Plot.

Mathematical Formulation: Power-Law Distribution

The degree of a node in a social network typically denotes the number of friends an individual has. The distribution of these values follows a **power-law distribution**.

Let k denote the degree of a node, and let p_k denote the fraction of individuals with degree k:

$$p_k = \frac{\text{frequency of observing } k}{|V|}$$

In a power-law distribution, the relationship is defined as:

$$p_k = ak^{-b}$$

Where:

- b is the power-law exponent.
- a is the power-law intercept.

To verify and analyze this distribution, logarithms are taken from both sides of the equation:

$$\ln p_k = -b \ln k + \ln a$$

This logarithmic transformation demonstrates that the **log-log plot** of a power-law distribution forms a straight line with a slope of $-b$ and a y-intercept of $\ln a$.

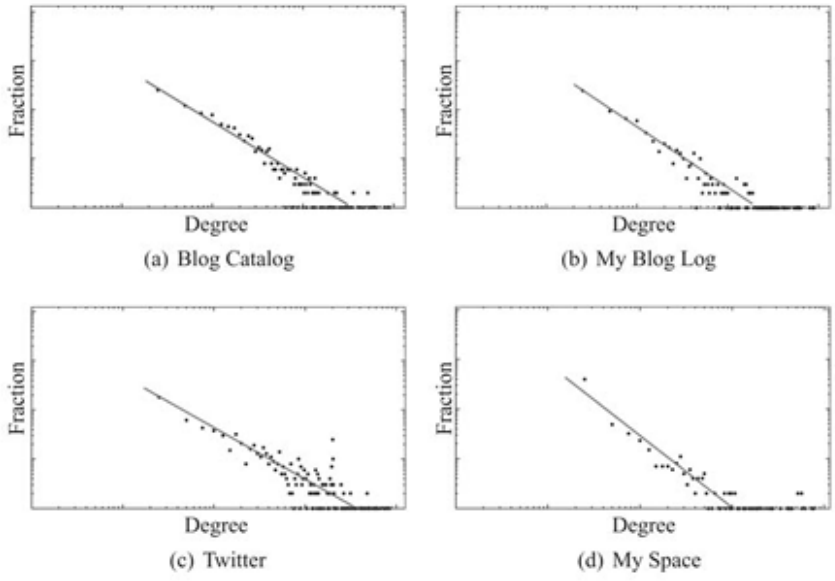


Figure 4.2: Log-Log Plots for Power-Law Degree Distribution in Social Media Networks. In these figures, the x-axis represents the logarithm of the degree, and the y-axis represents the logarithm of the fraction of individuals with that degree (i.e., $\log(p_k)$). The line demonstrates the linear trend observed in log-log plots of power-law distributions.

Methodology for Verifying a Power-Law Distribution

To check whether an empirical network exhibits a power-law distribution, the following process is conducted:

1. **Select a Measure:** Pick a popularity measure (such as the number of friends k in a social network) and compute it for the entire network.
2. **Compute Fractions:** Calculate p_k, which is the fraction of individuals having popularity k.
3. **Graph the Data:** Plot a log-log graph where the x-axis represents $\ln k$ and the y-axis represents $\ln p_k$.
4. **Identify Trends:** Evaluate the plot. If a power-law distribution exists, a clear linear trend (a straight line) will be observed. Real-world networks such as *Blog Catalog*, *My Blog Log*, *Twitter*, and *MySpace* consistently exhibit this linear trend.

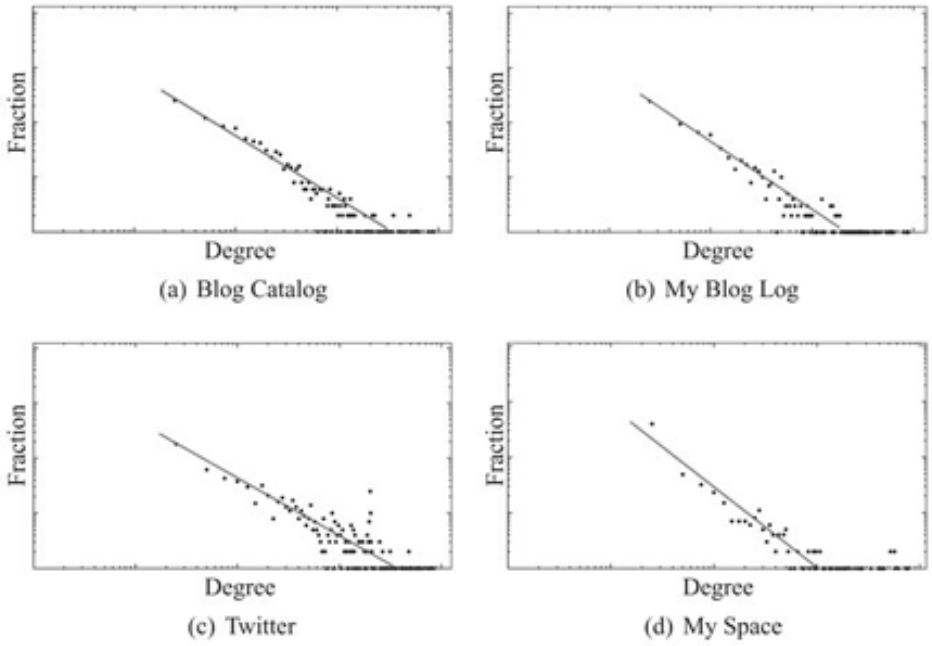


Figure 4.2: Log-Log Plots for Power-Law Degree Distribution in Social Media Networks. In these figures, the x-axis represents the logarithm of the degree, and the y-axis represents the logarithm of the fraction of individuals with that degree (i.e., $\log(p_k)$). The line demonstrates the linear trend observed in log-log plots of power-law distributions.

Scale-Free Networks

Networks that exhibit a power-law degree distribution are formally called **scale-free networks**. Because the vast majority of social networks display scale-free properties, network science places a strong emphasis on developing models capable of generating synthetic networks with a power-law degree distribution.

Empirical Observations: Average Local Clustering Coefficient

Real-world networks also show unique clustering behaviors. The table below outlines the average local clustering coefficients across various prominent platforms:

Network Platform	Average Local Clustering Coefficient
Web	0.081
Facebook	0.14 (measured with 100 friends)
Flickr	0.31
LiveJournal	0.33
Orkut	0.17
YouTube	0.13

4.1.2 Clustering Coefficient

- **Transitivity in Social Networks:** In real-world social networks, friendships exhibit high transitivity. This means that friends of an individual are highly likely to be friends with one another.
- **Friendship Triads:** These interconnected relationships form **triads of friendships**, which are frequently observed in social structures.
- **Clustering Impact:** The prevalence of these triads results in networks characterized by a **high average [local] clustering coefficient**.

Real-World Example (Facebook, May 2011):

- Facebook users with a degree of 2 (exactly two friends) had an average clustering coefficient of **0.5**.
- This indicates that for **50%** of all users with two friends, those two friends were also friends with each other.

4.1.3 Average Path Length

- **The Small-World Phenomenon:** In real-world networks, any two members are usually connected via relatively short paths, meaning the average path length is small.
- **Milgram's Small-World Experiment (1960s):** Stanley Milgram conjectured that individuals around the globe are connected to one another by a path of at most six individuals—a concept widely known as the **six degrees of separation**.
- **Modern Network Examples:** Social networks continue to demonstrate small average path lengths. For instance, in May 2011, the average path length between individuals in the global Facebook graph was **4.7**, while it dropped to **4.3** for users specifically within the United States.

Table 4.2: Average Path Length in Real-World Networks

Network	Average Path Length
Web	16.12
Facebook	4.7
Flickr	5.67
LiveJournal	5.88
Orkut	4.25
YouTube	5.10

Summary: Core Properties of Real-World Networks

There are **three key properties** consistently observed across real-world networks:

1. **Power-law degree distribution**
2. **High clustering coefficient**
3. **Small average path length**

Network Modeling Goal: Network models are designed using simple assumptions about how friendships form, with the objective of generating **scale-free networks** that accurately reproduce these three core properties.

- *Note:* The foundational starting point for studying these structures is the simplest network model: the **random graph model**.

4.2 Random Graphs

16 July 2026 11:11

4.2 Random Graphs

Core Foundational Assumption

The random graph model begins with the most basic assumption of network formation:

Basic Assumption: Edges (i.e., friendships) between nodes (i.e., individuals) are formed randomly.

While real-world networks are rarely truly random, assuming random edge formation simplifies mathematical analysis. The goal is to see if these simplified random networks can still exhibit the common structural characteristics observed in complex real-world networks.

Two Primary Models of Random Graphs

Random graphs are typically generated using one of two distinct classical frameworks:

1. The $G(n, p)$ Model

- **Proposed By:** Independently by Edgar Gilbert, and Solomonoff and Rapoport.
- **Definition:** For a graph with a **fixed number of nodes** n, each of the $\binom{n}{2}$ possible unique edges is formed independently with a fixed probability p.
- **Edge Variance:** The total number of edges is not fixed; it varies according to a probability distribution.

2. The $G(n, m)$ Model

- **Proposed By:** Paul Erdős and Alfred Rényi.
- **Definition:** Both the number of nodes n and the number of edges m are **fixed**. The graph is chosen uniformly at random from the set of all possible graphs that contain exactly n nodes and m edges.
- **Total Graph Space (Ω):** Let Ω denote the set of all possible graphs with n nodes and m edges. The total number of unique graphs in this set is:

$$|\Omega| = \binom{\binom{n}{2}}{m}$$

- **Selection Probability:** The probability of uniformly selecting any single specific graph from this set is $\frac{1}{|\Omega|}$, which serves as the conceptual analog to the probability p in the $G(n,p)$ model.

Model Comparison

- **Asymptotic Equivalence:** In the limit (as networks get large), both models act similarly. If we set the exact number of edges in the Erdős–Rényi model equal to the expected number of edges in the Gilbert model, i.e., $m = \binom{n}{2}p$, their behaviors converge.
- **Analytical Simplicity:** Mathematically, the $G(n,p)$ model is almost always simpler to analyze. Consequently, structural properties are typically calculated over all possible graphs generated by the $G(n,p)$ process and then averaged to understand large-scale graph behaviors.

Key Mathematical Propositions of the $G(n,p)$ Model

Proposition 4.1: Expected Degree of a Node

Proposition: The expected number of edges connected to a single node (expected degree, denoted as c) in $G(n,p)$ is:

$$c = (n-1)p$$

- **Proof:** A single node can connect to at most $n-1$ other nodes. Because every potential edge is selected independently with a probability p, the average number of successful connections is $(n-1)p$.
- **Notation Note:** While degree is often denoted as k or c in network literature, c is explicitly used here to represent the *expected* degree of a random graph.
- **Equivalent Expression for Probability:**

$$p = \frac{c}{n-1}$$

Proposition 4.2: Expected Total Number of Edges

Proposition: The expected total number of edges in a $G(n, p)$ graph is:

$$\text{Expected Edges} = \binom{n}{2} p$$

- **Proof:** Following the same independent logic, a graph with n nodes has a maximum potential capacity of $\binom{n}{2}$ edges. Since each edge has an independent probability p of existing, the expected value is the total potential edges multiplied by p.

Proposition 4.3: Exact Probability of Observing m Edges

Proposition: In a graph generated by the $G(n, p)$ model, the probability of observing a specific total number of edges m follows a **binomial distribution**:

$$P(|E| = m) = \binom{\binom{n}{2}}{m} p^m (1-p)^{\binom{n}{2}-m}$$

- **Proof:** To achieve a graph with exactly m edges, we must successfully choose m edges out of the total $\binom{n}{2}$ possible options (which happens with a probability of p^m). Simultaneously, the remaining $\binom{n}{2} - m$ edges must *fail* to form to ensure no extra edges exist (which happens with a probability of $(1-p)^{\binom{n}{2}-m}$).

4.2.1 Evolution of Random Graphs

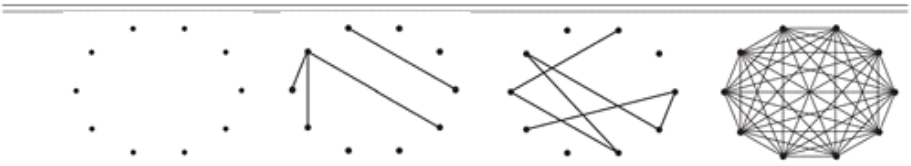
In a random graph model $G(n, p)$, as nodes form connections over time, a large fraction of them become interconnected, meaning a path exists between any pair among them. This large fraction forms a **connected component**, commonly referred to as the **largest connected component** or the **giant component**.

The structural behavior of the random graph can be tuned by selecting an appropriate probability value p:

- When $p = 0$: The size of the largest connected component is 0 (no pairs are connected).
- When $p = 1$: The size of the largest connected component is n (all possible pairs are connected).

As p increases, the graph becomes denser. The table below illustrates the structural metrics for a random graph with $n = 10$ nodes across different values of p.

Table 4.3: Evolution of Random Graphs. Here, p is the random graph generation probability, c is the average degree, ds is the diameter size, slc is the size of the largest component, and l is the average path length. The highlighted column denotes phase transition in the random graph



p	0.0	0.055	0.11	1.0
c	0.0	0.8	≈ 1	9.0
ds	0	2	6	1
slc	0	4	7	10
l	0.0	1.5	2.66	1.0

Table 4.3: Evolution of Random Graphs

- p: Random graph generation probability
- c: Average degree
- ds: Diameter size
- slc: Size of the largest component
- l: Average path length

Metric	p=0.0	p=0.055	p=0.11 (Phase Transition)	p=1.0
p	0	0.055	0.11	1
c	0	0.8	≈1	9
ds	0	2	6	1
slc	0	4	7	10
l	0	1.5	2.66	1

Structural Stages of Evolution

1. When p is Very Small

- No giant component is observed in the graph.
- Small, isolated connected components are formed.
- The diameter (ds) remains small because nodes exist only within these isolated components, connected to just a handful of other local nodes.

2. As p Gets Larger

- A giant component starts to appear.
- Previously isolated components begin to interconnect.
- The diameter values (ds) increase because nodes become connected to each other via longer, winding paths (e.g., see $p = 0.11$ in Table 4.3).

3. Phase Transition

As p continues to increase beyond a critical threshold, the structural properties shift significantly. The diameter starts shrinking because nodes connect via alternative, more direct paths that tend to be shorter.

Definition: The specific threshold where the diameter value reaches its peak and begins to shrink in a random graph is called the **Phase Transition**.

At the point of phase transition, two primary phenomena occur simultaneously:

1. The giant component, which just started to appear, begins to grow rapidly.
2. The diameter, having just reached its maximum value, begins to decrease.

Mathematical Proof of Phase Transition

It is formally proven that in random graphs, the phase transition occurs exactly when the average degree $c = 1$; which corresponds to a probability of:

$$p = \frac{1}{n-1}$$

Proposition 4.4.

In random graphs, phase transition happens at $c = 1$.

Proof (Sketch):

Let c be the expected node degree in a random graph, defined by:

$$c = p(n-1)$$

1. Consider any connected set of nodes S, and its complement set $\bar{S} = V - S$. Assume that the set S is significantly smaller than its complement ($|S| \ll |\bar{S}|$).
2. Given any node $v \in S$, moving one hop (edge) away from v visits approximately c nodes.
3. Extending this logic, moving one hop away from the entire set S allows us to visit approximately $|S|c$ nodes.
4. Because $|S|$ is assumed to be small, the nodes in S predominantly visit new nodes residing in $\bar{S}$. Consequently, the set of nodes "guaranteed to be connected" expands by a factor of c with each additional hop:
    - **1 Hop away:** Component grows by a factor of c
    - **2 Hops away:** Component grows by a factor of c^2
5. In the limit, for this expanding component of visited nodes to successfully scale and become the largest connected component after traveling n hops, the growth factor must satisfy:

$$c^n \geq 1 \quad \text{or equivalently} \quad c \geq 1$$

6. If $c < 1$, the number of newly visited nodes dies out exponentially rather than expanding.

Hence, the boundary for the phase transition to occur is precisely at $c = 1$. ■

Real-World Network Modeling Performance

While random graphs provide valuable theoretical insights into network growth, their performance in replicating real-world structures is mixed:

- **Success:** Random graphs can accurately model and predict the **average path length** observed in real-world networks.
- **Failure:** Random graphs fail to generate a realistic **degree distribution** or a realistic **clustering coefficient** matching real-world empirical networks.

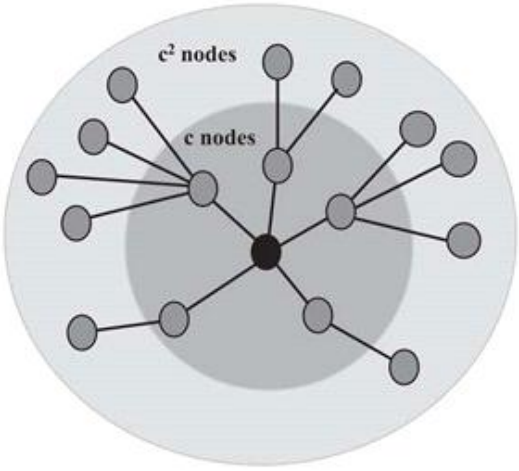


Figure 4.3: Nodes Visited by Moving n-hops away in a Random Graph. c denotes the expected node degree.

4.2.2 Properties of Random Graphs

Degree Distribution

When computing the degree distribution of a random graph, we estimate the probability $P(d_v = d)$ of observing a specific degree d for a given node v.

Proposition 4.5. For a graph generated by $G(n, p)$, a node v has a degree d (where $d \leq n - 1$) with the probability:

$$P(d_v = d) = \binom{n-1}{d} p^d (1-p)^{n-1-d}$$

This represents a **binomial degree distribution**.

Generalization in the Limit ($n \to \infty$)

Assuming that n is fixed provides a baseline, but we can generalize this result by evaluating the degree distribution as the number of nodes approaches infinity ($n \to \infty$).

Using the fact that $\lim_{x \to 0} \ln(1+x) = x$ and substituting $p = \frac{c}{n-1}$ (where c is the average degree), we can compute the limit for each term of the binomial distribution:

1. **Limit of the probability term:**

$$\lim_{n\to\infty} (1-p)^{n-1-d} = \lim_{n\to\infty} e^{\ln(1-p)^{n-1-d}} = \lim_{n\to\infty} e^{(n-1-d)\ln(1-p)}$$

$$= \lim_{n\to\infty} e^{(n-1-d)\ln\left(1-\frac{c}{n-1}\right)} = \lim_{n\to\infty} e^{-(n-1-d)\frac{c}{n-1}} = e^{-c}$$

1. **Limit of the combination term:**

$$\lim_{n\to\infty} \binom{n-1}{d} = \lim_{n\to\infty} \frac{(n-1)!}{(n-1-d)!d!}$$

$$= \lim_{n\to\infty} \frac{((n-1)\times(n-2)\times\cdots\times(n-d))(n-1-d)!}{(n-1-d)!d!}$$

$$= \lim_{n\to\infty} \frac{(n-1)\times(n-2)\times\cdots\times(n-d)}{d!} \approx \frac{(n-1)^d}{d!}$$

1. **Final Degree Distribution Limit:**
   Substituting these evaluations back into the original binomial expression gives:

$$\lim_{n\to\infty} P(d_v = d) = \lim_{n\to\infty} \binom{n-1}{d} p^d (1-p)^{n-1-d}$$

$$= \frac{(n-1)^d}{d!} \left(\frac{c}{n-1}\right)^d e^{-c} = e^{-c}\frac{c^d}{d!}$$

- **Key Takeaway:** In the limit, random graphs generate a **Poisson degree distribution** with mean c. This behaves differently from the **power-law degree distributions** typically observed in real-world networks.
- *Note:* At $c = 1$, the component size is stable, marking the point where the phase transition occurs.

Clustering Coefficient

Proposition 4.6. In a random graph generated by $G(n, p)$, the expected local clustering coefficient for a node v is p.

Proof

The local clustering coefficient $C(v)$ for a node v is defined as:

$$C(v) = \frac{\text{number of connected pairs of } v\text{'s neighbors}}{\text{number of pairs of } v\text{'s neighbors}}$$

Because v can have varying degrees depending on how edges are randomly formed, we compute the expected value $\mathbb{E}(C(v))$ across all possible degrees:

$$\mathbb{E}(C(v)) = \sum_{d=0}^{n-1} \mathbb{E}(C(v) \mid d_v = d) P(d_v = d)$$

Evaluating the local clustering coefficient given a specific degree d:

$$\mathbb{E}(C(v) \mid d_v = d) = \frac{\text{number of connected pairs of } v\text{'s } d \text{ neighbors}}{\text{number of pairs of } v\text{'s } d \text{ neighbors}} = \frac{p\binom{d}{2}}{\binom{d}{2}} = p$$

Substituting this back into the expectation summation:

$$\mathbb{E}(C(v)) = p \sum_{d=0}^{d=n-1} P(d_v = d) = p$$

(Since the sum of all probabilities in a distribution equals 1).

Proposition 4.7. The global clustering coefficient of a random graph generated by $G(n, p)$ is p.

Proof

The global clustering coefficient represents the probability that two neighbors of the same node are connected. In a random graph, this edge-creation probability is independent, uniform, and exactly equal to the generation probability p. Consequently, the **expected local clustering coefficient is equivalent to the global clustering coefficient** in random graphs.

Limitations in Modeling Real-World Networks

- To generate networks with a high clustering coefficient using the $G(n, p)$ model, p must be set to a high value.
- However, choosing a large p creates a highly dense graph. Real-world networks are typically **sparse**.
- **Conclusion:** Random graphs are generally incapable of generating networks with high clustering coefficients without compromising their structural sparseness.

Average Path Length

Proposition 4.8. The average path length l in a random graph is:

$$l \approx \frac{\ln |V|}{\ln c}$$

Proof Sketch

Let $\mathcal{D}$ denote the expected diameter size in the random graph. Starting from any arbitrary node with an expected degree c:

- Traveling **1 edge** allows you to visit approximately c nodes.
- Traveling **2 edges** allows you to visit approximately c^2 nodes.
- Traveling $\mathcal{D}$ ("diameter") number of edges allows you to visit approximately $c^{\mathcal{D}}$ nodes.

After traveling a distance equal to the diameter, almost all nodes in the network should be visited:

$$c^{\mathcal{D}} \approx |V|$$

In the limit for random graphs, the expected diameter size tends toward the average path length ($\mathcal{D} \approx l$). Therefore:

$$c^{\mathcal{D}} \approx c^{l} \approx |V|$$

Taking the logarithm of both sides yields:

$$l \ln c \approx \ln |V| \implies l \approx \frac{\ln |V|}{\ln c}$$

4.2.3 Modeling Real-World Networks with Random Graphs

Overview

Real-world networks can be simulated and evaluated by constructing a representative **random graph model**, denoted as $G(n, p)$.

Simulation Methodology

To model a real-world network using a random graph, the following parameters are determined:

- n: The total number of nodes in the given network.
- c: The average degree calculated from the network.
- p: The connection probability between nodes, which is derived using the formula:

$$p = \frac{c}{n-1}$$

Key Findings & Model Evaluation

- **Successes:** The random graph model performs remarkably well in accurately predicting and capturing the **average path lengths** of real-world networks.
- **Limitations:** When evaluating **transitivity**, the random graph model drastically underestimates the **average clustering coefficient** (C).
- **Next Steps:** Because of this structural shortcoming, the **small-world model** is introduced and studied to bridge the gap.

Empirical Comparison

The data below demonstrates how simulated random graphs match or diverge from original real-world networks across multiple domains.

Table: Comparison between Real-World Networks and Simulated Random Graphs

Note: C denotes the average clustering coefficient.

Table 4.4: A Comparison between Real-World Networks and Simulated Random Graphs. In this table, C denotes the average clustering coefficient. The last two columns show the average path length and the clustering coefficient for the random graph simulated for the real-world network. Note that average path lengths are modeled properly, whereas the clustering coefficient is underestimated

	Original Network				**Simulated Random Graph**	
Network	*Size*	*Average Degree*	*Average Path Length*	C	*Average Path Length*	C
Film Actors	225,226	61	3.65	0.79	2.99	0.00027
Medline Coauthorship	1,520,251	18.1	4.6	0.56	4.91	1.8×10^{-4}
E.Coli	282	7.35	2.9	0.32	3.04	0.026
C.Elegans	282	14	2.65	0.28	2.25	0.05

4.3 Small-World Model

16 July 2026 14:36

4.3 Small-World Model

Overview & Motivation

- **Random Graph Model Limitations:** The underlying assumption of random graph models is that connections form entirely at random. While this approach can properly simulate **average path lengths** in real-world networks, it severely **underestimates the clustering coefficient**.
- **The Small-World Model:** To resolve this discrepancy, Duncan J. Watts and Steven Strogatz introduced the small-world model in 1997.
- **The Egalitarian Assumption:** In real-world social circles, individuals typically maintain a limited, often fixed number of connections (e.g., family, friends, teachers). The small-world model leverages an *egalitarian* baseline where every individual has the exact same number of neighbors. While still an oversimplification, it captures the clustering coefficient of real networks much more accurately.

The Regular Ring Lattice

In graph theory terms, establishing an egalitarian baseline is equivalent to embedding individuals within a **regular network**. A **regular ring lattice** is a specific type of regular network where ordered nodes follow a strict geometric connection pattern:

- In a regular lattice of degree c, every node is directly connected to its previous $\frac{c}{2}$ neighbors and its subsequent $\frac{c}{2}$ neighbors.
- **Formal Definition:** For a set of nodes $V = \{v_1, v_2, v_3, \ldots, v_n\}$, an edge exists between node v_i and node v_j if and only if:

$$0 < |i - j| \leq \frac{c}{2}$$

Limitations of a Pure Regular Lattice

- **High Path Length:** Although a regular lattice successfully models high transitivity, its **average path length is too high** to represent real-world networks.
- **Fixed Clustering Coefficient:** The clustering coefficient is rigid and mathematically fixed to the following value:

$$\frac{3(c-2)}{4(c-1)} \approx \frac{3}{4}$$

  Because this value is non-adjustable, it cannot adapt to the diverse clustering coefficients observed in real-world environments.

The Small-World Solution

To bridge the gap between structured lattices and disordered networks, the small-world model dynamically operates between a regular lattice and a random network using a randomness control parameter, β.

- **The Parameter β ($0 \leq \beta \leq 1$):** Controls the degree of randomness injected into the network structure.
  - When $\beta = 0$, the network is completely ordered and behaves as a **regular lattice**.
  - When $\beta = 1$, the network is completely disordered and becomes a **random graph**.

Algorithm 4.1 Small-World Generation Algorithm

Require: Number of nodes $|V|$, mean degree c, parameter β

return A small-world graph $G(V, E)$
G = A regular ring lattice with $|V|$ nodes and degree c
for node v_i (starting from v_1), and all edges $e(v_i, v_j)$, $i < j$ **do**
  v_k = Select a node from V uniformly at random.
  if rewiring $e(v_i, v_j)$ to $e(v_i, v_k)$ does not create loops in the graph or multiple edges between v_i and v_k **then**
    rewire $e(v_i, v_j)$ with probability β: $E = E - \{e(v_i, v_j)\}$, $E = E \cup \{e(v_i, v_k)\}$;
  end if
end for
Return $G(V, E)$

The Rewiring Process

The network transitions from order to randomness through a mechanism called **rewiring**, executed via the following programmatic steps:

1. **Initialization:** Start with a standard regular ring lattice G containing $|V|$ nodes and uniform degree c.
2. **Iteration:** Iterate through each node v_i (beginning at v_1) and evaluate every existing edge $e(v_i, v_j)$ where $i < j$.
3. **Random Selection:** Uniformly choose a target node v_k at random from the total node set V.
4. **Validation Check:** Verify that shifting the connection from $e(v_i, v_j)$ to $e(v_i, v_k)$ does not introduce self-loops or duplicate (multiple) edges between v_i and v_k.
5. **Probabilistic Rewiring:** With a probability of β, disconnect the edge from its endpoint v_j and reconnect it to the new endpoint v_k:

$$E = E - \{e(v_i, v_j)\}, \quad E = E \cup \{e(v_i, v_k)\}$$

6. **Termination:** Return the updated graph $G(V, E)$.

Key Properties of Small-World Networks

- **Dual Advantages:** By tuning the parameter β, the resulting network successfully maintains a **high clustering coefficient** while simultaneously dropping to **short average path lengths**.
- **Remaining Limitation:** While the small-world model matches path lengths and clustering properties effectively, its generated **degree distribution** still does not fully mirror the distribution patterns observed in genuine real-world networks.

4.3.1 Properties of the Small-World Model

Degree Distribution

The degree distribution for the small-world model defines the probability of observing a specific degree d for a given node v:

$$P(d_v = d) = \sum_{n=0}^{\min(d-c/2, c/2)} \binom{c/2}{n} (1-\beta)^n \beta^{c/2-n} \frac{(\beta c/2)^{d-c/2-n}}{(d-c/2-n)!} e^{-\beta c/2}$$

- **Key Characteristics:**
  - This distribution behaves similarly to the Poisson degree distribution observed in purely random graphs.
  - **Lattice Influence:** In practice, graphs generated by the small-world model feature nodes that mostly share similar degrees due to the structured nature of the underlying regular lattice.
  - **Contrast with Real-World Networks:** In real-world networks, degrees typically follow a **power-law distribution**, where the vast majority of nodes have small degrees and only a select few ("hubs") possess exceptionally large degrees.

Clustering Coefficient

The clustering coefficient measures the density of closed triads (triangles) within a network. The coefficient varies heavily across different graph models:

- **Regular Lattice:** $C(0) = \frac{3(c-2)}{4(c-1)}$
- **Random Graph Model:** $p = \frac{c}{n-1}$
- **Small-World Network ($C(p)$):** Falls between the regular lattice and random graph values, depending heavily on the rewiring parameter β (where $\beta = p$).

The analytical relationship between a regular lattice and a small-world network's clustering coefficient is proven to be:

$$C(p) \approx (1-p)^3 C(0)$$

Intuition Behind the Formula:
For an existing triad in a regular lattice to stay connected after the rewiring process, all three of its edges must escape rewiring. Since each edge is rewired independently with a probability of p, the probability that an individual edge is *not* rewired is $(1-p)$. Therefore, the probability that all three edges remain intact is $(1-p)^3$. While new triads can form due to the rewiring process itself, their probability of occurrence is nominal and considered negligible.

- **Behavior and Preferences:**
  - The value of $C(p)$ remains high until p reaches 0.1 (10% of edges rewired), after which it decreases rapidly toward zero.
  - Because a high clustering coefficient is a desired characteristic for these networks, a rewiring parameter of $\beta \leq 0.1$ is preferred.

Average Path Length

The average path length tracking quantifies the typical separation between node pairs in a graph.

- **Regular Lattice:** The average path length is defined as:

$$L(0) = \frac{n}{2c}$$

- **Random Graph:** The average path length scales dynamically as:

$$\frac{\ln n}{\ln c}$$

- **Small-World Network ($L(p)$):** Unlike the clustering coefficient, there is **no analytical formula** to directly compare $L(p)$ to $L(0)$. Instead, this relationship must be computed empirically.
- **Behavior and Preferences:**
  - The average path length decays significantly faster than the clustering coefficient, stabilizing when only about 1% of the edges have been rewired.
  - To ensure short paths across the network, a parameter of $\beta \geq 0.01$ is preferred.
  - **Optimal Balance:** To simultaneously achieve both a high clustering coefficient and a small average path length, β values within the range of $0.01 \leq \beta = p \leq 0.1$ are ideal.

Table 4.5: A Comparison between Real-World Networks and Simulated Graphs Using the Small-World Model. In this table C denotes the average clustering coefficient. The last two columns show the average path length and the clustering coefficient for the small-world graph simulated for the real-world network. Both average path lengths and clustering coefficients are modeled properly

	Original Network				Simulated Graph	
Network	*Size*	*Average Degree*	*Average Path Length*	*C*	*Average Path Length*	*C*
Film Actors	225,226	61	3.65	0.79	4.2	0.73
Medline Coauthorship	1,520,251	18.1	4.6	0.56	5.1	0.52
E.Coli	282	7.35	2.9	0.32	4.46	0.31
C.Elegans	282	14	2.65	0.28	3.49	0.37

4.3.2 Modeling Real-World Networks with the Small-World Model

Core Objectives of Real-World Network Modeling

A desirable model for a real-world network should simultaneously satisfy two main topological properties:

- **High Clustering Coefficient:** Reflects the tendency of nodes to form tightly-knit groups or cliques.
- **Short Average Path Length:** Ensures that any two nodes in the network can be reached in a small number of steps.

Simulation and Parameter Tuning

To recreate a real-world network using the small-world model, the parameters must be carefully tuned:

- **Acceptable Range for β:** For $0.01 \leq \beta \leq 0.10$, the generated small-world network yields acceptable properties where the average path length drops significantly while the clustering coefficient remains high.
- **Parameter Selection Strategy:**
  - Suppose a real-world network has a known average degree c and a clustering coefficient C.
  - Set $C(p) = C$ and determine the rewiring probability β mathematically (e.g., using Equation 4.22).
  - Using the calculated β, along with the average degree c and network size n, the small-world model can be simulated.
- **Simulation Results:** The model effectively replicates realistic clustering coefficients and small average path lengths for various real-world network applications.

Limitations of the Small-World Model

⚠️ **Key Limitation:** Although the small-world model successfully captures clustering and path length characteristics, it is **incapable of generating a realistic degree distribution** for simulated real-world graphs.

Transition to Scale-Free Networks

- Most real-world networks are **scale-free**, meaning they follow a **power-law degree distribution** rather than the rigid structure generated by basic small-world setups.
- To accurately model this power-law property, it is necessary to transition to the **preferential attachment model**.

4.4 Preferential Attachment Model

16 July 2026 15:02

4.4 Preferential Attachment Model

There exist a variety of scale-free network-modeling algorithms. A well-established one is the model proposed by Barabási and Albert, known as **preferential attachment** or the **Barabási-Albert (BA) model**:

"When new nodes are added to networks, they are more likely to connect to existing nodes that many others have connected to."

- **The Rich-Get-Richer Phenomenon:** This connection likelihood is proportional to the degree of the node that the new node is aiming to connect to. This creates an *aristocrat network* where the higher a node's degree, the higher its probability of gaining new connections.
- **Social Dynamic:** Unlike random graphs where friendships form by pure chance, the preferential attachment model assumes individuals prefer to befriend more gregarious, well-connected others.

- **Algorithmic Setup:**
  - Starts with a small set of m_0 nodes.
  - Adds new nodes one at a time.
  - Each new node connects to $m \leq m_0$ other nodes.
  - Connection to an existing node v_i depends directly on its degree, expressed as:

$$P(v_i) = \frac{d_i}{\sum_j d_j}$$

  - *Note:* The initial m_0 nodes must have a degree of at least 1 to ensure that this probability is non-zero.

Algorithm 4.2 Preferential Attachment

Require: Graph $G(V_0, E_0)$, where $|V_0| = m_0$ and $d_v \geq 1 \; \forall \; v \in V_0$, number of expected connections $m \leq m_0$, time to run the algorithm t

return A scale-free network
`//Initial graph with` m_0 `nodes with degrees at least 1`
$G(V, E) = G(V_0, E_0)$;
for 1 to t **do**
  $V = V \cup \{v_i\}$; `//` `add new node` v_i
  while $d_i \neq m$ **do**
    Connect v_i to a random node $v_j \in V$, $i \neq j$ (i.e., $E = E \cup \{e(v_i, v_j)\}$) with probability $P(v_j) = \frac{d_j}{\sum_k d_k}$.
  end while
end for
Return $G(V, E)$

Two Core Ingredients for a Scale-Free Network:

1. **Growth Element:** New nodes are introduced progressively as time goes by.
2. **Preferential Attachment Element:** New edges connect to node v_i based on its degree probability $P(v_i)$.

Critical Limitation: Removing either of these two components generates networks that fail to be scale-free. While BA models successfully generate power-law degree distributions and small average path lengths, they unfortunately fail to reproduce the high clustering coefficients observed in real-world networks.

4.4.1 Properties of the Preferential Attachment Model

Degree Distribution

Empirical evidence found by simulating the model suggests it generates a scale-free network with a power-law exponent of $b = 2.9 \pm 0.1$.

Mean-Field Proof Breakdown:

Let d_i denote the degree of node v_i. The probability of an edge connecting from a new node to v_i is:

$$P(v_i) = \frac{d_i}{\sum_j d_j}$$

Assuming a mean-field setting, the expected temporal change in d_i is:

$$\frac{\mathrm{d}d_i}{\mathrm{d}t} = mP(v_i) = \frac{md_i}{\sum_j d_j} = \frac{md_i}{2mt} = \frac{d_i}{2t}$$

Note: At each time step, m edges are added; therefore, mt edges accumulate over time, making the total degree sum $\sum_j d_j = 2mt$.

Rearranging and solving this differential equation yields the time-dependent degree equation:

$$d_i(t) = m\left(\frac{t}{t_i}\right)^{0.5}$$

Where t_i represents the time that node v_i was added to the network. Since the expected initial degree is m, we have $d_i(t_i) = m$.

The probability that d_i is less than a given degree d is:

$$P(d_i(t) < d) = P\left(t_i > \frac{m^2 t}{d^2}\right)$$

Assuming uniform intervals for adding nodes, this expands to:

$$P\left(t_i > \frac{m^2 t}{d^2}\right) = 1 - P\left(t_i \leq \frac{m^2 t}{d^2}\right) = 1 - \frac{m^2 t}{d^2}\left(\frac{1}{t + m_0}\right)$$

The factor $\frac{1}{t+m_0}$ represents the probability that one time step has passed because, at the end of the simulation, there are exactly $t + m_0$ nodes in the network.

To find the probability density $P(d)$, we take the derivative:

$$P(d) = \frac{\partial P(d_i(t) < d)}{\partial d} = \frac{2m^2 t}{d^3(t + m_0)}$$

Taking the stationary solution as time approaches infinity ($t \to \infty$):

$$P(d) = \frac{2m^2}{d^3}$$

- **Conclusion:** This derivation establishes a power-law degree distribution with a fixed exponent $b = 3$.
- **Limitation:** While real-world network exponents typically fluctuate within a range (e.g., $[2, 3]$), this basic model exhibits zero variance in its exponent (b is always exactly 3). Other advanced models have been proposed to overcome this lack of flexibility.

Clustering Coefficient

In general, very few triangles are formed in the Barabási-Albert model because edges are created independently and one at a time. Using mean-field analysis, the expected clustering coefficient (C) is calculated as:

$$C = \frac{m_0 - 1}{8} \frac{(\ln t)^2}{t}$$

- **Limitation:** As time t passes, the clustering coefficient continuously shrinks, proving insufficient for modeling the consistently high clustering coefficients seen in real-world systems.

Average Path Length

The average path length (l) of the preferential attachment model increases logarithmically with the total number of nodes $|V|$:

$$l \sim \frac{\ln |V|}{\ln(\ln |V|)}$$

- On average, preferential attachment models generate shorter path lengths than standard random graphs.
- Random graphs are considered accurate at approximating average path lengths, and the same capability holds true for preferential attachment models.

4.4.2 Modeling Real-World Networks with the Preferential Attachment Model

We can simulate real-world structural networks using a preferential attachment model by matching and setting the expected degree parameter m to match empirical metrics.

While the model succeeds at generating a realistic degree distribution and properly accounts for small average path lengths, it consistently fails to capture high clustering coefficients, vastly underestimating them compared to their original real-world values.

Table 4.6: A Comparison between Real-World Networks and Simulated Graphs using Preferential Attachment. C denotes the average clustering coefficient. The last two columns show the average path length and the clustering coefficient for the preferential-attachment graph simulated for the real-world network. Note that average path lengths are modeled properly, whereas the clustering coefficient is underestimated

	Original Network				**Simulated Graph**	
Network	*Size*	*Average Degree*	*Average Path Length*	*C*	*Average Path Length*	*C*
Film Actors	225,226	61	3.65	0.79	4.90	≈ 0.005
Medline Coauthorship	1,520,251	18.1	4.6	0.56	5.36	≈ 0.0002
E.Coli	282	7.35	2.9	0.32	2.37	0.03
C.Elegans	282	14	2.65	0.28	1.99	0.05

4.5 Summary

This chapter reviews three well-established network models designed to replicate commonly observed characteristics of real-world networks: **Random Graphs**, the **Small-World Model**, and **Preferential Attachment**.

1. Random Graphs

Random graphs are built on the assumption that connections between nodes are completely random.

- **Variants:** The two primary variants discussed are $G(n, p)$ and $G(n, m)$.
- **Degree Distribution:** They exhibit a **Poisson degree distribution**.
- **Clustering Coefficient:** Characterized by a small clustering coefficient p.
- **Average Path Length:** They possess a realistic average path length defined by:

$$\frac{\ln |V|}{\ln c}$$

2. The Small-World Model

The small-world model assumes that individuals maintain a fixed number of regular connections alongside a set of random connections. This model successfully generates networks featuring **high transitivity** and **short path lengths**, both of which are prominent features of real-world networks.

- **Construction:** Networks are created by starting with an initial regular ring lattice and randomly rewiring edges. This rewiring process is controlled by the parameter β.
- **Clustering Coefficient:** The clustering coefficient of this model is approximately $(1 - p)^3$ times the clustering coefficient of a regular lattice.
- **Average Path Length:** No analytical solution has been found to approximate the average path length with respect to a regular ring lattice.
- **Empirical Fit:** The model closely resembles many real-world networks when between 1% to 10% of the edges are rewired ($0.01 \leq \beta \leq 0.1$).
- **Limitation:** A major drawback is that it generates a degree distribution similar to the unrealistic Poisson degree distribution found in random graphs.

3. Preferential Attachment Model

The preferential attachment model is centered on the concept that the likelihood of forming a new friendship (connection) depends on the number of friends (connections) an individual already possesses.

- **Network Type:** Generates a **scale-free network** characterized by a **power-law degree distribution**.
- **Mathematical Formula:** If k denotes the degree of a node and p_k represents the fraction of nodes with degree k, the power-law degree distribution is expressed as:

$$p_k = ak^{-b}$$

- **Exponent Value:** Using a mean-field approach, it is proven that this model yields a power-law degree distribution with an exponent of:

$$b = 2.9 \pm 0.1$$

- **Average Path Length:** Exhibits realistic average path lengths that are even smaller than those observed in random graphs.
- **Limitation:** The core caveat of this model is that it generates a small clustering coefficient, which directly contradicts the high clustering coefficients typically seen in real-world networks.

4.7 Exercises

16 July 2026 15:10

Question 1

A scale invariant function $f(\cdot)$ is one such that, for a scalar α,

$$f(\alpha x) = \alpha^c f(x),$$

for some constant c. Prove that the power-law degree distribution is scale invariant.

Answer

A power-law degree distribution is defined as

$$P(k) = Ck^{-\gamma},$$

where:

- C is a normalization constant,
- $\gamma > 1$ is the power-law exponent.

To test scale invariance, replace k with αk:

$$\begin{aligned} P(\alpha k) &= C(\alpha k)^{-\gamma} \\ &= C\alpha^{-\gamma} k^{-\gamma} \\ &= \alpha^{-\gamma} P(k). \end{aligned}$$

This has exactly the required form

$$f(\alpha x) = \alpha^c f(x),$$

where

$$c = -\gamma.$$

Conclusion

Since scaling the argument only multiplies the function by a constant factor,

$$P(\alpha k) = \alpha^{-\gamma} P(k),$$

the power-law degree distribution is **scale invariant**.

Question 2

Assuming that we are interested in a sparse random graph, what should we choose as our p value?

Answer

For an Erdős–Rényi random graph $G(n, p)$,

- Expected degree:

$$\langle k \rangle = p(n-1) \approx pn.$$

A sparse graph has an average degree that remains approximately constant as the number of nodes increases.

Therefore,

$$pn = \text{constant},$$

which implies

$$\boxed{p = \frac{c}{n}},$$

where c is a positive constant.

Conclusion

To obtain a sparse random graph,

$$\boxed{p = O\left(\frac{1}{n}\right)}$$

or equivalently,

$$\boxed{p = \frac{c}{n}}.$$

Question 3

Construct a random graph as follows. Start with n nodes and a given k. Generate all possible combinations of k nodes. For each combination, create a k-cycle with probability

$$\frac{\alpha}{\binom{n-1}{k-2}},$$

where α is a constant.

(a) Calculate the node mean degree and the clustering coefficient.

Answer

Step 1: Mean Degree

A node belongs to

$$\binom{n-1}{k-1}$$

possible k-node combinations.

Each combination is selected with probability

$$p = \frac{\alpha}{\binom{n-1}{k-2}}.$$

Whenever a k-cycle is created, each node gains **two edges**.

Hence,

$$\langle k \rangle = 2\binom{n-1}{k-1} \cdot \frac{\alpha}{\binom{n-1}{k-2}}.$$

Using

$$\frac{\binom{n-1}{k-1}}{\binom{n-1}{k-2}} = \frac{n-k+1}{k-1},$$

we obtain

$$\boxed{\langle k \rangle = 2\alpha \frac{n-k+1}{k-1}.}$$

For large n,

$$\boxed{\langle k \rangle \approx \frac{2\alpha n}{k-1}.}$$

Step 2: Clustering Coefficient

A k-cycle contains only cycle edges.

A node has two neighbors, but those neighbors are **not connected** (except when $k = 3$).

Therefore,

- If $k = 3$,

$$\boxed{C = 1.}$$

- If $k > 3$,

$$\boxed{C = 0.}$$

Final Answers

Mean degree:

$$\boxed{\langle k \rangle = 2\alpha \frac{n-k+1}{k-1}}$$

Clustering coefficient:

$$\boxed{C = \begin{cases} 1, & k = 3, \\ 0, & k > 3. \end{cases}}$$

(b) What is the node mean degree if you create a complete graph instead of the k-cycle?

Answer

A complete graph on k nodes gives each participating node

$$k-1$$

neighbors.

Therefore,

$$\begin{aligned}\langle k\rangle &= (k-1)\binom{n-1}{k-1}\cdot\frac{\alpha}{\binom{n-1}{k-2}}\\ &= (k-1)\alpha\frac{n-k+1}{k-1}.\end{aligned}$$

Hence,

$$\boxed{\langle k\rangle = \alpha(n-k+1).}$$

For large n,

$$\boxed{\langle k\rangle \approx \alpha n.}$$

Question 4

When does phase transition happen in the evolution of random graphs? What happens in terms of changes in network properties at that time?

Answer

In an Erdős–Rényi random graph $G(n,p)$, the phase transition occurs when the average node degree reaches approximately

$$\boxed{\langle k\rangle = 1.}$$

Since

$$\langle k\rangle = p(n-1),$$

the critical probability is

$$\boxed{p_c \approx \frac{1}{n}.}$$

Network changes at the phase transition

- A **giant connected component** suddenly emerges.
- Small isolated components merge into one large component.
- The largest component changes from logarithmic size $O(\log n)$ to linear size $O(n)$.
- Average path lengths become well-defined within the giant component.
- Network connectivity increases rapidly.
- Above the threshold, almost every node belongs to the giant component.
- As p increases further to approximately

$$p \approx \frac{\ln n}{n},$$

the graph becomes connected with high probability.

Conclusion

The critical phase transition occurs at

$$\boxed{p_c \approx \frac{1}{n}}$$

or equivalently,

$$\boxed{\langle k \rangle \approx 1.}$$

At this point, the network undergoes a qualitative change from many small disconnected clusters to a graph containing a giant connected component.

Question 5

Show that in a regular lattice the number of connections between neighbors is given by

$$\frac{3}{8}c(c-2),$$

where c is the average degree.

Answer

Consider a one-dimensional regular lattice (ring lattice) where each node is connected to its c nearest neighbors ($\frac{c}{2}$ on each side). Let

$$m = \frac{c}{2}.$$

Each node has c neighbors.

Step 1: Connections among neighbors on one side

Among the m neighbors on one side, the number of connections is

$$\sum_{i=1}^{m-1}(m-i) = \frac{m(m-1)}{2}.$$

Since there are two sides,

$$2 \cdot \frac{m(m-1)}{2} = m(m-1).$$

Step 2: Connections between opposite-side neighbors

Each left-side neighbor connects to some right-side neighbors. The number of these cross-connections is

$$\frac{m(m-1)}{2}.$$

Step 3: Total number of neighbor-neighbor connections

Adding both contributions,

$$\begin{aligned} E_N &= m(m-1) + \frac{m(m-1)}{2} \\ &= \frac{3}{2}m(m-1). \end{aligned}$$

Substituting $m = \frac{c}{2}$,

$$\begin{aligned} E_N &= \frac{3}{2}\left(\frac{c}{2}\right)\left(\frac{c}{2}-1\right) \\ &= \frac{3}{2} \cdot \frac{c(c-2)}{4} \\ &= \boxed{\frac{3}{8}c(c-2).} \end{aligned}$$

Conclusion

The number of connections between the neighbors of a node in a regular lattice is

$$\boxed{E_N = \frac{3}{8}c(c-2).}$$

Question 6

Show how the clustering coefficient can be computed in a regular lattice of degree k.

Answer

For a regular lattice:

- Every node has degree

$$k.$$

- The maximum possible number of links among its neighbors is

$$\binom{k}{2} = \frac{k(k-1)}{2}.$$

From Question 5, the actual number of links among neighbors is

$$\frac{3}{8}k(k-2).$$

The clustering coefficient is

$$C = \frac{\text{Actual links among neighbors}}{\text{Maximum possible links}}.$$

Substituting,

$$\begin{aligned} C &= \frac{\frac{3}{8}k(k-2)}{\frac{k(k-1)}{2}} \\ &= \frac{3(k-2)}{4(k-1)}. \end{aligned}$$

Hence,

$$\boxed{C = \frac{3(k-2)}{4(k-1)}.}$$

Conclusion

The clustering coefficient of a regular lattice of degree k is

$$\boxed{C = \frac{3(k-2)}{4(k-1)}.}$$

As k becomes large,

$$C \to \frac{3}{4}.$$

Question 7

Why are random graphs incapable of modeling real-world graphs? What are the differences between random graphs, regular lattices, and small-world models?

Answer

Random graphs fail to accurately model most real-world networks because they lack the structural characteristics commonly observed in social, biological, technological, and information networks.

Comparison of Network Models

Property	Random Graph	Regular Lattice	Small-World Model
Edge placement	Random	Regular and ordered	Mostly regular with a few random shortcuts
Degree distribution	Binomial/Poisson	Uniform	Similar to lattice
Clustering coefficient	Low	High	High
Average path length	Short	Long	Short
Local structure	Weak	Strong	Strong
Global connectivity	Good	Poor	Good
Models real-world networks?	No	No	Yes (better approximation)

Why random graphs are inadequate

- Very low clustering coefficient.
- Little or no community structure.
- Degree distribution is typically Poisson rather than heavy-tailed.
- Cannot reproduce the coexistence of high clustering and short path lengths found in many real networks.

Regular lattices

- High clustering.
- Very long average path lengths.
- Information spreads slowly due to the absence of shortcuts.

Small-world networks

- Introduced by randomly rewiring a small fraction of lattice edges.
- Preserve high clustering.
- Greatly reduce average path length through shortcut connections.
- Capture both local neighborhood structure and efficient global communication.

Conclusion

Small-world networks provide a much better approximation of many real-world systems because they combine the high clustering of regular lattices with the short average path lengths of random graphs.

Question 8

Compute the average path length in a regular lattice.

Answer

Consider a one-dimensional ring lattice with:

- n nodes,
- each node connected to k nearest neighbors.

Each step along the lattice can move up to

$$\frac{k}{2}$$

nodes closer to the destination.

The average distance between two randomly chosen nodes on the ring is approximately

$$\frac{n}{4}.$$

Since each hop advances by $\frac{k}{2}$ nodes, the average number of hops is

$$L \approx \frac{\frac{n}{4}}{\frac{k}{2}} = \frac{n}{2k}.$$

Hence,

$$\boxed{L \approx \frac{n}{2k}.}$$

Interpretation

- The average path length grows **linearly** with the number of nodes:

$$L = O(n).$$

- As the degree k increases, the average path length decreases inversely with k.

Conclusion

The average path length of a regular lattice with n nodes and degree k is approximately

$$\boxed{L \approx \frac{n}{2k},}$$

which is much larger than the logarithmic path lengths observed in random and small-world networks.

Question 9

Question:
As a function of k, what fraction of pages on the web have k in-links, assuming that a normal distribution governs the probability of webpages choosing their links? What if we have a power-law distribution instead?

Answer:

Let $P(k)$ denote the fraction (probability) of webpages having exactly k incoming links (in-degree).

Case 1: Normal (Gaussian) Distribution

If webpage linking follows a normal distribution, then the probability of a page having k in-links is

$$P(k) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(k-\mu)^2}{2\sigma^2}\right),$$

where:

- μ = average number of in-links,
- σ = standard deviation.

Characteristics:

- Most webpages have approximately μ in-links.
- Very few pages have extremely high or extremely low numbers of in-links.
- The distribution is symmetric about the mean.

Case 2: Power-Law Distribution

If the web follows a power-law distribution, then

$$P(k) = Ck^{-\gamma},$$

where:

- C is the normalization constant,
- γ is typically between 2 and 3.

Characteristics:

- Most webpages have very few in-links.
- A small number of webpages (hubs) accumulate a very large number of in-links.
- The distribution has a long tail and is not symmetric.

Conclusion

The real World Wide Web is observed to follow a **power-law distribution** rather than a normal distribution. This explains the existence of highly connected hubs such as major search engines, social media platforms, and popular websites.

Question 10

Question:
In the Barabási–Albert (BA) model, consider two simplified models:

- **Growth model (G):** New nodes are continuously added, and each new node connects uniformly at random.
- **Preferential Attachment model (A):** No new nodes are added. Existing nodes are selected according to degree and connected to another existing node.

1. Compute the degree distribution for these two models.
2. Determine whether these models generate scale-free networks. What does this prove?

Answer

(a) Growth Model (G)

In this model:

- One new node is added at every time step.
- Each new node makes m connections.
- Existing nodes are chosen uniformly at random.

The probability that an existing node receives a new edge is

$$P = \frac{m}{m_0 + t - 1},$$

where

- m_0 = initial number of nodes,
- t = current time step.

Since every node has an equal chance of receiving new edges, degree growth is independent of current degree.

The resulting degree distribution is

$$P(k) \propto e^{-k/\kappa},$$

where κ is a characteristic degree.

This is an **exponential distribution**, meaning high-degree nodes become exponentially rare.

(b) Preferential Attachment Model (A)

In this model:

- No new nodes are added.
- Existing nodes are selected with probability proportional to their degree,

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}.$$

At every step, new edges are added between existing nodes.

Because the number of nodes remains fixed while edges increase, the average degree continually increases with time.

The degree distribution does **not** converge to a stationary power law. Instead, it progressively shifts toward larger degrees as the network becomes denser.

Thus,

$$P(k) \neq Ck^{-\gamma}.$$

Instead, the distribution evolves over time and is not scale-free.

(c) Do These Models Generate Scale-Free Networks?

Model	Degree Distribution	Scale-Free?
Growth only (G)	Exponential $P(k) \propto e^{-k/\kappa}$	**No**
Preferential Attachment only (A)	Non-stationary, increasingly dense	**No**

(d) What Does This Prove?

Neither **growth alone** nor **preferential attachment alone** is sufficient to generate a scale-free network.

A scale-free network emerges **only when both mechanisms operate together**, as in the full **Barabási–Albert (BA) model**, where

- new nodes continuously enter the network (**growth**), and
- new nodes preferentially attach to already well-connected nodes (**preferential attachment**).

The BA model produces the characteristic power-law degree distribution

$$P(k) \propto k^{-3},$$

which explains the emergence of hubs observed in many real-world networks such as the World Wide Web, citation networks, and social networks.

T2-Chapter 2 Background and Traditional Approaches

09 July 2026 09:14

Chapter 2

Background and Traditional Approaches

- This chapter establishes the methodological context for graph representation learning by exploring the machine learning methods used on graphs prior to the rise of modern deep learning.
- It introduces foundational graph analysis concepts that are essential for understanding more advanced approaches.
- The study of traditional approaches is organized around different graph learning tasks:
  - **Node and Graph Classification:** Addressed through basic graph statistics and kernel methods.
  - **Relation Prediction:** Evaluated by measuring the overlap between node neighborhoods, serving as a strong heuristic.
  - **Clustering and Community Detection:** Introduced via spectral clustering using graph Laplacians, a highly studied algorithm that introduces recurring mathematical concepts.

2.1 Graph Statistics and Kernel Methods

- Traditional graph classification relies on the standard machine learning paradigm that predates deep learning.
- **The Traditional Machine Learning Pipeline for Graphs:**
  1. **Feature Extraction:** Statistics or features are extracted using heuristic functions or domain knowledge.
  2. **Classification:** These extracted features are fed into standard machine learning classifiers, such as logistic regression.

- **Feature Progression:**
  - The methodology starts by defining important **node-level statistics and features**.
  - These node-level features are then generalized into **graph-level statistics**.
  - Ultimately, this generalization is extended to design **kernel methods** over graphs.
- The overarching goal of this traditional approach is to utilize these heuristic statistics and graph properties as reliable features within standard machine learning pipelines.
- **Visual Example:** Graph structures are highly effective for mapping complex relationships, such as visualizing the marriage ties between prominent families (like the Medici, Strozzi, and Guadagni) in 15th-century Florence.

2.1.1 Node-level statistics and features

Overview & Motivation

- **Context**: Node-level statistics and features can be motivated by looking at a classic social network: the **15th-century Florentine marriage network** (studied by Padgett and Ansell, 1993).
- **The Historical Case**: This network illustrates the rise in power of the **Medici family**, who came to dominate Florentine politics. Political marriages were crucial for consolidating power, meaning the network's structural connections encode substantial information about the era's political climate.
- **Machine Learning Perspective**: The core objective is to determine what features or statistics a machine learning model could use to characterize individual nodes or predict phenomena like the Medici's rise to power (i.e., distinguishing a specific node from the rest of the graph).
- **Applications**: These properties serve as input features for node classification models (e.g., logistic regression). While the Florentine network itself is too small to practically train an ML model, its properties generalize across a wide variety of real-world node classification tasks.

Node degree

- **Definition**: The most fundamental and straightforward node feature. For a given node $u \in \mathcal{V}$, the degree d_u simply counts the number of edges incident to it.
- **Mathematical Formula**:

$$d_u = \sum_{v \in \mathcal{V}} \mathbf{A}[u, v]$$

- **Graph Variations**:
  - For **directed** and **weighted** graphs, you can differentiate between incoming (in-degree) and outgoing (out-degree) edges by summing over the rows or columns of the adjacency matrix $\mathbf{A}$.
- **Significance**: It is an essential statistic and often one of the most informative features in traditional ML models applied to node-level tasks.
- **Florentine Network Example**:
  - The Medici family has the **highest degree** in the graph.
  - However, their degree only outmatches the two closest families (the Strozzi and the Guadagni) by a ratio of 3 to 2, indicating a need for more discriminative features to fully isolate their influence.

Node centrality

While node degree measures local connectivity (number of neighbors), it does not necessarily capture the global **importance** of a node within a graph. Node centrality measures offer more sophisticated metrics to evaluate importance.

1. Eigenvector Centrality

- **Core Concept**: A node's importance is proportional to the average centrality of its neighbors. It takes into account how important a node's neighbors are, rather than just counting them.
- **Mathematical Definition (Recurrence Relation)**:

$$e_u = \frac{1}{\lambda} \sum_{v \in \mathcal{V}} \mathbf{A}[u, v] e_v \quad \forall u \in \mathcal{V}$$

(where λ is a constant)

- **Vector Notation**: Rewriting the recurrence relation defines the standard eigenvector equation for the adjacency matrix:

$$\lambda \mathbf{e} = \mathbf{A}\mathbf{e}$$

- **Perron-Frobenius Theorem**: To guarantee that we obtain strictly **positive centrality values**, this theorem is applied, dictating that the vector of centrality values $\mathbf{e}$ corresponds to the eigenvector associated with the **largest eigenvalue** of $\mathbf{A}$.

2. Random Walk & Power Iteration Interpretation

- **Random Walk View**: Eigenvector centrality can be interpreted as ranking the likelihood that a node is visited during an infinite-length random walk on the graph.
- **Power Iteration**: Because λ is the leading eigenvalue of $\mathbf{A}$, the centrality vector $\mathbf{e}$ can be computed iteratively via:

$$\mathbf{e}^{(t+1)} = \mathbf{A}\mathbf{e}^{(t)}$$

- **Step-by-Step Evolution**:
  - If initialized with $\mathbf{e}^{(0)} = (1, 1, \ldots, 1)^\top$, the first iteration $\mathbf{e}^{(1)}$ yields the **degrees** of all nodes.
  - At any iteration $t \geq 1$, the vector $\mathbf{e}^{(t)}$ contains the number of **paths of length-t** arriving at each node.
  - Iterating indefinitely yields a score proportional to the number of times a node is visited on paths of infinite length.

3. Application and Alternative Centrality Measures

- **Florentine Network Evaluation**: Eigenvector centrality clearly highlights the Medici family as the most influential. Their normalized centrality value is **0.43**, compared to the next-highest value of **0.36**.
- **Other Essential Centrality Measures**:
  - **Betweenness Centrality**: Measures how often a node lies on the shortest path between two other nodes.
  - **Closeness Centrality**: Measures the average shortest path length between a node and all other nodes in the network.

The clustering coefficient

- Measures of importance, such as degree and centrality, are highly effective for distinguishing prominent nodes within a graph (e.g., the prominent Medici family in the Florentine marriage network).
- However, these measures may fail to distinguish between less prominent nodes that have different structural roles.
- **Example from the Florentine network:**
  - The Peruzzi and Guadagni nodes have very similar degrees (3 vs. 4) and similar eigenvector centralities (0.28 vs. 0.29).
  - Despite these similarities, their visual structures are distinct: the Peruzzi family is located within a relatively tight-knit cluster, whereas the Guadagni family exists in a "star-like" role.
- This structural distinction is quantified using variations of the *clustering coefficient*, which calculates the proportion of closed triangles within a node's local neighborhood.
- The standard *local variant* (Watts and Strogatz, 1998) is defined by the following formula:

$$c_u = \frac{|\{(v_1, v_2) \in \mathcal{E} : v_1, v_2 \in \mathcal{N}(u)\}|}{\binom{d_u}{2}}$$

- **Numerator**: Counts the actual number of edges existing between the neighbors of node u. The node neighborhood is denoted mathematically as $\mathcal{N}(u) = \{v \in \mathcal{V} : (u, v) \in \mathcal{E}\}$.
- **Denominator**: Calculates the total possible number of pairs of nodes in u's neighborhood.

- **Interpretation of the metric:**
  - The clustering coefficient measures how tightly clustered a specific node's neighborhood is.
  - A coefficient of 1 indicates perfect clustering, meaning all of node u's neighbors are also directly connected to each other.
  - In the Florentine graph example, the highly clustered Peruzzi node has a coefficient of 0.66, while the star-like Guadagni node has a coefficient of 0.
- **Real-world characteristics**: Real-world networks across biological and social sciences tend to possess significantly higher clustering coefficients than one would expect if the network's edges were sampled at random (Watts and Strogatz, 1998).
- There are numerous variations of the clustering coefficient, including versions adapted for directed graphs, which are reviewed by Newman (2018).

Closed triangles, ego graphs, and motifs.

- An alternative perspective on the clustering coefficient is that it acts as a count of closed triangles inside a node's local neighborhood.
- More precisely, the coefficient is related to the ratio between the *actual* number of triangles and the *total possible* number of triangles within a node's **ego graph**.
  - An *ego graph* is defined as the specific subgraph that contains the target node, its neighbors, and all of the edges connecting those nodes within that neighborhood.
- **Generalization to complex structures:**
  - The core idea of counting triangles can be expanded to counting arbitrary *motifs* or *graphlets* within a node's ego graph.
  - Instead of just triangles, analysis can include more complex structures, such as cycles of a particular length.
  - Nodes can then be characterized based on the frequency count of how often these different motifs appear in their respective ego graphs.
- By analyzing a node's ego graph through motif counting, the process effectively transforms the task from computing node-level statistics into a graph-level task.

2.1.2 Graph-level features and graph kernels

2.1.2 Graph-level features and graph kernels

- Moving from node-level classification to graph-level classification requires specific features to represent the entire graph.
- An example task is classifying the solubility of molecules based on their underlying graph structure.
- **Graph kernel methods** are approaches used to design features for graphs or to create implicit kernel functions that can be utilized in machine learning models.
- This area encompasses methods that extract explicit feature representations rather than just defining implicit similarity measures between graphs.

Bag of nodes

- This is the simplest approach for defining a graph-level feature.
- It works by simply aggregating node-level statistics into histograms or other summary statistics.
- These statistics can be based on properties such as node degrees, centralities, and clustering coefficients.
- **Drawback:** Because this method relies entirely on local node-level information, it can miss important global properties present within the graph.

The Weisfeiler-Lehman kernel

- This approach improves upon the basic "bag of nodes" model by utilizing a strategy called *iterative neighborhood aggregation*.
- The goal is to extract richer node-level features that contain information beyond a node's local ego graph, and then aggregate these into a graph-level representation.
- The Weisfeiler-Lehman (WL) algorithm is widely recognized as one of the most important and well-known strategies in this domain.

Steps of the WL Algorithm:

1. **Initialization:** First, assign an initial label $l^{(0)}(v)$ to each node. In most graphs, this label is simply the node's degree (i.e., $l^{(0)}(v) = d_v \ \forall v \in V$).
2. **Iteration:** Next, iteratively assign a new label to each node by hashing the multi-set of the current labels found within that node's neighborhood. This is calculated as $l^{(i)}(v) = \text{HASH}(\{\{l^{(i-1)}(u) \ \forall u \in \mathcal{N}(v)\}\})$. The double-braces denote a multi-set, and the HASH function maps each unique multi-set to a completely new, unique label.
3. **Aggregation:** After running K iterations of this re-labeling process, each node possesses a label $l^{(K)}(v)$ that summarizes the structure of its K-hop neighborhood. Histograms or other summary statistics are then computed over these labels to form the final feature representation for the graph.

Key Properties:

- The WL kernel is computed by measuring the difference between the resultant label sets for two given graphs.
- It is a well-studied kernel with important theoretical properties.
- The WL algorithm provides a popular way to approximate graph isomorphism; checking if two graphs share the exact same label set after K rounds is known to solve the isomorphism problem for a broad set of graphs.

Graphlets and path-based methods

Graphlet Methods:

- Another strategy for defining features over graphs is to count the occurrences of different small subgraph structures, which are referred to as *graphlets*.
- The graphlet kernel involves formally enumerating all possible graph structures of a specific size and counting how many times they occur within the full graph.
- For instance, in a simple graph, there are four different possible size-3 graphlets: the complete graphlet, the disconnected graphlet, the single-edge graphlet, and the two-edge graphlet.
- **Challenge:** Counting these graphlets is a combinatorially difficult problem, though various approximations have been proposed to help mitigate this issue.

Path-Based Methods:

- As an alternative to enumerating all graphlets, path-based methods examine the different types of paths that occur in a graph.
- **Random walk kernel:** This method involves running random walks over the graph and subsequently counting the occurrences of different degree sequences.
- **Shortest-path kernel:** This approach is similar to the random walk kernel but relies exclusively on the shortest-paths between nodes instead of random walks.
- **Advantage:** Characterizing graphs based on walks and paths is a powerful technique because it successfully extracts rich structural information while avoiding many of the combinatorial pitfalls typically associated with graph data.

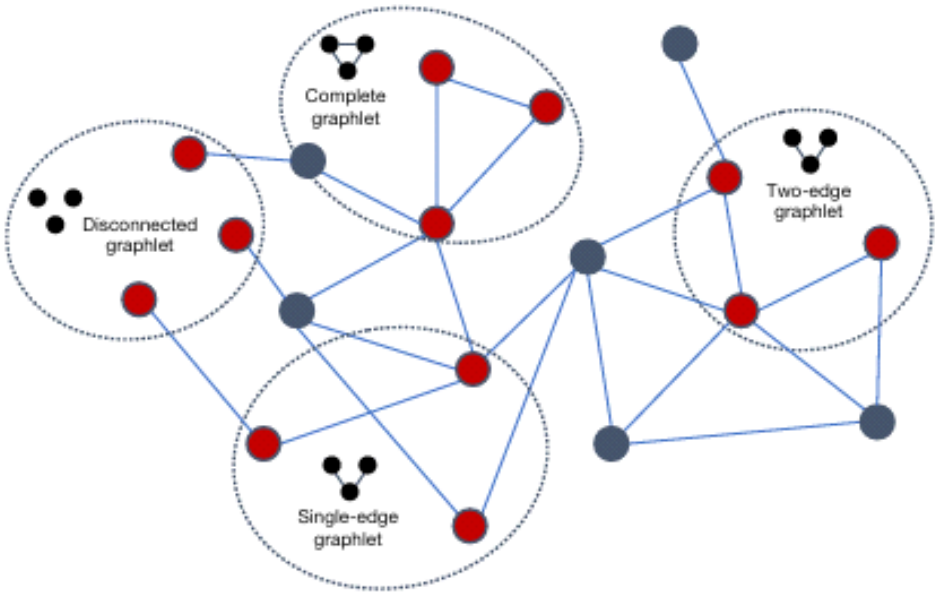


Figure 2.2: The four different size-3 graphlets that can occur in a simple graph.

2.2 Neighborhood Overlap Detection

09 July 2026 09:23

2.2 Neighborhood Overlap Detection

As seen in `image_6697de.png`, individual node-level or graph-level statistics are highly useful for classification tasks but are fundamentally limited because they **do not quantify the relationships between pairs of nodes**. Consequently, they are ineffective for **relation prediction** (the task of predicting whether an edge exists between two nodes).

To address this, statistical measures of **neighborhood overlap** are used to quantify the extent to which two nodes are related.

1. Basic Neighborhood Overlap

The simplest measure of neighborhood overlap directly counts the number of common neighbors shared by two nodes, u and v:

$$\mathbf{S}[u, v] = |\mathcal{N}(u) \cap \mathcal{N}(v)|$$

- $\mathcal{N}(u)$: Denotes the set of neighbors of node u.
- $\mathbf{S} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$: Represents the **similarity matrix** summarizing all pairwise node statistics.

2. Relation Prediction Baseline Strategy

Although these statistical metrics do not involve machine learning, they serve as powerful baseline models. A common strategy is to assume that the likelihood of an edge (u, v) existing is directly proportional to its neighborhood overlap score:

$$P(\mathbf{A}[u, v] = 1) \propto \mathbf{S}[u, v]$$

To implement this practically, a threshold is set to determine when to predict the existence of an edge.

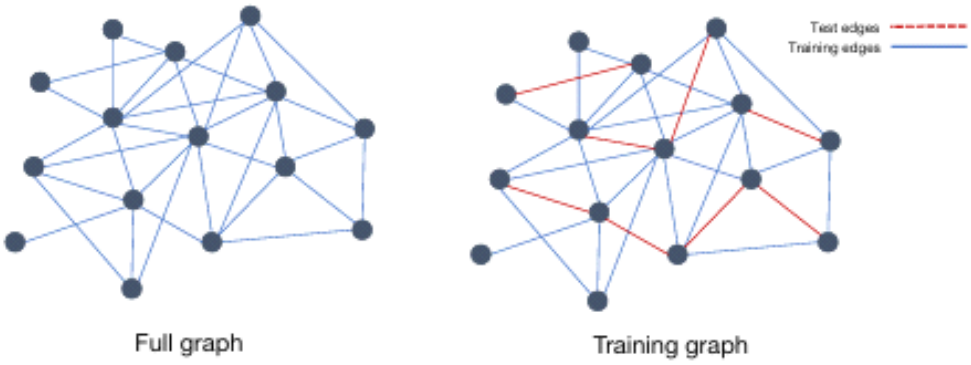


Figure 2.3: An illustration of a full graph and a subsampled graph used for training. The dotted edges in the training graph are removed when training a model or computing the overlap statistics. The model is evaluated based on its ability to predict the existence of these held-out *test edges*.

3. Evaluation Setup (Figure 2.3)

As shown in Figure 2.3 within `image_66981e.png`, the machine learning task is framed as follows:

- **Training Graph**: We assume we only know a subset of true edges, denoted as $\mathcal{E}_{\text{train}} \subset \mathcal{E}$.
- **Test Edges**: Unseen/held-out edges (shown as red dashed lines in `image_66981e.png`) are removed during training and while computing overlap statistics.
- **Goal**: Compute node-node similarity measures on the training graph to accurately predict the existence of these hidden test edges.

2.2.1 Local overlap measures

As detailed in `image_6697fc.png`, local overlap statistics are functions that scale or normalize the number of common neighbors $|\mathcal{N}(u) \cap \mathcal{N}(v)|$.

The Importance of Normalization

Note: Normalization is critical. Without it, neighborhood overlap measures become heavily biased toward predicting edges for high-degree nodes (hubs) simply because they naturally have more neighbors.

The text across `image_6697fc.png` and `image_66981e.png` defines several standard normalized local overlap metrics:

- **Sorensen Index:** Normalizes the common neighbor count by the sum of the node degrees.

$$\mathbf{S}_{\text{Sorensen}}[u, v] = \frac{2|\mathcal{N}(u) \cap \mathcal{N}(v)|}{d_u + d_v}$$

- **Salton Index:** Normalizes the common neighbor count by the geometric mean (product) of the node degrees.

$$\mathbf{S}_{\text{Salton}}[u, v] = \frac{|\mathcal{N}(u) \cap \mathcal{N}(v)|}{\sqrt{d_u d_v}}$$

- **Jaccard Overlap:** Normalizes the intersection of the neighborhoods by their union.

$$\mathbf{S}_{\text{Jaccard}}[u, v] = \frac{|\mathcal{N}(u) \cap \mathcal{N}(v)|}{|\mathcal{N}(u) \cup \mathcal{N}(v)|}$$

Moving Beyond Simple Counts: Neighbor Importance

Advanced measures consider the **importance** of the common neighbors instead of treating all shared neighbors equally. Based on `image_66981e.png` and `image_66983b.png`, the **Resource Allocation (RA)** index and the **Adamic-Adar (AA)** index implement this philosophy:

- **Resource Allocation (RA) Index:** Sums the inverse degrees of all mutual neighbors.

$$\mathbf{S}_{\text{RA}}[v_1, v_2] = \sum_{u \in \mathcal{N}(v_1) \cap \mathcal{N}(v_2)} \frac{1}{d_u}$$

- **Adamic-Adar (AA) Index:** Sums the inverse logarithm of the degrees of all mutual neighbors.

$$\mathbf{S}_{\text{AA}}[v_1, v_2] = \sum_{u \in \mathcal{N}(v_1) \cap \mathcal{N}(v_2)} \frac{1}{\log(d_u)}$$

- **Intuition for RA and AA indices:** Both of these specific measures assign more weight to common neighbors that have a low degree. This is based on the intuition that sharing a low-degree neighbor is much more informative than sharing a high-degree neighbor.

2.2.2 Global overlap measures

2.2.2 Global overlap measures

- Local overlap measures act as highly effective heuristics for link prediction and can achieve performance competitive with advanced deep learning methods.
- A major limitation of local approaches is that they only consider local node neighborhoods.
- For instance, two nodes might lack local overlap in their immediate neighborhoods but still belong to the same community within the broader graph.
- Global overlap statistics are designed to take these broader, non-local relationships into account.

Katz index

- **Definition:** The Katz index is recognized as the most fundamental global overlap statistic.
- **Computation:** It is calculated by counting the number of paths of *all lengths* that exist between a given pair of nodes.
- **Formula (Equation 2.14):**

$$\mathbf{S}_{\text{Katz}}[u, v] = \sum_{i=1}^{\infty} \beta^i \mathbf{A}^i[u, v]$$

- **The Weighting Parameter (β):** The parameter $\beta \in \mathbb{R}^+$ is a user-defined value that dictates the weight assigned to short paths versus long paths. Choosing a small value for $\beta < 1$ effectively down-weights the importance of longer paths in the network.
- **Matrix Solution (Equation 2.15):** Based on the properties of matrix series, the full matrix of node-node similarity values, $\mathbf{S}_{\text{Katz}} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, is computed as:

$$\mathbf{S}_{\text{Katz}} = (\mathbf{I} - \beta \mathbf{A})^{-1} - \mathbf{I}$$

Geometric series of matrices

The Katz index represents a specific application of a geometric series of matrices, which are common in graph analysis and representation learning.

Theorem 1
Let $\mathbf{X}$ be a real-valued square matrix and let λ_1 denote the largest eigenvalue of $\mathbf{X}$. Then:

$$(\mathbf{I} - \mathbf{X})^{-1} = \sum_{i=0}^{\infty} \mathbf{X}^i$$

if and only if $\lambda_1 < 1$ *and* $(\mathbf{I} - \mathbf{X})$ *is non-singular*.

Proof Steps:

- Let $s_n = \sum_{i=0}^{n} \mathbf{X}^i$.
- Multiplying by $\mathbf{X}$ yields: $\mathbf{X}s_n = \sum_{i=0}^{n} \mathbf{X}^{i+1} = \sum_{i=1}^{n+1} \mathbf{X}^i$.
- Subtracting the two equations gives: $s_n - \mathbf{X}s_n = \sum_{i=0}^{n} \mathbf{X}^i - \sum_{i=1}^{n+1} \mathbf{X}^i$.
- Factoring out s_n: $s_n(\mathbf{I} - \mathbf{X}) = \mathbf{I} - \mathbf{X}^{n+1}$.
- Solving for s_n: $s_n = (\mathbf{I} - \mathbf{X}^{n+1})(\mathbf{I} - \mathbf{X})^{-1}$.
- If $\lambda_1 < 1$, the limit as $n \to \infty$ of $\mathbf{X}^n = 0$.
- Therefore, the limit is $\lim_{n\to\infty} s_n = \mathbf{I}(\mathbf{I} - \mathbf{X})^{-1} = (\mathbf{I} - \mathbf{X})^{-1}$.

Leicht, Holme, and Newman (LHN) similarity

- **The Bias in Katz Index:** A major issue with the Katz index is its strong bias toward node degree; it naturally assigns higher similarity scores to high-degree nodes because they participate in a larger number of paths.
- **The LHN Solution:** To mitigate this bias, the proposed metric calculates the ratio between the actual number of observed paths and the *number of expected paths between two nodes*.
- **Expected Paths Ratio (Equation 2.16):**

$$\frac{\mathbf{A}^i}{\mathbb{E}[\mathbf{A}^i]}$$

- **Configuration Model:** The expectation $\mathbb{E}[\mathbf{A}^i]$ is derived using a configuration model, which assumes a random graph drawn with the exact same set of node degrees as the original graph.
- **Expected Paths of Length 1 (Equation 2.17):** Under this random model, the likelihood of an edge is proportional to the product of the two node degrees, where $m = |\mathcal{E}|$ represents total edges:

$$\mathbb{E}[\mathbf{A}[u, v]] = \frac{d_u d_v}{2m}$$

- **Expected Paths of Length 2 (Equation 2.18):** Calculated by accounting for an intermediate vertex u, where an edge from v_1 hits u and another edge leaves u to hit v_2:

$$\mathbb{E}[\mathbf{A}^2[v_1, v_2]] = \frac{d_{v_1} d_{v_2}}{(2m)^2} \sum_{u \in \mathcal{V}} (d_u - 1) d_u$$

- **Intractability and Approximation:** Analytical computation becomes intractable for paths longer than three ($i > 2$). For larger lengths, the expected growth in paths is approximated using the dominant eigenvector and the largest eigenvalue λ_1 of the adjacency matrix $\mathbf{A}$.
  - Growth approximation relation (Equation 2.19): $\mathbf{A}\mathbf{p}_i = \lambda_1 \mathbf{p}_{i-1}$.
- **Approximated Expectation (Equation 2.20):**

$$\mathbb{E}[\mathbf{A}^i[u, v]] = \frac{d_u d_v \lambda_1^{i-1}}{2m}$$

- **The Normalized LNH Index (Equation 2.21):** By combining these concepts, the normalized LNH index (which stands for the authors' initials) is formed, prioritizing nodes that have more paths than statistically expected:

$$\mathbf{S}_{\text{LNH}}[u, v] = \mathbf{I}[u, v] + \frac{2m}{d_u d_v} \sum_{i=0}^{\infty} \beta^i \lambda_1^{1-i} \mathbf{A}^i[u, v]$$

- **Matrix Series Solution for LNH (Equation 2.22):** Ignoring diagonal terms and using Theorem 1, the matrix series solution is written as:

$$\mathbf{S}_{\text{LNH}} = 2\alpha m \lambda_1 \mathbf{D}^{-1} (\mathbf{I} - \frac{\beta}{\lambda_1} \mathbf{A})^{-1} \mathbf{D}^{-1}$$

(where $\mathbf{D}$ is a matrix with node degrees along the diagonal)

Geometric series of matrices The Katz index is one example of a geometric series of matrices, variants of which occur frequently in graph analysis and graph representation learning. The solution to a basic geometric series of matrices is given by the following theorem:

Theorem 1. *Let* $\mathbf{X}$ *be a real-valued square matrix and let* λ_1 *denote the largest eigenvalue of* $\mathbf{X}$*. Then*

$$(\mathbf{I} - \mathbf{X})^{-1} = \sum_{i=0}^{\infty} \mathbf{X}^i$$

if and only if $\lambda_1 < 1$ *and* $(\mathbf{I} - \mathbf{X})$ *is non-singular.*

Proof. Let $s_n = \sum_{i=0}^{n} \mathbf{X}^i$ then we have that

$$\begin{aligned} \mathbf{X}s_n &= \mathbf{X}\sum_{i=0}^{n} \mathbf{X}^i \\ &= \sum_{i=1}^{n+1} \mathbf{X}^i \end{aligned}$$

and

$$\begin{aligned} s_n - \mathbf{X}s_n &= \sum_{i=0}^{n} \mathbf{X}^i - \sum_{i=1}^{n+1} \mathbf{X}^i \\ s_n(\mathbf{I} - \mathbf{X}) &= \mathbf{I} - \mathbf{X}^{n+1} \\ s_n &= (\mathbf{I} - \mathbf{X}^{n+1})(\mathbf{I} - \mathbf{X})^{-1} \end{aligned}$$

And if $\lambda_1 < 1$ we have that $\lim_{n\to\infty} \mathbf{X}^n = 0$ so

$$\begin{aligned} \lim_{n\to\infty} s_n &= \lim_{n\to\infty} (\mathbf{I} - \mathbf{X}^{n+1})(\mathbf{I} - \mathbf{X})^{-1} \\ &= \mathbf{I}(\mathbf{I} - \mathbf{X})^{-1} \\ &= (\mathbf{I} - \mathbf{X})^{-1} \end{aligned}$$

□

Random walk methods

- **Overview:** Unlike measures that rely on exact counts of paths over a graph, this set of global similarity measures utilizes **random walks**.
- **Personalized PageRank:** A primary example is the Personalized PageRank algorithm, which is a variant of the standard PageRank approach.
- **Stochastic Matrix:** To compute this, we first define the stochastic matrix:

$$\mathbf{P} = \mathbf{A}\mathbf{D}^{-1}$$

The Recurrence Relation

The algorithm relies on computing a stationary probability distribution for a random walk starting at a specific node, defined by the following recurrence:

$$\mathbf{q}_u = c\mathbf{P}\mathbf{q}_u + (1-c)\mathbf{e}_u$$

(Equation 2.23)

- **Components of the Equation:**
  - $\mathbf{e}_u$: A one-hot indicator vector for the starting node u.
  - $\mathbf{q}_u[v]$: Represents the stationary probability that a random walk originating at node u will visit node v.
  - c: The probability term dictating whether the random walk restarts at node u at any given timestep.
- **The Role of Restart Probability (c):** * Without this restart probability, the random walk would merely converge to a normalized variant of eigenvector centrality.
  - With the restart probability included, the walks are continually "teleported" back to node u. This results in a measure of importance that is inherently specific and localized to node u.

The Closed-Form Solution
The mathematical solution to the recurrence equation above is given by:

$$\mathbf{q}_u = (1-c)(\mathbf{I} - c\mathbf{P})^{-1}\mathbf{e}_u$$

(Equation 2.24)

Node-Node Random Walk Similarity
Based on the personalized random walk probabilities, a global node-node similarity measure is defined as:

$$\mathbf{S}_{\mathrm{RW}}[u, v] = \mathbf{q}_u[v] + \mathbf{q}_v[u]$$

(Equation 2.25)

- **Interpretation:** This final similarity metric between a pair of nodes is directly proportional to the likelihood of reaching each node via a random walk that started from the other node.

2.3 Graph Laplacians and Spectral Methods

09 July 2026 09:47

2.3 Graph Laplacians and Spectral Methods

- This section transitions from traditional classification and relation prediction approaches to the problem of learning to cluster nodes in a graph.
- The concepts discussed serve to motivate the task of learning low-dimensional embeddings of nodes.
- It introduces important matrices used to represent graphs and provides foundational concepts for *spectral graph theory*.

2.3.1 Graph Laplacians

- While adjacency matrices can represent graphs without losing any information, there are alternative matrix representations that possess useful mathematical properties.
- These alternative representations are known as *Laplacians* and are created through various transformations of the adjacency matrix.

Unnormalized Laplacian

- The most basic Laplacian matrix is the unnormalized Laplacian.
- The unnormalized Laplacian is defined by the equation:

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

  where $\mathbf{A}$ represents the adjacency matrix and $\mathbf{D}$ represents the degree matrix.
- The Laplacian matrix of a simple graph is symmetric ($\mathbf{L}^\top = \mathbf{L}$).
- The Laplacian matrix is positive semi-definite ($\mathbf{x}^\top \mathbf{L}\mathbf{x} \geq 0, \forall \mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$).
- A specific vector identity holds for all $\mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{x}^\top \mathbf{L}\mathbf{x} = \frac{1}{2} \sum_{u \in \mathcal{V}} \sum_{v \in \mathcal{V}} \mathbf{A}[u, v](\mathbf{x}[u] - \mathbf{x}[v])^2$$

which simplifies to:

$$\sum_{(u,v)\in\mathcal{E}} (\mathbf{x}[u] - \mathbf{x}[v])^2$$

- The matrix $\mathbf{L}$ possesses $|V|$ non-negative eigenvalues ordered as: $0 = \lambda_{|\mathcal{V}|} \leq \lambda_{|\mathcal{V}|-1} \leq \ldots \leq \lambda_1$.

The Laplacian and connected components

- The Laplacian summarizes many significant properties of the graph.
- **Theorem 2:** The geometric multiplicity of the 0 eigenvalue of the Laplacian $\mathbf{L}$ corresponds to the number of connected components in the graph.
- For any eigenvector $\mathbf{e}$ of the eigenvalue 0, the equation $\mathbf{e}^\top \mathbf{L}\mathbf{e} = 0$ holds true by definition.
- Based on the vector identity, this implies that:

$$\sum_{(u,v)\in\mathcal{E}} (\mathbf{e}[u] - \mathbf{e}[v])^2 = 0$$

- This equality indicates that $\mathbf{e}[u] = \mathbf{e}[v]$ for all edges $(u, v) \in \mathcal{E}$, meaning $\mathbf{e}[u]$ is a constant for all nodes u situated in the same connected component.
- In a fully connected graph, the eigenvector for eigenvalue 0 will be a constant vector of ones for all nodes.

- If a graph is composed of K multiple connected components, the node ordering can be arranged so the Laplacian matrix $\mathbf{L}$ is written as a block diagonal matrix composed of blocks $\mathbf{L}_1$ through $\mathbf{L}_K$.
- Each block $\mathbf{L}_k$ is a valid graph Laplacian of a fully connected subgraph, possessing an eigenvalue of 0 with multiplicity 1 and an eigenvector of all ones.
- Because $\mathbf{L}$ is a block diagonal matrix, its spectrum is the union of the spectra of all the $\mathbf{L}_k$ blocks.
- Each block contributes one eigenvector for the eigenvalue 0, and this eigenvector acts as an indicator vector for the nodes in that specific connected component.

Normalized Laplacians

- There are two popular normalized variants of the unnormalized Laplacian.
- The **symmetric normalized Laplacian** is defined as:

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}}$$

- The **random walk Laplacian** is defined as:

$$\mathbf{L}_{\text{RW}} = \mathbf{D}^{-1}\mathbf{L}$$

- These normalized matrices share similar properties with the standard Laplacian, but their algebraic properties differ by small constants because of the normalization.
- Theorem 2 holds exactly for the random walk Laplacian ($\mathbf{L}_{\text{RW}}$).
- For the symmetric normalized Laplacian ($\mathbf{L}_{\text{sym}}$), Theorem 2 holds, but the eigenvectors for the 0 eigenvalue are scaled by $\mathbf{D}^{\frac{1}{2}}$.

The Laplacian and connected components The Laplacian summarizes many important properties of the graph. For example, we have the following theorem:

Theorem 2. *The geometric multiplicity of the 0 eigenvalue of the Laplacian* $\mathbf{L}$ *corresponds to the number of connected components in the graph.*

Proof. This can be seen by noting that for any eigenvector $\mathbf{e}$ of the eigenvalue 0 we have that

$$\mathbf{e}^\top \mathbf{L}\mathbf{e} = 0 \tag{2.29}$$

by the definition of the eigenvalue-eigenvector equation. And, the result in Equation (2.29) implies that

$$\sum_{(u,v)\in\mathcal{E}} (\mathbf{e}[u] - \mathbf{e}[v])^2 = 0. \tag{2.30}$$

The equality above then implies that $\mathbf{e}[u] = \mathbf{e}[v], \forall (u,v) \in \mathcal{E}$, which in turn implies that $\mathbf{e}[u]$ is the same constant for all nodes u that are in the same connected component. Thus, if the graph is fully connected then the eigenvector for the eigenvalue 0 will be a constant vector of ones for all nodes in the graph, and this will be the only eigenvector for eigenvalue 0, since in this case there is only one unique solution to Equation (2.29).

Conversely, if the graph is composed of multiple connected components then we will have that Equation 2.29 holds independently on each of the blocks of the Laplacian corresponding to each connected component. That is, if the graph is composed of K connected components, then there exists

an ordering of the nodes in the graph such that the Laplacian matrix can be written as

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_1 & & & \\ & \mathbf{L}_2 & & \\ & & \ddots & \\ & & & \mathbf{L}_K \end{bmatrix}, \tag{2.31}$$

where each of the $\mathbf{L}_k$ blocks in this matrix is a valid graph Laplacian of a fully connected subgraph of the original graph. Since they are valid Laplacians of fully connected graphs, for each of the $\mathbf{L}_k$ blocks we will have that Equation (2.29) holds and that each of these sub-Laplacians has an eigenvalue of 0 with multiplicity 1 and an eigenvector of all ones (defined only over the nodes in that component). Moreover, since $\mathbf{L}$ is a block diagonal matrix, its spectrum is given by the union of the spectra of all the $\mathbf{L}_k$ blocks, i.e., the eigenvalues of $\mathbf{L}$ are the union of the eigenvalues of the $\mathbf{L}_k$ matrices and the eigenvectors of $\mathbf{L}$ are the union of the eigenvectors of all the $\mathbf{L}_k$ matrices with 0 values filled at the positions of the other blocks. Thus, we can see that each block contributes one eigenvector for eigenvalue 0, and this *eigenvector is an indicator vector for the nodes in that connected component.* □

2.3.2 Graph Cuts and Clustering

- In Theorem 2, we saw that eigenvectors corresponding to the 0 eigenvalue of the Laplacian can be used to assign nodes to clusters based on which connected component they belong to.
- However, this approach only clusters nodes that are already in disconnected components, which is trivial.
- The goal in this section is to take this idea further and show that the Laplacian can be used to give an optimal clustering of nodes *within a fully connected graph*.

Graph cuts

- To motivate the Laplacian spectral clustering approach, we must first define an *optimal* cluster using the notion of a *cut* on a graph.
- Let $\mathcal{A} \subset \mathcal{V}$ denote a subset of the nodes in the graph and let $\bar{\mathcal{A}}$ denote the complement of this set, i.e., $\mathcal{A} \cup \bar{\mathcal{A}} = \mathcal{V}$ and $\mathcal{A} \cap \bar{\mathcal{A}} = \emptyset$.
- Given a partitioning of the graph into K non-overlapping subsets $\mathcal{A}_1, ..., \mathcal{A}_K$, we define the cut value of this partition as the count of how many edges cross the boundary between the partition of nodes:

$$\text{cut}(\mathcal{A}_1, ..., \mathcal{A}_K) = \frac{1}{2} \sum_{k=1}^{K} |(u, v) \in \mathcal{E} : u \in \mathcal{A}_k, v \in \bar{\mathcal{A}}_k|$$

- Minimizing this cut value is one way to define optimal clustering. However, efficient algorithms for this task tend to create trivial clusters consisting of a single node (Stoer and Wagner, 1997).
- Instead of simply minimizing the cut, we seek to minimize it while enforcing that the partitions are reasonably large.
- **Ratio Cut:** One popular method is minimizing the Ratio Cut, which penalizes the solution for choosing small cluster sizes:

$$\text{RatioCut}(\mathcal{A}_1, ..., \mathcal{A}_K) = \frac{1}{2} \sum_{k=1}^{K} \frac{|(u, v) \in \mathcal{E} : u \in \mathcal{A}_k, v \in \bar{\mathcal{A}}_k|}{|\mathcal{A}_k|}$$

- **Normalized Cut (NCut):** Another solution is minimizing the NCut, which enforces that all clusters have a similar number of edges incident to their nodes:

$$\text{NCut}(\mathcal{A}_1, ..., \mathcal{A}_K) = \frac{1}{2} \sum_{k=1}^{K} \frac{|(u, v) \in \mathcal{E} : u \in \mathcal{A}_k, v \in \bar{\mathcal{A}}_k|}{\text{vol}(\mathcal{A}_k)}$$

- *Note:* $\text{vol}(\mathcal{A}) = \sum_{u \in \mathcal{A}} d_u$.

Approximating the RatioCut with the Laplacian spectrum

- We can derive an approach to find a cluster assignment minimizing the RatioCut using the Laplacian spectrum (a similar approach works for NCut).
- For simplicity, consider $K = 2$ (separating nodes into two clusters). The goal is to solve:

$$\min_{\mathcal{A} \subset \mathcal{V}} \text{RatioCut}(\mathcal{A}, \bar{\mathcal{A}})$$

- To rewrite this conveniently, define the vector $\mathbf{a} \in \mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{a}[u] = \begin{cases} \sqrt{\frac{|\bar{\mathcal{A}}|}{|\mathcal{A}|}} & \text{if } u \in \mathcal{A} \\ -\sqrt{\frac{|\mathcal{A}|}{|\bar{\mathcal{A}}|}} & \text{if } u \in \bar{\mathcal{A}} \end{cases}$$

- Combining this vector with the properties of the graph Laplacian, we can express the Ratio Cut in terms of the Laplacian. The derivation is as follows:

$$\mathbf{a}^\top \mathbf{L}\mathbf{a} = \sum_{(u,v) \in \mathcal{E}} (\mathbf{a}[u] - \mathbf{a}[v])^2$$

$$= \sum_{(u,v) \in \mathcal{E}: u \in \mathcal{A}, v \in \bar{\mathcal{A}}} (\mathbf{a}[u] - \mathbf{a}[v])^2$$

$$= \sum_{(u,v) \in \mathcal{E}: u \in \mathcal{A}, v \in \bar{\mathcal{A}}} \left(\sqrt{\frac{|\bar{\mathcal{A}}|}{|\mathcal{A}|}} - \left(-\sqrt{\frac{|\mathcal{A}|}{|\bar{\mathcal{A}}|}} \right) \right)^2$$

$$= \text{cut}(\mathcal{A}, \bar{\mathcal{A}}) \left(\frac{|\mathcal{A}|}{|\bar{\mathcal{A}}|} + \frac{|\bar{\mathcal{A}}|}{|\mathcal{A}|} + 2 \right)$$

$$= \text{cut}(\mathcal{A}, \bar{\mathcal{A}}) \left(\frac{|\mathcal{A}| + |\bar{\mathcal{A}}|}{|\bar{\mathcal{A}}|} + \frac{|\mathcal{A}| + |\bar{\mathcal{A}}|}{|\mathcal{A}|} \right)$$

$$= |\mathcal{V}| \text{RatioCut}(\mathcal{A}, \bar{\mathcal{A}})$$

- Additionally, the vector $\mathbf{a}$ has two important properties:
  1. $\sum_{u \in \mathcal{V}} \mathbf{a}[u] = 0$, or equivalently, $\mathbf{a} \perp \mathbf{1}$ (where $\mathbf{1}$ is the vector of all ones).
  2. $\|\mathbf{a}\|^2 = |\mathcal{V}|$
- Putting this together, the Ratio Cut minimization problem can be rewritten as:

$$\min_{\mathcal{A} \subset \mathcal{V}} \mathbf{a}^\top \mathbf{L} \mathbf{a}$$

Subject to:
$\mathbf{a} \perp \mathbf{1}$
$\|\mathbf{a}\|^2 = |\mathcal{V}|$
$\mathbf{a}$ defined as the discrete vector above.

- Unfortunately, this is an NP-hard problem because optimizing $\mathbf{a}$ restricts us to a discrete set.
- The obvious relaxation is to remove this discreteness condition and simplify the minimization to be over real-valued vectors:

$$\min_{\mathbf{a} \in \mathbb{R}^{|\mathcal{V}|}} \mathbf{a}^\top \mathbf{L} \mathbf{a}$$

Subject to:
$\mathbf{a} \perp \mathbf{1}$
$\|\mathbf{a}\|^2 = |\mathcal{V}|$

- By the **Rayleigh-Ritz Theorem**, the solution to this optimization problem is given by the second-smallest eigenvector of $\mathbf{L}$ (since the smallest eigenvector is equal to $\mathbf{1}$).
- Thus, we approximate the minimization of the RatioCut by setting $\mathbf{a}$ to the second-smallest eigenvector of the Laplacian.
- To turn this real-valued continuous vector back into a discrete set of cluster assignments, we assign nodes based on the sign of $\mathbf{a}[u]$:

$$\begin{cases} u \in \mathcal{A} & \text{if } \mathbf{a}[u] \geq 0 \\ u \in \bar{\mathcal{A}} & \text{if } \mathbf{a}[u] < 0 \end{cases}$$

- **Summary:** The second-smallest eigenvector of the Laplacian is a continuous approximation to the discrete vector that provides an optimal cluster assignment for the RatioCut. An analogous result approximates the NCut value, but relies on the second-smallest eigenvector of the normalized Laplacian, $\mathbf{L}_{\text{RW}}$ (Von Luxburg, 2007).

2.3.3 Generalized spectral clustering

Concept Overview

Building upon the method of partitioning a graph into two clusters using the second-smallest eigenvector of the Laplacian, this approach can be extended to find an arbitrary number of K clusters. This generalization is achieved by examining the K smallest eigenvectors of the graph Laplacian.

Algorithm Steps

The general approach to generalized spectral clustering involves the following steps:

1. **Identify Eigenvectors:** Find the K smallest eigenvectors of the Laplacian matrix $\mathbf{L}$ (excluding the absolute smallest):

$$\mathbf{e}_{|\mathcal{V}|-1}, \mathbf{e}_{|\mathcal{V}|-2}, \ldots, \mathbf{e}_{|\mathcal{V}|-K}$$

2. **Construct Matrix:** Form the matrix $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times (K-1)}$ using the eigenvectors obtained in Step 1 as its columns.
3. **Extract Row Representations:** Represent each node by its corresponding row in the matrix $\mathbf{U}$, such that:

$$\mathbf{z}_u = \mathbf{U}[u] \quad \forall u \in \mathcal{V}$$

4. **Cluster Embeddings:** Run standard K-means clustering on the resulting *embeddings* $\mathbf{z}_u \quad \forall u \in \mathcal{V}$.

Extensions and Theoretical Connections

- **Normalized Laplacian:** Just like in the $K = 2$ case, this approach can be easily adapted to use the normalized Laplacian matrix. Furthermore, the approximation results established for two clusters generalize effectively to cases where $K > 2$ (Von Luxburg, 2007).
- **Principled Approximation:** The core strength of spectral clustering lies in representing graph nodes using the spectrum of the graph Laplacian. This serves as a mathematically principled approximation to an optimal graph clustering solution.
- **Broader Context:** Spectral clustering has deep theoretical connections to other network analysis concepts, particularly **random walks on graphs** and the field of **graph signal processing** (Ortega et al., 2018).

2.4 Towards Learned Representations

09 July 2026 09:55

2.4 Towards Learned Representations

Recap of Traditional Approaches to Graph Learning

Traditional methods for learning over graphs rely on manually extracting features. Key approaches discussed previously include:

- **Graph Statistics and Kernels:** Utilized to extract feature information primarily for classification tasks.
- **Neighborhood Overlap Statistics:** Provide powerful heuristics for relation prediction.
- **Spectral Clustering:** A principled method used to cluster nodes into distinct communities.

Limitations of Traditional Approaches

While useful, traditional methods—particularly those relying on node-level and graph-level statistics—have significant drawbacks:

- **Reliance on Hand-Engineering:** They require the careful, manual design of statistics and measures.
- **Inflexibility:** Hand-engineered features are static; they cannot adapt or improve dynamically through a machine learning process.
- **Inefficiency:** Designing these manual features is often a time-consuming and computationally or manually expensive process.

The Paradigm Shift: Graph Representation Learning

To overcome the limitations of hand-crafted features, modern approaches introduce a powerful alternative: **Graph Representation Learning**.

Core Concept: Instead of manually engineering and extracting features, the objective is to dynamically *learn* representations that inherently encode the structural information and properties of the graph.

Module 4 Representation Learning for graphs, & Node & Graph Embedding

T1-Chapter 1 Representation Learning

16 July 2026 15:29

Chapter 1: Representation Learning

Authors: Liang Zhao, Lingfei Wu, Peng Cui, and Jian Pei

Abstract: This chapter explores what representation learning is and why it is essential. The primary focus is on **deep learning methods**—specifically, those formed by the composition of multiple non-linear transformations to create abstract and useful representations. The chapter summarizes representation learning techniques across various domains, focusing on the unique challenges and models for different data types (images, natural languages, speech signals, and networks).

1.1 Representation Learning: An Introduction

The effectiveness of machine learning techniques heavily relies on two primary pillars: the design of the algorithms themselves and a **good representation (feature set)** of the data.

- **The Problem with Poor Representations:** Ineffective data representations that lack critical information or contain massive amounts of redundant information directly lead to poor algorithmic performance across tasks.
- **The Core Goal:** The objective of representation learning is to extract **sufficient but minimal information** from data.

Feature Engineering: The Traditional Approach

Historically, creating data representations relied on human effort, prior knowledge, and domain expertise—a process known as **feature engineering**. A significant portion of human effort in AI deployment went into designing preprocessing pipelines and data transformations.

- **Examples of Feature Engineering:**
  - *Social Media Text Classification:* Political scientists defining a specific keyword list as features to detect societal events.
  - *Speech Transcription:* Extracting features from raw sound waves using operations like Fourier transformations.

Drawbacks of Feature Engineering

1. **Intensive Manual Labor:** It requires tight, extensive, and continuous collaboration between model developers and domain experts.
2. **Incomplete and Biased Extraction:** The capacity and discriminative power of the features are strictly capped by the limited knowledge of the domain experts.
3. **Open Questions in Complex Domains:** In complex fields like early cancer prediction, human beings have limited knowledge, meaning experts often do not know which features need to be extracted in the first place.

To bypass these limitations, a highly desired goal in ML and AI is to make learning algorithms **less dependent on manual feature engineering**, allowing novel applications to be constructed faster and addressed more effectively.

The Evolution of Representation Learning Techniques

Representation learning has evolved significantly from traditional, older techniques to modern deep learning methods:

1. "Shallow" Models (Traditional)

These methods aim to learn transformations of data that make it easier to extract useful information when building predictors or classifiers. Notable examples include:

- **Principal Component Analysis (PCA)** (Wold et al., 1987)
- **Gaussian Markov Random Field (GMRF)** (Rue and Held, 2005)
- **Locality Preserving Projections (LPP)** (He and Niyogi, 2004)

2. Deep Learning-Based Models (Modern)

Formed by the composition of **multiple non-linear transformations**, these methods yield highly abstract and ultimately more useful representations.

Categorization of Deep Learning-Based Representation Learning

Deep representation learning can be broadly categorized into the following types:

- **Supervised Learning:** Requires a large amount of labeled data to train deep models. For well-trained networks, the output *before the last fully-connected layer* is typically utilized as the final representation of the input data.
- **Unsupervised Learning (including Self-Supervised Learning):** Facilitates the analysis of input data without corresponding labels. It aims to learn the underlying inherent structure or distribution of the data. It often utilizes **pre-tasks** to extract supervision information from large amounts of unlabeled data, training the network to extract meaningful representations for downstream tasks.
- **Transfer Learning:** Methods that utilize any available knowledge resource (e.g., data, models, labels) to increase model learning and generalization on a target task. It encompasses scenarios such as:
  - Multi-task learning (MTL)
  - Model adaptation
  - Knowledge transfer
  - Co-variance shift
- **Other Important Methods:** Reinforcement learning, few-shot learning, and disentangled representation learning.

Defining and Evaluating a "Good" Representation

According to Bengio (2008), representation learning is fundamentally about learning the underlying features of the data that make it easier to extract useful information when building classifiers or predictors. Therefore, evaluating a representation is intimately tied to its performance on **downstream tasks**.

Task Type	Definition of a "Good" Representation
Data Generation Tasks *(Generative Models)*	One that accurately captures the **posterior distribution** of the underlying explanatory factors for the observed input.
Prediction Tasks	One that captures the **minimal but sufficient** information of input data required to correctly predict the target label.

General Properties of Good Representations

Beyond task-specific performance, ideal representations typically exhibit several core qualities:

- **Smoothness**
- **Linearity**
- Capturing **multiple explanatory and causal factors**
- Holding **shared factors** across different tasks
- Maintaining **simple factor dependencies**

1.2 Representation Learning in Different Areas

This section explores the development of representation learning across four core representative domains:

1. **Image Processing**
2. **Speech Recognition**
3. **Natural Language Processing (NLP)**
4. **Network Analysis**

Core Research Questions

Research within each area is fundamentally driven by the following inquiries:

- **Quality & Computation:** What characteristics make one representation superior to another, and how should it be computed?
- **Importance:** Why is representation learning vital for that specific domain?
- **Objectives:** What constitute the appropriate objectives for learning high-quality representations?

Methodological Categories

Developments are evaluated through the lens of three main methodologies:

- **Supervised Representation Learning**
- **Unsupervised Representation Learning**
- **Transfer Learning**

1.2.1 Representation Learning for Image Processing

Primary Goal: To bridge the **semantic gap** between raw pixel data and the actual high-level semantics of visual media (e.g., photographs, medical scans, document images, and video streams).

Key Real-World Applications

Successful representation learning underpins several practical applications:

- Image search & target detection
- Facial recognition
- Medical image analysis
- Photo manipulation

The Paradigm Shift: Hand-Crafted Features vs. Deep Learning

- **Traditional Hand-Crafted Feature Engineering:**
  - Features were manually extracted by human experts leveraging domain prior knowledge.
  - *Examples:* **Huang et al. (2000)** extracted stroke-based structural features for handwritten character recognition; **Rui (2005)** applied morphology methods to enhance local features alongside PCA for character feature extraction.
  - *Limitations:* Manual feature extraction is exceptionally cumbersome and impractical due to the extremely high dimensionality of visual feature vectors. Prediction performance is heavily bottlenecked by human prior knowledge.
- **Modern Deep Learning Approach:**
  - Learns representations automatically in an **end-to-end fashion** from raw data.
  - Effectively extracts hidden, complex, and highly meaningful patterns from high-dimensional visual data, significantly outperforming hand-crafted methods when provided with sufficient training data quality and volume.

1. Supervised Representation Learning for Image Processing

Supervised learning algorithms rely on annotated datasets to map inputs to precise targets.

- **Common Architectures:** Convolutional Neural Networks (CNNs / ConvNets) and Deep Belief Networks (DBNs) are widely deployed.
- **Milestones & Breakthroughs:**
  - **Hinton et al. (2006):** One of the earliest deep supervised works focused on the MNIST digit classification task, outperforming state-of-the-art Support Vector Machines (SVMs).
  - **ConvNet Advantages:** The immense success of deep ConvNets is attributed to their intrinsic architectural properties: **shift invariance**, **weight sharing**, and **local pattern capturing**.
- **Evolution of Networks & Data:**
  - Increasingly sophisticated network architectures have been developed to scale model capacity, including **AlexNet** (2012), **VGG** (2014b), **GoogLeNet** (2015), **ResNet** (2016a), and **DenseNet** (2017a).
  - Massive datasets such as **ImageNet** and **OpenImage** have empowered these deeper network configurations to consistently surpass prior baselines across computer vision tasks.

2. Unsupervised Representation Learning for Image Processing

Gathering human annotations for massive datasets is highly time-consuming and expensive (e.g., ImageNet requires human classification labels for roughly 1.3 million images across 1,000 distinct classes). Unsupervised methods eliminate this barrier by extraction features from unlabeled visual media.

- **Pretext Tasks:** Models are trained to solve predefined auxiliary tasks where **pseudo labels** are automatically generated from data attributes without human intervention.
  - *Examples:* Colorizing gray-scale images (**Zhang et al., 2016d**) and image in-painting (**Pathak et al., 2016**).
- **Hierarchical Feature Capture:**
  - **Shallower blocks** of deep networks trained on pretext tasks automatically focus on low-level, general features (e.g., edges, corners, and textures).
  - **Deeper blocks** focus on complex, high-level, task-specific features (e.g., whole objects, abstract scenes, and constituent parts).
- **Downstream Utility:** Once unsupervised pre-training concludes, these general visual kernels can be transferred to downstream target tasks—drastically improving performance and preventing overfitting, particularly when labeled data is scarce.

3. Transfer Learning for Image Processing

In real-world settings, it is common to encounter scenarios where labeled testing and training data do not share the same feature space or underlying distribution, or labeled data is simply too costly to obtain.

Concept: Transfer learning mimics the human visual system by leveraging extensive prior knowledge acquired from an external domain (**source domain**) to improve performance on a new task within a different domain (**target domain**).

Typically, a transfer task involves a single target domain alongside single or multiple source domains. The methodologies broadly divide into two categories:

- **Feature Representation Knowledge Transfer:** Maps the target domain directly into the source domain using extracted features. This alignment substantially diminishes the cross-domain data divergence, raising performance metrics in the target domain.
- **Classifier-Based Knowledge Transfer:** Utilizes pre-trained source domain models as foundational prior knowledge to help train the target model. Rather than altering instance representations to reduce cross-domain dissimilarity, it aims to learn a brand-new model that directly minimizes generalization errors in the target domain using available data from both domains.

4. Other Representation Learning Frameworks

Beyond traditional supervised, unsupervised, and transfer structures, image processing successfully adopts other frameworks:

- **Semi-Supervised Learning:** Combines small quantities of labeled data with large amounts of unlabeled data.
- **Reinforcement Learning (RL):** Formalizes the learning process as a structural sequence of actions dictated by a policy network.
  - *Examples:* Explored widely in tasks like image captioning (**Liu et al., 2018a**; **Ren et al., 2017**) and automated image editing (**Kosugi and Yamasaki, 2020**).

1.2.2 Representation Learning for Speech Recognition

Introduction and Motivation

- **Ubiquity of Speech Interfaces:** Speech systems (such as Siri, Cortana, and Google Voice Search) are widely integrated into daily life and real-life applications for millions of users.
- **Core Research Goals:** Driven by the desire to enable seamless human-machine verbal interaction, researchers focus on enabling machines to:
  - Understand human speech
  - Identify speakers
  - Detect human emotion

- **Key Research Areas:** Research has spanned more than sixty years across distinct areas, primarily:
  - **ASR:** Automatic Speech Recognition
  - **SR:** Speaker Recognition
  - **SER:** Speaker Emotion Recognition

Traditional Approaches vs. Representation Learning

- **Traditional Approach:** Historically relied on designing **hand-crafted acoustic features** separate from the design of prediction and classification models.
- **Drawbacks of Traditional Features:**
  - **Cumbersome:** Feature engineering requires significant human domain knowledge.
  - **Sub-optimal:** Hand-crafted features might not be perfectly optimized for the specific speech recognition task at hand.
- **The Shift to Representation Learning:** Modern trends leverage machine learning to automatically extract intermediate representations from raw input signals, resulting in better task fit and enhanced performance.
- **Speech Data vs. Image Data:**
  - *Images* can be analyzed as a whole or in localized patches.
  - *Speech* must be formatted **sequentially** to accurately capture temporal dependencies and sequential patterns.

Supervised Representation Learning

Supervised methods leverage labeled datasets to learn robust feature representations.

- **Key Architectures:** Restricted Boltzmann Machines (**RBMs**) and Deep Belief Networks (**DBNs**) are widely utilized to extract features from speech signals across ASR, SR, and SER.
- **Milestone Case Study (2012):** Microsoft released a new version of its **MAVIS** (Microsoft Audio Video Indexing Service) speech system based on context-dependent deep neural networks.
  - **Impact:** Reduced the Word Error Rate (**WER**) by approximately **30%** (dropping from 27.4% to 18.5% on the RT03S benchmark) compared to traditional Gaussian mixture models.
- **Convolutional Neural Networks (CNNs):** Widely utilized for feature learning from speech signals in ASR and SER.
- **Temporal Modeling:** Combining **LSTMs** or **GRUs** with CNNs allows the system to learn more useful features by capturing both **local** and **long-term temporal dependencies**.

Unsupervised Representation Learning

Unsupervised learning exploits the practically unlimited availability of unlabelled speech corpora to discover high-quality intermediate feature representations.

- **Variational Auto-encoders (VAEs):** Commonly used in ASR and SR. They jointly learn a generative model and an inference model to capture latent representations from observed speech data.
    - **Hierarchical VAEs:** Used to capture interpretable and disentangled speech representations completely without supervision.
- **Denoising Autoencoders (DAEs):** Highly promising for finding robust speech representations in **noisy speech recognition** environments.
- **Adversarial Learning (AL):** Generative Adversarial Nets (**GANs**) employ a generator and a discriminator trained iteratively. The generator attempts to produce realistic data to obfuscate the discriminator, while the discriminator strives to deobfuscate it, resulting in highly discriminative and robust features.
- **Adversarial Autoencoders (AAEs):** Increasingly popular for modeling speech problems across ASR, SR, and SER.

Transfer Learning

Transfer Learning (TL) encompasses approaches like Multi-Task Learning (MTL), model adaptation, knowledge transfer, and managing covariance shift.

- **Domain Adaptation:** Addresses the frequent mismatch between the probability distributions of source data and target domain data to build robust, real-world speech systems. Deep Neural Networks (DNNs) are trained to explicitly minimize the distribution divergence between source and target domains.
- **Multi-Task Learning (MTL):** Successfully improves speech recognition performance without requiring explicit contextual speech data. It utilizes auxiliary tasks to provide complementary information about the acoustic environment, which lowers the Word Error Rate (WER).
    - **Common Auxiliary Tasks:** Gender recognition, speaker adaptation, and speech enhancement.

Other Representation Learning Techniques

Beyond the primary three categories, other paradigms are actively explored:

- **Semi-Supervised Learning:** Primarily used in ASR to circumvent the lack of sufficient training data. This is achieved by:
  - Creating specialized feature front-ends.
  - Developing multilingual acoustic representations.
  - Extracting intermediate representations from large, unpaired datasets.
- **Reinforcement Learning (RL):** Gaining traction for optimization and modeling across various complex speech problems, including:
  - Dialog modeling and optimization.
  - Speech recognition.
  - Emotion recognition.

1.2.3 Representation Learning for Natural Language Processing

Representation learning in Natural Language Processing (NLP) is used to map linguistic inputs into a continuous semantic space. A notable example is Google's image search, which exploits massive amounts of data to map both images and text queries into the same shared space.

Two Main Types of Applications in NLP

- **Pre-trained Semantic Representations:** These involve training representations (such as word embeddings) on a pre-training task using a language modeling objective, or explicitly designing them by human experts. They are then transferred to a target model to serve as inputs for downstream NLP tasks.
- **End-to-End Hidden State Representations:** Here, the semantic representation lies directly within the hidden states of a deep learning model. The objective is optimized in an end-to-end fashion to directly improve performance on specific target tasks (e.g., sentiment classification, natural language inference, and relation extraction).

Advantages Over Conventional NLP

Conventional NLP relies heavily on manual **feature engineering**, which demands careful design and significant domain expertise. Deep learning-based representation learning avoids this by offering three key advantages:

- **Unified Multi-Level Space:** NLP deals with language entries at multiple levels (characters, words, phrases, sentences, paragraphs, and documents). Representation learning unifies these entries into a single semantic space, modeling complex semantic dependencies between them.
- **Multi-Task Efficiency:** Multiple tasks (such as word segmentation, named entity recognition, relation extraction, co-reference linking, and machine translation) can be performed on the exact same input. A unified representation space makes multi-task learning more robust and efficient.
- **Cross-Domain and Cross-Lingual Capabilities:** Natural language data is gathered from diverse domains (news, scientific literature, advertisements, social media) and various languages (English, Chinese, Spanish, Japanese, etc.). Representation learning automatically builds features from large-scale data and establishes bridges across different domains and languages.

Supervised Representation Learning for NLP

Supervised representation learning in NLP has transitioned through a clear evolutionary path: **Distributed Representations** $\rightarrow$ **CNN Models** $\rightarrow$ **RNN Models**.

1. Early Stage & Distributed Representations

- Pioneered in the context of statistical neural network language models (e.g., Bengio, 2008).
- Focused on learning a **distributed representation** for individual words, commonly referred to as a **word embedding**.

2. Convolutional Neural Networks (CNNs)

- Adopted to extract higher-level feature functions from combinations of words or n-grams.
- CNNs excel at capturing salient n-gram features from an input sentence to create an informative latent semantic representation for downstream tasks.

3. Recurrent Neural Networks (RNNs)

- Introduced recurrence to the hidden layers, allowing the model to process sequential information effectively.
- **Mechanism:** An RNN performs the same computation over each token of a sequence, where each step is dependent on the results of the previous computations. Tokens are fed one by one into a recurrent unit to produce a final fixed-size vector.
- **Memory Feature:** RNNs possess a form of "memory" over the elements processed so far.
- **Performance Metrics:** RNNs achieved state-of-the-art performance by drastically reducing the Word Error Rate (WER) in speech recognition and improving **perplexity** in language modeling.

$$\text{Perplexity} = e^{\text{average negative log-likelihood of predicting the correct next word}}$$

- **Applications:** Naturally suited for sequence-heavy tasks like language modeling, machine translation, and image captioning.

Unsupervised Representation Learning for NLP

Unsupervised representation learning (which includes self-supervised learning) leverages the abundant knowledge and patterns intrinsically present in plain text without requiring human-annotated labels.

Key Characteristics & Mechanics

- **Baseline Models:** Raw text words are mapped to initial embeddings using highly successful architectures like **word2vec**, **GloVe**, and **BERT** before being processed by neural networks.
- **Intrinsic Objectives:** Because human labels are missing, the training objectives are generated directly from the structure of the data itself. A prime example is language modeling, which builds a probability distribution over word sequences.
- **The Distributional Hypothesis:** Unsupervised NLP models rely on this hypothesis, using language modeling objectives to construct hidden representations that intrinsically encode word semantics.
- **Auto-encoders (AE):** A typical unsupervised architecture consisting of a **reduction (encoding) phase** and a **reconstruction (decoding) phase**.
    - *Extension:* **Recursive auto-encoders** (which generalize recurrent networks using Variational Auto-encoders, or VAEs) have been successfully applied to full-sentence paraphrase detection, nearly doubling the task's F1 score.

Transfer Learning for NLP

Transfer learning has driven major advancements across a wide range of state-of-the-art NLP applications, primarily through domain adaptation.

Sequential Transfer Learning Stages

1. **Pre-training Phase:** The model learns general language representations on a large source task or domain.
2. **Adaptation Phase:** The accumulated knowledge is transferred and applied directly to a specific target task or domain.

Categorization of Transfer Learning Approaches

Approach	Description & Focus
Model-centric	Focuses on modifying the model architecture, augmenting the feature space, adjusting model parameters, or altering the loss function.
Data-centric	Focuses on data manipulation and formatting. Commonly utilizes **pseudo-labeling (bootstrapping)** where a small number of classes are shared between source and target datasets.
Hybrid	Combines elements of both data-centric and model-centric methods.

Multi-Task Learning

Jointly training a model on different tasks results in a much better overall representation of text.

- **The SENNA System:** A landmark example that shares representations across multiple core tasks: language modeling, part-of-speech (POS) tagging, chunking, named entity recognition (NER), semantic role labeling, and syntactic parsing. It proves to be simpler and much faster than traditional, isolated predictors.
- **Multimodal Integration:** These textual representation systems can also be combined with image representation models to create shared multi-modal associations between text and graphics.

Other Representation Learning for NLP

As NLP tasks become more specialized and fine-grained, systems encounter data bottlenecks.

- **The Annotation Challenge:** Complex or fine-grained tasks require deep domain-expert knowledge to annotate training instances, which drastically increases data-labeling costs.
- **The Solution:** Development of efficient architectures capable of learning from very few labeled instances, known as **one-shot** or **few-shot learning**.
- **Applications in NLP:** Initially derived from computer vision paradigms, few-shot techniques are now heavily researched in NLP for scenarios such as:
  - *Few-shot relation extraction:* Identifying relationships between entities when each relation type contains only a handful of labeled examples.
  - *Low-resource machine translation:* Training translation systems where the available parallel text corpus between two languages is highly limited.

1.2.4 Representation Learning for Networks

Network data has become ubiquitous across a wide range of real-world applications. It serves as a powerful and flexible mathematical formulation where **vertices (nodes)** and their **relationships (edges)** jointly characterize complex information.

Networks are so versatile that other popular data types can be viewed as special cases:

- **Images:** Can be considered as grids of nodes with RGB attributes.
- **Texts:** Can be organized into sequential-, tree-, or graph-structured information.

Real-World Applications & Importance

Analyzing information networks is crucial across multiple disciplines, though it is highly challenging due to the intrinsic complexity and massive scale (ranging from hundreds to billions of vertices) of real-world data.

- **Cyber-Networks (e.g., Social & Citation Networks):** Classifying users into meaningful social groups to facilitate user search, targeted advertising, and recommendation systems.
- **Telecommunication & Communication Networks:** Detecting community structures to better model and understand rumor-spreading processes.
- **Physical & Biological Networks (e.g., Transportation & Protein Networks):** Inferring interactions between proteins to discover new treatments for diseases.

Limitations of Traditional Feature Engineering

Historically, network analysis relied heavily on hand-crafted, predefined features designed at various structural levels:

Feature Level	Examples
Graph Level	Diameter, average path length, clustering coefficient
Node Level	Node degree, centrality
Subgraph Level	Frequent subgraphs, graph motifs

Core Drawbacks of Traditional Approaches:

- **Incomplete Pattern Coverage:** Hand-crafted features discard critical structural patterns that fall outside their predefined definitions.
- **Inability to Capture Complex Phenomena:** Real-world network interactions are highly complicated and require sophisticated, unknown combinations of features that existing metrics cannot characterize.
- **High Computational Complexity:** Traditional graph feature engineering often involves super-linear or exponential complexity. For example, classical community detection utilizing spectral decomposition requires **at least quadratic time complexity** relative to the number of vertices, making it intractable for large-scale networks.

The Emergence of Network Representation Learning (NRL)

To overcome the bottlenecks of manual feature engineering, **Network Representation Learning (NRL)** aims to automatically learn latent, low-dimensional vector representations of network vertices.

Core Objective: Map vertices into a compact, low-dimensional space while preserving the network's underlying topology structure, vertex content, and auxiliary side information.

Historical Evolution of NRL:

- **Early 2000s (Graph Embedding & Dimensionality Reduction):** Initial methods operated on independent and identically distributed (i.i.d.) data. They calculated pairwise similarities to construct an affinity graph (e.g., k-nearest neighbor graphs) and embedded it into a lower-dimensional space. However, these methods suffered from at least quadratic time complexity relative to the vertex count.
- **Post-2008 (Scalable & Direct Network Embedding):** Modern research shifted toward developing effective, scalable representation learning techniques designed directly for complex information networks. These methods preserve both **structure proximity** and **attribute affinity** efficiently.

Downstream Task Enablement:

Once low-dimensional vertex representations are learned, complex network analytical tasks can be seamlessly and efficiently executed using conventional vector-based machine learning algorithms. These tasks include:

- Node classification
- Link prediction
- Clustering
- Network synthesis

1.3 Summary

Core Concept of Representation Learning

- **Definition:** Representation learning focuses on automatically discovering the underlying representations of data to make it easier to extract useful and discriminative information when building classifiers or other predictors.
- **Significance:** It is a highly active and critical field that heavily influences the overall effectiveness of modern machine learning techniques.

The Role of Deep Learning

- Deep learning algorithms have increasingly become the standard approach for representation learning across many domains.
- **Key Advantage:** They allow high-quality representations to be learned in an **efficient and automatic** manner.
- **Data Handling:** They excel at processing large scales of **complex and high-dimensional data**.

Evaluating and Defining a "Good" Representation

- **Evaluation Metric:** The quality of a learned representation is fundamentally evaluated by its performance on subsequent **downstream tasks**.
- **Desirable Properties:** Good representations typically exhibit characteristics such as:
  - **Smoothness**
  - **Linearity**
  - **Disentanglement**
  - **Causal Capture:** The ability to capture multiple underlying explanatory and causal factors.

Application Domains and Learning Paradigms

Representation learning techniques address unique challenges and models across diverse data types, categorized into distinct learning paradigms:

- **Core Target Domains:**
  - Images
  - Natural Language
  - Speech Signals
  - Networks (including its relationships to images, texts, and speech)
- **Algorithmic Categories:**
  - Supervised Learning
  - Unsupervised Learning
  - Transfer Learning
  - Disentangled Representation Learning
  - Reinforcement Learning

T1-Chapter 2 Graph Representation Learning

16 July 2026 16:12

Chapter 2: Graph Representation Learning

Authors: Peng Cui, Lingfei Wu, Jian Pei, Liang Zhao, and Xiao Wang

Abstract: Graph representation learning aims to assign nodes in a graph to low-dimensional representations while effectively preserving the graph structures. This field encompasses three primary categories of methods: traditional graph representation learning, modern graph representation learning, and graph neural networks.

2.1 Graph Representation Learning: An Introduction

Many complex real-world systems take the form of graphs, including:

- **Social networks**
- **Biological networks**
- **Information networks**

Because graph data is inherently sophisticated, finding an effective, concise representation is a critical first step. An optimized representation ensures that advanced analytic tasks—such as pattern discovery, analysis, and prediction—can be conducted efficiently in both time and space.

Traditional Graph Representation & Its Challenges

Traditionally, a graph is represented as:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E})$$

Where:

- $\mathcal{V}$ is the node set.
- $\mathcal{E}$ is the edge set.

For large-scale graphs containing billions of nodes, this traditional structural representation poses three severe challenges:

1. **High Computational Complexity**
    - The relationships encoded by the edge set $\mathcal{E}$ require graph processing algorithms to use iterative or combinatorial optimization steps.
    - *Example:* Computing node distance via shortest or average path lengths requires enumerating many possible paths, creating an exponential combinatorial problem that does not scale well to real-world networks.
2. **Low Parallelizability**
    - Parallel and distributed computing is required for large-scale data, but traditional graphs make algorithm design highly difficult because nodes are explicitly coupled together by $\mathcal{E}$.
    - Distributing different nodes across distinct shards or servers creates a severe bottleneck, causing high communication costs among servers and limiting the parallel speed-up ratio.
3. **Inapplicability of Machine Learning Methods**
    - Off-the-shelf machine learning and deep learning tools typically assume that data samples can be represented as independent vectors in a vector space.
    - Conversely, graph nodes are highly dependent on one another based on $\mathcal{E}$.
    - While a node can be represented by its row vector in an adjacency matrix, the extremely high dimensionality of large graphs makes downstream processing and analysis mathematically difficult.

The Paradigm Shift: Graph Representation Learning

To overcome these limitations, graph representation learning focuses on learning **dense, continuous, low-dimensional vector representations** for individual nodes.

- **Mechanism:** Noise and redundant information are filtered out, while the intrinsic structural properties of the graph are preserved.
- **Vector Space Properties:** Topological relationships (originally represented by explicit edges or high-order network measures) are mapped to spatial distances in the learned embedding space. A node's unique structural features are completely encoded directly inside its vector.

Core Goals of Representation Learning

A well-designed representation space must fulfill two core objectives:

- **Graph Reconstruction:** The original graph structural layout must be reconstructible from the vector space. If an edge or relationship exists between two nodes, the spatial distance between their respective low-dimensional vectors should be relatively small.
- **Graph Inference:** The representation space must effectively support learning predictive patterns. Optimizing purely for graph reconstruction is insufficient for advanced inference tasks.

Downstream Tasks & Applications

Once the low-dimensional node representations are obtained, they can be utilized directly to solve key downstream machine learning tasks:

- **Node Classification:** Predicting unlabeled node properties/categories.
- **Node Clustering:** Grouping similar nodes together based on topological behavior.
- **Graph Visualization:** Projecting the graph structure into interpretable layouts.
- **Link Prediction:** Forecasting unseen, missing, or future relationships between nodes.

Taxonomy of Methods

Graph representation learning approaches are systematically classified into three main categories:

1. **Traditional Graph Embedding**
2. **Modern Graph Embedding**
3. **Graph Neural Networks (GNNs)**

2.2 Traditional Graph Embedding

16 July 2026 16:15

2.2 Traditional Graph Embedding

Overview & Core Goals

- **Origin**: Traditional graph embedding methods were originally studied as **dimensionality reduction techniques**.
- **Graph Construction**: Typically constructed from a feature-represented dataset (such as an image dataset).
- **Two Main Goals of Graph Embedding**:
  1. Reconstructing original graph structures.
  2. Supporting graph inference.
- **Primary Focus**: The objective functions of traditional methods mainly target the goal of **graph reconstruction**.

Key Traditional Graph Embedding Methods

1. Isomap *(Tenenbaum et al., 2000)*

- **Mechanism**:
  - Constructs a neighborhood graph G using connectivity algorithms such as K-nearest neighbors (KNN).
  - Computes the shortest path between different data entries based on G.
  - Generates a matrix of graph distances for all N data entries in the dataset.
  - Applies classical multidimensional scaling (MDS) to the matrix to obtain low-dimensional coordinate vectors.
- **Property**: The learned representations approximately preserve the geodesic distances of the entry pairs in the low-dimensional space.
- **Limitation**: Suffers from high computational complexity due to computing pairwise shortest paths.

2. Locally Linear Embedding (LLE) *(Roweis and Saul, 2000)*

- **Objective**: Proposed to eliminate the need to estimate pairwise distances between widely separated entries.
- **Assumption**: Assumes that each entry and its neighbors lie on or close to a locally linear patch of a manifold.
- **Mechanism**: Characterizes local geometry by reconstructing each entry from its neighbors. In the low-dimensional space, it constructs a neighborhood-preserving mapping based on this locally linear reconstruction.

3. Laplacian Eigenmaps (LE) *(Belkin and Niyogi, 2002)*

- **Mechanism**:
  - Begins by constructing a graph using ε-neighborhoods or K-nearest neighbors.
  - Utilizes a heat kernel *(Berline et al., 2003)* to determine the edge weight between two nodes.
  - Obtains low-dimensional node representations based on **Laplacian matrix regularization**.

4. Locality Preserving Projection (LPP) *(Berline et al., 2003)*

- **Core Concept**: Developed as a **linear approximation** of the nonlinear Laplacian Eigenmaps (LE) method.

Traditional vs. Modern Graph Embedding

Feature Component	Traditional Graph Embedding	Modern Graph Embedding
Data Source Type	Mostly works on graphs constructed from feature-represented datasets (e.g., images).	Works on naturally formed networks (e.g., social, biological, and e-commerce networks).
Proximity Definition	Proximity among nodes is well-defined by edge weights in the original feature space.	Proximities are not explicitly or directly defined; they depend on specific analytical tasks and application scenarios.

Edge Interpretation	Direct indicator of node proximity.	An edge implies a relationship but does not indicate specific proximity magnitude (and a lack of an edge does *not* mean proximity is zero).
Information Scope	Focuses on basic structural/feature relationships.	Incorporates rich information including network structures, properties, side information, and advanced information.
Primary Objective	Targeted almost exclusively toward **graph reconstruction**.	Targets **both** graph reconstruction and **network inference**, with recent progress focusing heavily on inference.

Key Takeaway: Traditional graph embedding can be regarded as a special case of modern graph embedding. While traditional methods focus primarily on reconstructing the geometric graph structure from features, modern approaches adapt to complex, naturally occurring networks to prioritize downstream network inference.

2.3 Modern Graph Embedding

16 July 2026 16:18

2.3 Modern Graph Embedding

Modern graph embedding techniques consider richer, multi-faceted graph information to effectively support network inference. Depending on the type of information preserved during graph representation learning, existing methods are classified into three primary categories:

1. **Graph structures and properties preserving graph embedding**
2. **Graph representation learning with side information**
3. **Advanced information preserving graph representation learning**

Technologically, different models are adopted to incorporate these varying information types or address specific goals. Commonly utilized models include:

- Matrix factorization
- Random walks
- Deep neural networks (and their variations)

2.3.1 Structure-Property Preserving Graph Representation Learning

Graph structures and properties are two critical elements that significantly impact graph inference. Consequently, a fundamental requirement of representation learning is to preserve graph structures and capture graph properties appropriately:

- **Graph Structures:** Include both first-order (local/immediate) structures and higher-order structures (e.g., second-order structures and community structures).
- **Graph Properties:** Vary depending on the type of graph. For instance, directed graphs exhibit **asymmetric transitivity**, while signed graphs must satisfy the principles of **structural balance theory**.

2.3.1.1 Structure Preserving Graph Representation Learning

Graph structures present themselves at different granularities. The most commonly exploited structures in representation learning are neighborhood structures, high-order node proximity, and graph communities.

- **Neighborhood Structure Challenge:** Defining the exact neighborhood structure of a graph is a primary challenge.
  - **DeepWalk (Perozzi et al., 2014):** Discovered that node distributions in short random walks mirror word distributions in natural language. It utilizes random walks to capture neighborhood structures, using Skip-Gram to maximize the probability of neighbors appearing together in a walk sequence.
  - **Node2vec:** Introduces a flexible notion of a node's neighborhood by designing a second-order random walk strategy. This strategy smoothly interpolates between Breadth-First Search (BFS) and Depth-First Search (DFS) to sample neighborhood nodes.

- **LINE (Tang et al., 2015b):** Designed for large-scale network embedding. It preserves first-order and second-order proximities:
  - *First-order proximity:* The observed pairwise proximity directly between two nodes.
  - *Second-order proximity:* Determined by the similarity of "contexts" (shared neighbors) between two nodes.
  - *Limitation:* It is based on a shallow model, meaning its representation ability is fundamentally limited.
- **SDNE (Wang et al., 2016):** A deep network embedding model. It uses a deep auto-encoder architecture with multiple non-linear layers to preserve second-order proximity. To preserve first-order proximity, it adopts the concept of Laplacian eigenmaps (Belkin and Niyogi, 2002).

- **Community Structure Preservation:**
  - **M-NMF (Wang et al., 2017g):** A modularized nonnegative matrix factorization model. It aims to preserve both microscopic structures (first and second-order proximities) and mesoscopic community structures (Girvan and Newman, 2002).
    - Uses the NMF model (Févotte and Idier, 2011) for microscopic structure preservation.
    - Uses modularity maximization (Newman, 2006a) to detect communities.
    - Introduces an auxiliary community representation matrix to bridge node representations with the overall community structure, constraining node representations by both structural granularities.

Summary: Network embedding methods strive to map local, high-order, and community structures into a latent, low-dimensional space using both linear and non-linear models.

2.3.1.2 Property Preserving Graph Representation Learning

Property-preserving methods focus primarily on graph transitivity in general networks and structural balance in signed networks.

- **Graph Transitivity vs. Non-Transitivity:**
  - In a metric space, transitivity naturally satisfies the triangle inequality. However, real-world networks often exhibit **non-transitivity**.
  - *Non-transitivity:* For nodes v_1, v_2, v_3, if (v_1, v_2) and (v_2, v_3) are similar, (v_1, v_3) may be highly dissimilar (e.g., a student is connected to classmates and family, but classmates and family are highly dissimilar).
  - **Ou et al. (2015):** Aimed to preserve non-transitivity via latent similarity components. They learn multiple node embeddings and compare nodes across these multiple similarities. If two nodes have high semantic similarity, at least one of their structural similarities is large; otherwise, all are small.

- **Asymmetric Transitivity:**
  - In directed graphs, if a directed edge exists from node i to j ($i \rightarrow j$) and from j to v ($j \rightarrow v$), there is a high likelihood of a directed edge from $i \rightarrow v$, but *not* from $v \rightarrow i$.
  - **HOPE (Ou et al., 2016):** Summarizes four measurements of high-order proximity under asymmetric transitivity into a general formulation. It utilizes a generalized SVD problem (Paige and Saunders, 1981) to factorize high-order proximity, significantly reducing time complexity and enabling scalability for large-scale networks.
- **Structural Balance Theory (Cartwright and Harary, 1956; Cygan et al., 2012):**
  - Applicable to signed graphs containing positive and negative edges.
  - Based on the social intuition that users should have "friends" (positive edges) placed closer in the embedding space than "foes" (negative edges).
  - **SiNE (Wang et al., 2017f):** Utilizes structural balance theory within a deep learning model consisting of two deep graphs with non-linear functions.

Key Challenge: The primary difficulty lies in addressing the disparity and heterogeneity between the original high-dimensional graph space and the low-dimensional embedding vector space.

2.3.2 Graph Representation Learning with Side Information

Beyond structural topology, side information offers crucial context for graph representation learning. This can be categorized into **node content** and **types of nodes and edges**.

- **Graph Representation Learning with Node Content:**
  - Nodes are often accompanied by rich labels, attributes, or semantic text descriptions.
  - **MMDW (Tu et al., 2016):** A semi-supervised graph embedding algorithm. It is based on DeepWalk-derived matrix factorization and incorporates label information using Support Vector Machines (SVM; Hearst et al., 1998) to discover an optimal classifying boundary.
  - **TADW (Yang et al., 2015b):** Text-Associated DeepWalk. It incorporates rich text features into low-dimensional representation learning.
  - **Pan et al. (2016):** Proposed a coupled deep model that fuses graph structures, node attributes, and node labels simultaneously.

- **Heterogeneous Graph Representation Learning:**
  - Heterogeneous graphs consist of multiple, distinct types of nodes and links.
  - **Jacob et al. (2014):** Proposed a heterogeneous social graph representation learning algorithm to project all node types into a single, common vector space.
  - **Chang et al. (2015):** Developed a deep graph representation learning framework for heterogeneous graphs (e.g., images and text). It leverages a Convolutional Neural Network (CNN) for image data and fully connected layers for text.
  - **Huang and Mamoulis (2017):** Formulated a meta-path similarity-preserving method. Meta-paths (Sun et al., 2011)—defined as sequences of object types mapped with edge types—are used to accurately model specific complex relationships.

2.3.3 Advanced Information Preserving Graph Representation Learning

Unlike general side information, "advanced information" refers to task-specific supervised or pseudo-supervised information. Advanced information-preserving network embedding usually consists of two stages:

1. **Preserving graph structure** to learn robust representations of nodes.
2. **Establishing a connection** between those node representations and the specific target task.

- **Information Diffusion (Guille et al., 2013):**
  - An important phenomenon in social networks.
  - **Bourigault et al. (2014):** Proposed learning node representations in a latent space where a diffusion kernel explains cascades in the training set. It maps observed discrete diffusion processes into continuous heat diffusion processes.
  - *Cascade Prediction:* Predicting the growth/increment of cascade size after a designated time interval.
  - **Li et al. (2017a):** Argued that prior cascade prediction models relied too heavily on hand-crafted features. They proposed an end-to-end deep learning model using cascade graph embedding.

- **Anomaly Detection (Akoglu et al., 2015):**
  - Aims to identify structural inconsistencies, such as anomalous nodes connecting to various diverse, influential communities.
  - **Hu et al. (2016) / Burt (2004):** Developed a graph embedding-based anomaly detection method. Under the assumption that linked nodes should share similar community memberships, anomalous nodes are identified if they link to several different, unrelated communities. By capturing node-community correlations, they evaluate an anomaly score; higher values indicate a higher propensity of being an anomaly.
- **Graph Alignment:**
  - The goal is to establish correspondences (predict anchor links) between nodes across two different graphs (e.g., mapping user accounts across different social media platforms).
  - **Man et al. (2016):** Proposed a graph representation learning algorithm that solves the alignment task by preserving the individual graph structures while respecting the observed anchor links.

Conclusion: Incorporating advanced information and domain-specific knowledge allows for the development of highly effective end-to-end network application solutions. This approach bypasses the performance limitations of hand-crafted network features (such as centrality measures) and opens up significant potential for future research.

2.4 Graph Neural Networks

16 July 2026 16:21

2.4 Graph Neural Networks

While deep learning has achieved massive success in grid-structured domains such as acoustics, image processing, and natural language processing (NLP), adapting these techniques to graph-structured data is highly non-trivial.

Key Challenges in Graph Deep Learning

1. **Irregular Structures of Graphs**
   Unlike images, audio, or text, graphs do not possess a regular grid structure. This makes it difficult to generalize fundamental mathematical operations—such as **convolution** and **pooling** (which are basic components of Convolutional Neural Networks)—directly to graph data.
2. **Heterogeneity and Diversity**
   Graphs can be highly complex, consisting of diverse types of nodes, edges, and properties. Different graph tasks and properties require customized model architectures rather than a one-size-fits-all approach.
3. **Large-Scale Graphs**
   Modern real-world graphs frequently scale to millions or billions of nodes and edges. Designing scalable models that maintain **linear time complexity** (i.e., $\mathcal{O}(|V| + |E|)$ relative to graph size) is a critical engineering bottleneck.
4. **Incorporating Interdisciplinary Knowledge**
   Graphs often model relationships in fields like biology, chemistry, and social sciences. While incorporating domain-specific knowledge is highly beneficial for solving targeted problems, doing so greatly complicates model design and integration.

Graph Neural Network (GNN) Taxonomies & Architectures

The training strategies and architectures of GNNs vary widely (ranging from supervised to unsupervised, and convolutional to recursive). The primary variants include:

- **Graph Recurrent Neural Networks (Graph RNNs):** Capture sequential and recursive graph patterns by modeling states at either the node-level or graph-level.
- **Graph Convolutional Networks (GCNs):** Redefine convolution and readout operations for irregular graph topologies to capture both local and global structural patterns.
- **Graph Autoencoders (GAEs):** Unsupervised frameworks that assume low-rank graph structures to learn effective node representation.
- **Graph Reinforcement Learning (Graph RL):** Formulate graph tasks using actions and rewards to obtain optimization feedback while satisfying specified constraints.
- **Graph Adversarial Methods:** Leverage adversarial training techniques to enhance the overall generalization of graph models and evaluate their resilience against adversarial attacks.

Future and Ongoing Research Directions

- Developing new models tailored for unstudied, unique graph structures.
- Exploring the **compositionality** of existing graph models.
- Handling **dynamic graphs** that evolve over time.
- Improving the **interpretability** and **robustness** of GNN predictions.

2.5 Summary

This chapter outlines the foundational paradigm and progression of learning representations on graphs:

- **The Core Foundation:** Learning representations that preserve both graph structure and intrinsic properties. Failing to preserve these characteristics in the embedding space leads to severe information loss, which degrades the accuracy of downstream analytical tasks.
- **Downstream Flexibility:** Once structures and properties are successfully preserved in the representation space, off-the-shelf machine learning algorithms can be easily applied.
- **Advanced Integration:** When auxiliary information (such as side features or domain-specific insights) is available, it can be integrated directly into the representation learning pipeline.
- **The Path Forward:** Deep learning on graphs stands as a pivotal building block for relational data modeling, offering a vital pathway toward more robust, capable machine learning and artificial intelligence systems.

T2-Chapter 3 Neighborhood Reconstruction Methods

09 July 2026 09:57

Chapter 3: Neighborhood Reconstruction Methods

This chapter focuses on methods for learning **node embeddings** in simple and weighted graphs. It establishes the foundational concepts of mapping graph structures into a low-dimensional space.

Core Goal of Node Embedding

The objective is to encode graph nodes into low-dimensional vectors that effectively summarize two key aspects:

- **Graph Position:** The node's relative location within the overall network.
- **Local Neighborhood Structure:** The connectivity and relationship details of the graph immediately surrounding the node.

In practice, this means projecting nodes into a continuous **latent space** (or embedding space). The geometric relations (such as distance or vector similarity) between points in this latent space are designed to mirror the actual topological relationships (such as edges or path proximities) in the original graph.

The Encoding Framework

The process relies on an encoder function to bridge the discrete graph structure and the continuous vector space:

- **The Encoder (ENC):** A mapping function that takes a node from the original network and outputs a low-dimensional vector.

$$\mathrm{ENC}(u) = \mathbf{z}_u$$

$$\mathrm{ENC}(v) = \mathbf{z}_v$$

- **Optimization Objective:** The learned embeddings ($\mathbf{z}_u, \mathbf{z}_v$) are optimized so that their geometric distances or similarities in the embedding space directly reflect the relative structural positions and proximity of the nodes u and v in the original graph.

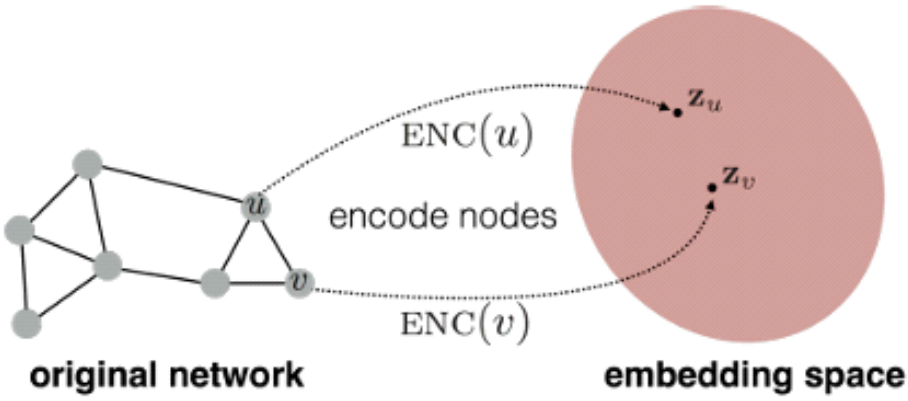


Figure 3.1: Illustration of the node embedding problem. Our goal is to learn an encoder (ENC), which maps nodes to a low-dimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect the relative positions of the nodes in the original graph.

Structural Overview of Embedding Types

- **Simple and Weighted Graphs:** Covered in this chapter (Chapter 3), focusing on neighborhood reconstruction techniques.
- **Multi-Relational Graphs:** Analogous embedding approaches tailored for networks with multiple types of edges/relations are detailed in Chapter 4.

3.1 An Encoder-Decoder Perspective

09 July 2026 10:00

Chapter 3: Neighborhood Reconstruction Methods

3.1 An Encoder-Decoder Perspective

The framework of encoding and decoding graphs organizes the discussion of node embeddings. This perspective breaks the graph representation learning problem into two key operations:

1. **Encoder:** Maps each node in the graph into a low-dimensional vector or embedding.
2. **Decoder:** Takes the low-dimensional node embeddings and uses them to reconstruct structural information about each node's neighborhood in the original graph.

3.1.1 The Encoder

Formally, the encoder is a function that maps nodes $v \in \mathcal{V}$ to vector embeddings $\mathbf{z}_v \in \mathbb{R}^d$, where $\mathbf{z}_v$ corresponds to the embedding for node v. In its simplest form, the encoder has the following signature:

$$\text{ENC} : \mathcal{V} \to \mathbb{R}^d$$

This signature means the encoder takes node IDs as input to generate the node embeddings.

Shallow Embeddings vs. Generalized Encoders

- **Shallow Embedding:** The encoder function is simply an embedding lookup based on the node ID. It is represented mathematically as:

  $$\text{ENC}(v) = \mathbf{Z}[v]$$

  where $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is a matrix containing the embedding vectors for all nodes, and $\mathbf{Z}[v]$ denotes the row of $\mathbf{Z}$ corresponding to node v.
- **Generalized Encoders:** The encoder can extend beyond shallow lookups to utilize node features or local graph structures around each node. These advanced architectures—often called **Graph Neural Networks (GNNs)**—form the core of deeper graph representation learning.

3.1.2 The Decoder

The role of the decoder is to reconstruct certain graph statistics from the node embeddings generated by the encoder. For example, given an embedding $\mathbf{z}_u$ of a node u, the decoder might attempt to predict u's set of neighbors $\mathcal{N}(u)$ or its row $\mathbf{A}[u]$ in the graph adjacency matrix.

Pairwise Decoders

The standard practice is to define pairwise decoders, which predict the relationship or similarity between pairs of nodes:

$$\text{DEC} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}^+$$

Applying the pairwise decoder to a pair of embeddings $(\mathbf{z}_u, \mathbf{z}_v)$ results in the reconstruction of the relationship between nodes u and v. The goal is to optimize the encoder and decoder to minimize reconstruction loss so that:

$$\text{DEC}(\text{ENC}(u), \text{ENC}(v)) = \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v]$$

Here, $\mathbf{S}[u, v]$ is a graph-based similarity measure between nodes.

- A simple reconstruction objective might predict whether two nodes are neighbors, corresponding to $\mathbf{S}[u, v] \triangleq \mathbf{A}[u, v]$.
- Alternatively, $\mathbf{S}[u, v]$ can be defined more generally by leveraging pairwise neighborhood overlap statistics.

3.1.3 Optimizing an Encoder-Decoder Model

To achieve the reconstruction objective, the standard practice is to minimize an empirical reconstruction loss $\mathcal{L}$ over a set of training node pairs $\mathcal{D}$:

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} \ell\left(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v), \mathbf{S}[u, v]\right)$$

Where $\ell : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is a loss function measuring the discrepancy between the decoded (estimated) similarity values $\text{DEC}(\mathbf{z}_u, \mathbf{z}_v)$ and the true values $\mathbf{S}[u, v]$.

- **Loss Functions:** Depending on the decoder and similarity function, ℓ might be a mean-squared error or a classification loss (such as cross-entropy).
- **Optimization Techniques:** The overall objective is trained using stochastic gradient descent (SGD). However, certain specialized cases (e.g., based on matrix factorization) can be solved using exact algebraic methods.

3.1.4 Overview of the Encoder-Decoder Approach

The encoder-decoder framework allows us to succinctly define and compare different embedding methods based on three components:

1. Their **decoder** function
2. Their **graph-based similarity** measure
3. Their **loss** function

The table below summarizes well-known shallow embedding algorithms within this framework:

Method	Decoder	Similarity Measure	Loss Function
Laplacian Eigenmaps	$\|\mathbf{z}_u - \mathbf{z}_v\|_2$	General	$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot S[u, v]$
Graph Factorization	$\mathbf{z}_u^\top \mathbf{z}_v$	$A[u, v]$	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - S[u, v]\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$A[u, v], \ldots, A^k[u, v]$	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - S[u, v]\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	General	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - S[u, v]\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_G(v \mid u)$	$-S[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_G(v \mid u)$ (biased)	$-S[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

Note on Random-Walk Methods: For random-walk-based methods (DeepWalk and node2vec), the similarity functions are asymmetric. The notation $p_g(v|u)$ corresponds to the probability of visiting node v on a fixed-length random walk starting from node u. These approaches are conceptually motivated by inspirations from natural language processing but share close theoretical ties to spectral graph theory.

from Hamilton et al. [2017a].

Method	Decoder	Similarity measure	Loss function
Lap. Eigenmaps	$\|\mathbf{z}_u - \mathbf{z}_v\|_2^2$	general	$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$
Graph Fact.	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v]$	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\|_2^2$
GraRep	$\mathbf{z}_u^\top \mathbf{z}_v$	$\mathbf{A}[u, v], ..., \mathbf{A}^k[u, v]$	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\|_2^2$
HOPE	$\mathbf{z}_u^\top \mathbf{z}_v$	general	$\|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\|_2^2$
DeepWalk	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v\|u)$	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$
node2vec	$\frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{k \in V} e^{\mathbf{z}_u^\top \mathbf{z}_k}}$	$p_{\mathcal{G}}(v\|u)$ (biased)	$-\mathbf{S}[u, v] \log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v))$

3.2 Factorization-based approaches

09 July 2026 10:08

3.2 Factorization-based approaches

One way of viewing the encoder-decoder idea is from the perspective of **matrix factorization**.

- **The Core Challenge:** Decoding local neighborhood structure from a node's embedding is closely related to reconstructing entries in the graph adjacency matrix.
- **Generalization:** More generally, this task can be viewed as using matrix factorization to learn a low-dimensional approximation of a node-node similarity matrix $\mathbf{S}$.
- **Similarity Matrix ($\mathbf{S}$):** This matrix generalizes the adjacency matrix and captures a user-defined notion of node-node similarity.

Laplacian eigenmaps

One of the earliest and most influential factorization-based approaches is the **Laplacian eigenmaps (LE)** technique, which builds upon spectral clustering ideas.

In this approach, the decoder is defined based on the L_2**-distance** between the node embeddings:

$$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \|\mathbf{z}_u - \mathbf{z}_v\|_2^2$$

The loss function then weighs pairs of nodes according to their similarity in the graph:

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \cdot \mathbf{S}[u, v]$$

Intuition: This loss function penalizes the model when very similar nodes have embeddings that are far apart.

Mathematical Optimization

If $\mathbf{S}$ is constructed so that it satisfies the properties of a Laplacian matrix, then the node embeddings that minimize the loss in the equation above are identical to the solution for spectral clustering.

In particular, if we assume the embeddings $\mathbf{z}_u$ are d-dimensional, then the optimal solution that minimizes the loss function is given by the d **smallest eigenvectors** of the Laplacian (excluding the eigenvector of all ones).

Inner-product methods

Following the Laplacian eigenmaps technique, more recent work generally employs an **inner-product based decoder**, defined as follows:

$$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \mathbf{z}_u^\top \mathbf{z}_v$$

Here, it is assumed that the similarity between two nodes—e.g., the overlap between their local neighborhoods—is proportional to the **dot product** of their embeddings.

Algorithmic Examples

Some examples of node embedding algorithms using this style include:

- **Graph Factorization (GF)** approach
- **GraRep**
- **HOPE**

All three of these methods combine the inner-product decoder with the following **mean-squared error** loss function:

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} \|\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) - \mathbf{S}[u, v]\|_2^2$$

Key Differences

They differ primarily in how they define $\mathbf{S}[u, v]$ (the notion of node-node similarity or neighborhood overlap that they use):

- **GF Approach:** Directly uses the adjacency matrix and sets $\mathbf{S} \triangleq \mathbf{A}$.
- **GraRep:** Defines $\mathbf{S}$ based on powers of the adjacency matrix.
- **HOPE:** Supports general neighborhood overlap measures (e.g., any neighborhood overlap measure).

Matrix-Factorization Equivalence

These methods are referred to as matrix-factorization approaches because their loss functions can be minimized using classic factorization algorithms, such as **singular value decomposition (SVD)**.

By stacking the node embeddings $\mathbf{z}_u \in \mathbb{R}^d$ into a matrix $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}| \times d}$, the reconstruction objective for these approaches can be written globally as:

$$\mathcal{L} \approx \|\mathbf{Z}\mathbf{Z}^\top - \mathbf{S}\|_2^2$$

Summary Intuition: The goal of these methods is to learn embeddings for each node such that the inner product between the learned embedding vectors approximates some deterministic measure of node similarity ($\mathbf{S}$).

3.3 Random walk embeddings

09 July 2026 13:45

3.3 Random walk embeddings

Traditional inner-product methods use *deterministic* measures of node similarity (e.g., defining similarity as a polynomial function of the adjacency matrix where $\mathbf{z}_u^\top \mathbf{z}_v \approx \mathbf{S}[u, v]$). Recent methods have shifted to using *stochastic* measures of neighborhood overlap.

- **Key Innovation:** Node embeddings are optimized so that two nodes have similar embeddings if they tend to co-occur on short random walks over the graph.

DeepWalk and node2vec

Both DeepWalk and node2vec utilize a shallow embedding approach and an inner-product decoder. Unlike earlier methods that reconstruct the adjacency matrix $\mathbf{A}$ directly, these approaches optimize embeddings to encode the statistics of random walks.

- **Objective:** Learn embeddings such that the following stochastic and asymmetric similarity measure holds:

$$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \triangleq \frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{v_k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}} \approx p_{\mathcal{G},T}(v|u) \quad (3.10)$$

  - $p_{\mathcal{G},T}(v|u)$: The probability of visiting node v on a length-T random walk starting at node u.
  - T: Typically in the range $T \in \{2, \ldots, 10\}$.
- **Training Strategy:** Use the decoder from Equation (3.10) and minimize the cross-entropy loss over a training set of random walks $\mathcal{D}$:

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} -\log(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v)) \quad (3.11)$$

- **Computational Challenges & Solutions:** Naively evaluating Equation (3.11) is computationally expensive because the denominator in (3.10) has a time complexity of $O(|\mathcal{V}|)$, making the overall loss function $O(|\mathcal{D}||\mathcal{V}|)$. The primary difference between DeepWalk and node2vec is how they approximate this:
  1. **DeepWalk:** Uses a *hierarchical softmax* approach, leveraging a binary-tree structure to accelerate computation.
  2. **node2vec:** Employs a *noise contrastive approach* (negative sampling) to approximate the normalizing factor:

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} -\log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \gamma \mathbb{E}_{v_n \sim P_n(\mathcal{V})}[\log(\sigma(-\mathbf{z}_u^\top \mathbf{z}_{v_n}))] \quad (3.12)$$

     - σ: Logistic function.
     - $P_n(\mathcal{V})$: A distribution over nodes (often uniform) used to draw negative samples.
     - $\gamma > 0$: Hyperparameter.
     - The expectation is approximated using Monte Carlo sampling.

- **Flexibility in node2vec:** Unlike DeepWalk (which uses uniform random walks), node2vec introduces hyperparameters that allow random walk probabilities to smoothly interpolate between breadth-first search (BFS) and depth-first search (DFS).

Large-scale information network embeddings (LINE)

LINE shares conceptual motivations with DeepWalk and node2vec but does *not* explicitly sample random walks. Instead, it combines two encoder-decoder objectives to explicitly reconstruct neighborhood information.

- **First Objective (First-order adjacency):** Encodes direct connections using the following decoder:

$$\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \frac{1}{1 + e^{-\mathbf{z}_u^\top \mathbf{z}_v}} \quad (3.13)$$

  This uses an adjacency-based similarity measure where $\mathbf{S}[u, v] = \mathbf{A}[u, v]$.

- **Second Objective (Second-order adjacency):** Encodes two-hop adjacency information (found in $\mathbf{A}^2$). It uses the same probabilistic decoder as Equation (3.10) but is trained using KL-divergence instead of random walk sampling.

Additional variants of the random-walk idea

The random walk approach is highly extensible by biasing or modifying how the walks are generated:

- **Skipping Nodes:** Perozzi et al. (2016) proposed random walks that "skip" over nodes, generating a similarity measure similar to the GraRep algorithm.
- **Structural Relationships:** Ribeiro et al. (2017) defined random walks based on structural relationships rather than basic neighborhood proximity. This generates node embeddings that specifically encode structural roles within the graph.

3.3.1 Random walk methods and matrix factorization

3.3.1 Random walk methods and matrix factorization

Recent research (Qiu et al., 2018) demonstrates a close mathematical relationship between random walk methods and matrix factorization approaches.

- **Node-Node Similarity Matrix:**
  We can define a specific matrix of node-node similarity values for DeepWalk, denoted as $\mathbf{S}_{\mathrm{DW}}$:

$$\mathbf{S}_{\mathrm{DW}} = \log\left(\frac{\mathrm{vol}(\mathcal{V})}{T}\left(\sum_{t=1}^{T}\mathbf{P}^{t}\right)\mathbf{D}^{-1}\right) - \log(b) \quad (3.14)$$

  - b: A constant.
  - $\mathbf{P}$: The transition matrix, defined as $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$.
- **The DeepWalk Connection:**
  Given the similarity matrix defined above, the embeddings ($\mathbf{Z}$) learned by the DeepWalk algorithm actually satisfy the following approximation:

$$\mathbf{Z}^\top \mathbf{Z} \approx \mathbf{S}_{\mathrm{DW}} \quad (3.15)$$

- **Spectral Decomposition:**
  The interior summation part of Equation (3.14) can be further decomposed to reveal deeper connections:

$$\left(\sum_{t=1}^{T} \mathbf{P}^t\right) \mathbf{D}^{-1} = \mathbf{D}^{-\frac{1}{2}} \left(\mathbf{U} \left(\sum_{t=1}^{T} \mathbf{\Lambda}^t\right) \mathbf{U}^\top\right) \mathbf{D}^{-\frac{1}{2}} \quad (3.16)$$

  - $\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top = \mathbf{L}_{\mathrm{sym}}$: This represents the eigendecomposition of the symmetric normalized Laplacian matrix.

- **Key Takeaways and Insights:**
  - **Relation to Spectral Clustering:** Equation (3.16) reveals that embeddings learned by DeepWalk are fundamentally related to spectral clustering embeddings.
  - **The Role of T:** The primary difference between DeepWalk and standard spectral clustering is that DeepWalk uses the length of the random walk (T) to control the influence of different eigenvalues.
  - **Extensions to node2vec:** Similar mathematical connections to matrix factorization have also been derived for the node2vec algorithm, inspiring new factorization-based approaches based on these insights.

3.4 Limitations of Shallow Embeddings

10 July 2026 10:27

3.4 Limitations of Shallow Embeddings

In shallow embedding methods, the encoder model that maps graph nodes to vector embeddings is simply an **embedding lookup**. This process trains a unique, dedicated embedding vector for every individual node in the graph. While this approach has achieved significant success, it suffers from three major drawbacks:

1. Lack of Parameter Sharing

- **The Issue:** Shallow embedding methods do not share any parameters between nodes in the encoder. The encoder directly optimizes a unique embedding vector for each individual node.
- **Statistical Inefficiency:** Parameter sharing normally improves learning efficiency and acts as a powerful form of regularization; shallow embeddings miss out on these benefits.
- **Computational Inefficiency:** Because parameters are not shared, the total number of parameters grows as $O(|V|)$, where $|V|$ is the number of nodes in the graph. This linear scaling becomes completely intractable when dealing with massive, real-world graphs.

2. Failure to Leverage Node Features

- Many real-world graph datasets possess rich, informative feature information associated with individual nodes.
- Shallow embedding approaches are fundamentally unable to leverage or incorporate these node features into the encoding process, discarding potentially valuable data.

3. Inherently Transductive Nature

- **The Restriction:** Shallow embedding methods are strictly **transductive**, meaning they can only generate embeddings for nodes that were explicitly present in the graph during the training phase.
- **The Limitation:** They cannot generate embeddings for new, unseen nodes observed after the training phase unless additional optimization steps are executed specifically to learn them.
- **The Consequence:** This restriction prevents shallow embedding methods from being applied to **inductive** applications, which require the model to generalize dynamically to entirely unseen nodes or graphs after training.

Moving Forward: Graph Neural Networks (GNNs)

To alleviate these key limitations, shallow encoders can be replaced with more sophisticated encoders that depend more broadly on both the overall **structure** and the **attributes (features)** of the graph. The most popular paradigm defining these advanced encoders is **Graph Neural Networks (GNNs)**.